AI-POWERED COGNITIVE SCREENING & DEMENTIA RISK ANALYSIS PLATFORM

(Speech-Based Dementia Screening and AI Clinical Assistant Implemented in a Multimodal Cognitive Assessment Platform)

25-26J-439
Project Final Thesis

Natheer H A H - IT22888952

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information
Technology Sri Lanka

May 2026

AI-POWERED COGNITIVE SCREENING & DEMENTIA RISK ANALYSIS PLATFORM

(Speech-Based Dementia Screening and AI Clinical Assistant Implemented in a Multimodal Cognitive Assessment Platform)

25-26J-439
Project Final Thesis

Natheer H A H - IT22888952

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

May 2026

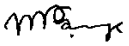
DECLARATION

I declare that this is my own work, and this Thesis does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my Thesis, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Natheer H A H	IT22888952	

The above candidate has carried out this research thesis for the Degree of Bachelor of Science (honors) Information Technology (Specializing in Software Engineering) under my supervision.



Signature of the supervisor

Date

(Ms. Manori Gamage)

Signature of co-supervisor



Date 2026/05/12.

(Ms. Kaushalya Rajapakse)

ABSTRACT

Dementia is a progressive neurological disorder that affects memory, language, reasoning, and daily functioning, creating major challenges for patients, caregivers, and healthcare systems worldwide. Early detection of cognitive decline is essential for timely intervention and improved quality of life. However, traditional dementia screening methods often require in-person clinical assessments and specialist involvement, limiting accessibility in many settings. This research presents the development of a Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform to support early cognitive impairment detection through speech analysis and explainable clinical interpretation. The system combines guided speech tasks, automatic transcription, acoustic feature extraction, deep learning-based speech classification, and an AI-powered clinical explanation layer.

The speech module includes three guided tasks: Self Introduction, Morning Recall, and Picture Description, selected to evaluate verbal fluency, memory retrieval, lexical access, and narrative coherence. Audio recordings are processed using a fine-tuned Wav2Vec2-XLSR-300M model, a transformer-based multilingual speech representation model capable of identifying dementia-related speech patterns such as hesitation, fluency breakdown, and speech rhythm irregularities. To improve clinical usability, an AI Clinical Assistant was integrated using a Gemini-powered conversational framework. This module converts model outputs into clinician-friendly explanations and recommendation guidance while maintaining non-diagnostic safety constraints. Bilingual UI support and usability improvements were also introduced to enhance accessibility.

The system uses a multi-service architecture with React.js, Node.js/Express, FastAPI, MongoDB, and Docker-based deployment. The proposed solution demonstrates that speech-based digital cognitive screening can provide a scalable, interpretable, and clinically supportive platform for early dementia risk assessment.

Keywords: Dementia Screening, Wav2Vec2-XLSR-300M, Speech Analysis, Cognitive Assessment, AI Clinical Assistant, Machine Learning, Explainable AI.

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our **supervisor, Ms. Manori Gamage**, and **co-supervisor, Ms. Kaushalya Rajapakse**, for their continuous guidance, support, and encouragement throughout the development of this project. Their insightful feedback, patience, and dedication were instrumental in helping us shape and refine our ideas from inception to completion. We are also sincerely thankful to our **external supervisor, Dr. Malsha Gunathilaka** for her valuable input, suggestions, and support, which added great value to the progress and success of this project.

Furthermore, we extend our appreciation to the lecturers, assistant lecturers, instructors, and academic and non-academic staff of **SLIIT** for their support and assistance throughout the module. We are also grateful to our group members for their collaboration and teamwork. Lastly, we thank our families and friends for their unwavering support, motivation, and understanding, which helped us persevere through challenges and stay committed to completing this project.

TABLE OF CONTENTS

DECLARATION ..... **iii**

ABSTRACT ..... **iv**

ACKNOWLEDGEMENT ..... **iv**

TABLE OF CONTENTS ..... **vi**

LIST OF FIGURES **8**

LIST OF TABLES **10**

LIST OF ABBREVIATIONS **11**

1 INTRODUCTION **12**

1.1 Background Study and Literature Review **12**

1.1.1 Background Study **12**

1.1.2 Literature Review **13**

1.2 Research Gap **15**

1.3 Research Problem **18**

1.4 Research Objectives **19**

1.4.1 Main Objective **19**

1.4.2 Specific Objectives **19**

1.4.3 Business Objectives **20**

2 METHODOLOGY **21**

2.1 Methodology **21**

2.1.1 Feasibility Study/ Planning **23**

2.1.2 Requirement Gathering & Analysis **28**

2.1.3 Designing **34**

2.1.4 Implementation 38

2.1.5 Testing 59

2.1.6 Deployment & Maintenance 74

2.2 Commercialization 81

3. RESULTS & DISCUSSION 86

4. FUTURE SCOPE 91

5. CONCLUSION 93

REFERENCES 94

APPENDICES 95

LIST OF FIGURES

Figure 1.1: Dementia Overview..... 12

Figure 1.2: Competitive Analysis 16

Figure 1.3: MoCA Digital Cognitive Screening Tool 17

Figure 1.4: PHQ-Based Online Assessment Tool 17

Figure 2.1: Agile Scrum Workflow 22

Figure 2.2: Requirement Collection Process 29

Figure 2.3: User Stories for Requirements 31

Figure 2.4: Use Case Diagram 31

Figure 2.5: Sequence Diagram 32

Figure 2.6: System Architecture Diagram 35

Figure 2.7: Speech-Based Screening Flow 37

Figure 2.8: User Flow Diagram 38

Figure 2.9: Work Breakdown Structure 39

Figure 2.10: Fine-Tuning the Wav2Vec2-XLSR-300M Model ..... .47

Figure 2.11: Backend Agent Routes Implementation 51

Figure 2.12: Audio Validation Logic 52

Figure 2.13: ML Client Integration 53

Figure 2.14: AI Clinical Assistant Integration 53

Figure 2.15: Speech Fusion Service Implementation 55

Figure 2.16: FastAPI ML Service Implementation 56

Figure 2.17: Classification report of the speech model.....57

Figure 2.18: Confusion Matrix of the Model.....58

Figure 2.19: Training and Validation Loss.....58

Figure 2.20: Frontend Unit Testing 61

Figure 2.21: Backend API Unit Testing 62

Figure 2.22: ML Service Unit Testing 63

Figure 2.23: Speech-to-Text Evaluation Script 64

Figure 2.24: Backend Integration Testing 66

Figure 2.25: ML Service Integration Testing 67

Figure 2.26: Final Output Score Display 69

Figure 2.27: Speech Assessment Starting Interface 71

Figure 2.28: Speech Assessment Instructions Interface 71

Figure 2.29: Speech Self-Introduction Interface 72

Figure 2.30: Speech Morning Recall Interface 72

Figure 2.31: Speech Picture Description Interface 73

Figure 2.32: Final Speech Assessment Report 73

Figure 2.33: Secure SSH Connection to the Ubuntu Cloud Server 78

Figure 2.34: Docker Image Build Process for the Speech ML Service 78

Figure 2.35: Execution of the Speech ML Service Container Using Docker 79

Figure 2.36: Azure Container App Overview for the Deployed ML Service 79

Figure 2.37: Docker Image Configuration in Azure Container Apps 79

Figure 2.38: Swagger API Documentation of the CogniCare ML Service 80

Figure 2.39: Successful Audio-Based Dementia Risk Prediction Response 80

Figure 3.1: Speech Prediction Result Output 87

Figure 3.2: Final Speech Screening Session Result 90

LIST OF TABLES

Table 2.1: Risk Management Plan 25

Table 2.2: Communication Management Plan 27

Table 3.1: Performance Metrics of Speech Models 86

Table 3.2: Speech-to-Text Evaluation using Word Error Rate 88

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACE-III	Addenbrooke's Cognitive Examination III
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CNN	Convolutional Neural Network
CI/CD	Continuous Integration / Continuous Deployment
DSM-5	Diagnostic and Statistical Manual of Mental Disorders (Fifth Edition)
FFmpeg	Fast Forward Moving Picture Experts Group
F1 Score	Harmonic Mean of Precision and Recall
IADL	Instrumental Activities of Daily Living
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
MRI	Magnetic Resonance Imaging
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
React.js	JavaScript Frontend Library
RMS	Root Mean Square
STT	Speech-to-Text
UI	User Interface
UX	User Experience
Wav2Vec2-XLSR-300M	Cross-Lingual Speech Representation Model (300 Million Parameters)
ZCR	Zero Crossing Rate

1 INTRODUCTION

1.1 Background Study and Literature Review

1.1.1 Background Study

Dementia is a progressive neurological disorder that affects memory, language, reasoning, decision-making, and the ability to perform daily activities independently. It is one of the leading causes of disability and dependency among older adults worldwide and has become a major global public health concern. According to the World Health Organization (WHO), more than 55 million people worldwide are currently living with dementia, with nearly 10 million new cases reported each year. As global life expectancy continues to rise, the number of dementia patients is expected to increase significantly, creating substantial pressure on healthcare systems, caregivers, and families.

Early identification of cognitive decline is critical because timely clinical intervention can delay disease progression, improve patient management, and enhance the quality of life of affected individuals. However, traditional dementia diagnosis often depends on face-to-face clinical assessments, cognitive screening tests, behavioral observations, and neuroimaging evaluations conducted by trained specialists. Common tools such as Addenbrooke's Cognitive Examination III (ACE-III), Mini-Mental State Examination (MMSE), and Lawton Instrumental Activities of Daily Living (IADL) provide valuable insights into memory, language, executive functioning, and daily independence. Although these methods are clinically reliable, they are often time-consuming, expensive, and difficult to scale for large populations, particularly in low-resource settings.



Figure 1.1: Dementia overview

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) have created new opportunities for developing automated cognitive screening systems capable of supporting early dementia detection. Behavioral biomarkers such as speech patterns, handwriting dynamics, daily activity levels, and emotional expressions have shown strong potential for identifying subtle cognitive impairment before severe symptoms become clinically visible. Among these, speech analysis has emerged as one of the most promising non-invasive screening methods because speech production directly reflects memory retrieval, lexical access, executive control, verbal fluency, and narrative coherence.

Changes in speech characteristics such as hesitation, reduced fluency, repetition, lexical retrieval difficulty, slower speech rate, and narrative disorganization are often early indicators of mild cognitive impairment and dementia. Unlike traditional assessments, speech-based screening can be performed remotely, repeatedly, and with minimal clinical supervision, making it highly suitable for scalable early detection systems. Transformer-based deep learning models such as Wav2Vec2-XLSR-300M have further improved the capability of automatic speech analysis by learning deep acoustic representations directly from raw audio without requiring extensive handcrafted feature engineering.

In addition to prediction accuracy, explainability remains a critical requirement in healthcare AI systems. Clinicians require not only a dementia risk score but also understandable reasoning behind the prediction. Therefore, integrating an AI Clinical Assistant capable of translating technical model outputs into clinician-friendly explanations significantly improves usability and trust in AI-assisted healthcare systems. Conversational AI models such as Google Gemini enable structured interpretation of screening results, confidence levels, speech domain analysis, and recommendation guidance while maintaining safe non-diagnostic boundaries.

This research introduces the Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform, a multimodal cognitive assessment system designed to support early dementia risk screening. The system combines guided speech task assessment, fine-tuned Wav2Vec2-XLSR-300M speech classification, feature extraction, explainable AI interpretation, bilingual accessibility, and multi-service deployment architecture to provide a practical and scalable clinical decision-support platform for early cognitive impairment detection.

1.1.2 Literature Review

Speech-based dementia screening has become an important research area within Artificial Intelligence (AI), Machine Learning (ML), and speech processing because language and speech changes often appear during the early stages of cognitive decline. Dementia can affect verbal fluency, memory retrieval, lexical access, sentence formation, and narrative organization. Therefore, automated speech analysis provides a non-invasive and scalable method for identifying early cognitive impairment indicators.

Traditional cognitive screening tools such as ACE-III, MMSE, and IADL assessments are widely used in clinical environments to evaluate memory, attention, language, executive function, and daily functioning. However, these assessments usually require trained professionals and face-to-face administration, which limits their scalability. Digital cognitive screening systems have been introduced to reduce manual scoring errors and improve consistency. For example, computerized versions of cognitive tests have shown that digital platforms can support standardized assessment and improve clinical efficiency [1].

Several studies have explored speech and language features for dementia detection. Speech production is closely connected with cognitive processes such as attention, semantic memory, working memory, and executive control. As a result, individuals with cognitive impairment may show hesitation, slower speech rate, repeated words, reduced vocabulary, word-finding difficulty, and poor narrative coherence. Previous research has demonstrated that automated speech analysis can identify dementia-related markers from spontaneous speech and guided speech tasks [2], [3].

Guided speech tasks are commonly used because they allow the system to evaluate specific cognitive domains. Self-introduction tasks can capture general verbal fluency and spontaneous speech ability. Morning recall tasks can evaluate episodic memory and sequential recall. Picture description tasks can assess narrative structure, attention, language organization, and semantic expression. These tasks provide meaningful speech samples for analyzing early signs of cognitive decline.

Deep learning has significantly improved speech-based dementia screening by enabling models to learn acoustic representations directly from raw audio. Transformer-based speech models, especially Wav2Vec2 and Wav2Vec2-XLSR, have shown strong performance in speech representation learning. Wav2Vec2-XLSR-300M is a multilingual speech model pretrained on large-scale speech data, making it suitable for cross-lingual and low-resource environments. When fine-tuned for dementia classification, it can capture acoustic patterns such as hesitation, fluency breakdown, rhythm irregularities, and other subtle speech characteristics related to cognitive decline [4].

In addition to raw audio classification, speech-based screening systems often use acoustic feature extraction. Features such as speech duration, tempo, zero-crossing rate, root mean square energy, pause-related indicators, and speech rhythm can provide useful information about fluency and motor speech control. These acoustic features support the identification of irregular speech patterns that may be associated with memory decline or neurological impairment.

Automatic Speech Recognition (ASR) has also become an important part of digital speech assessment systems. ASR tools such as Google Cloud Speech-to-Text convert recorded speech into text, enabling further linguistic analysis and supporting clinical interpretation. Although the core predictive model may use raw audio, transcription improves usability by allowing clinicians to review spoken responses and understand the context of the user's speech.

Recent healthcare AI systems also emphasize explainability and decision support. In clinical settings, model predictions alone are not sufficient because clinicians need understandable explanations behind the system output. AI Clinical Assistants powered by Large Language Models (LLMs), such as Gemini, can convert technical model outputs into clinician-friendly summaries. These assistants can explain risk levels, highlight speech-related observations, provide recommendation guidance, and support patient or caregiver communication while maintaining non-diagnostic safety boundaries.

System architecture is another important consideration in AI-based healthcare platforms. Modern digital health systems commonly use modular architectures where frontend interfaces, backend gateways, machine learning services, databases, and AI assistant services are separated. This approach improves scalability, maintainability, and deployment flexibility. Technologies such as React.js, Node.js, FastAPI, MongoDB, and Docker are commonly used to build such systems because they support web-based access, API communication, machine learning inference, and containerized deployment.

In summary, the literature supports the development of speech-based dementia screening systems that combine guided speech tasks, deep learning-based audio classification, automatic transcription, explainable AI, and modular deployment architectures. The CogniCare speech component follows this direction by using a fine-tuned Wav2Vec2-XLSR-300M model, guided cognitive speech tasks, and an AI Clinical Assistant to support early dementia risk screening in an accessible and clinically interpretable manner.

1.2 Research Gap

Speech-based dementia screening has strong potential to improve early cognitive impairment detection through scalable and non-invasive healthcare solutions. However, several research gaps remain, especially in developing clinically usable systems rather than model-focused research.

A major research gap is the lack of clinically interpretable prediction systems. Many existing studies focus mainly on classification accuracy and benchmark results but do not provide sufficient explanation for clinicians. In medical practice, a dementia risk score alone is not enough. Clinicians require understandable reasoning such as speech fluency changes, hesitation patterns, memory-related language issues, and narrative coherence. Research is needed to develop explainable AI systems that provide both prediction and clinical justification.

Another challenge is the limited use of structured guided speech tasks. Many studies rely on spontaneous speech datasets that may not reflect real clinical workflows. There is a need for standardized tasks such as self-introduction, morning recall, and picture description to evaluate verbal fluency, episodic memory, and narrative organization more consistently.

Multilingual and low-resource environments also present an important gap. Most dementia speech models are trained using English-based datasets and may not perform well in multilingual contexts like Sri Lanka. Research is needed to improve multilingual models such as Wav2Vec2-XLSR-300M for practical dementia screening in diverse speech environments.

System integration and deployment remain another major limitation. Many previous studies focus only on model performance without considering frontend usability, backend workflows, API communication, clinician dashboards, and deployment architecture. Real-world healthcare systems require complete end-to-end solutions rather than isolated prediction models.

Another significant gap is the absence of AI-assisted clinician communication. Existing systems often generate only prediction outputs without helping clinicians explain results to patients or caregivers. AI Clinical Assistants are needed to convert technical outputs into safe, clinician-friendly explanations and recommendation guidance.

Finally, accessibility and usability are often neglected. Many platforms are designed only for English-speaking users and do not provide bilingual support or simplified interfaces for elderly users. Research is needed to improve usability through localized language support and patient-friendly design.

By addressing these gaps, this project develops a Speech-Based Dementia Screening and AI Clinical Assistant module that integrates guided speech tasks, fine-tuned Wav2Vec2-XLSR-300M classification, explainable AI interpretation, bilingual accessibility, and deployment-ready architecture for practical early dementia risk screening.

Feature	MoCA / MMSE	Existing Speech Studies	CogniCare Speech Component
Remote web-based screening	Limited	Limited	Yes
Guided multi-task speech assessment	No	Limited	Yes
Acoustic + linguistic analysis	No	Yes	Yes
Automatic risk scoring	No	Yes	Yes
Explainable clinician-friendly output	No	Limited	Yes
Final screening report	Manual	Limited	Yes

Figure 1.2: Competitive Analysis

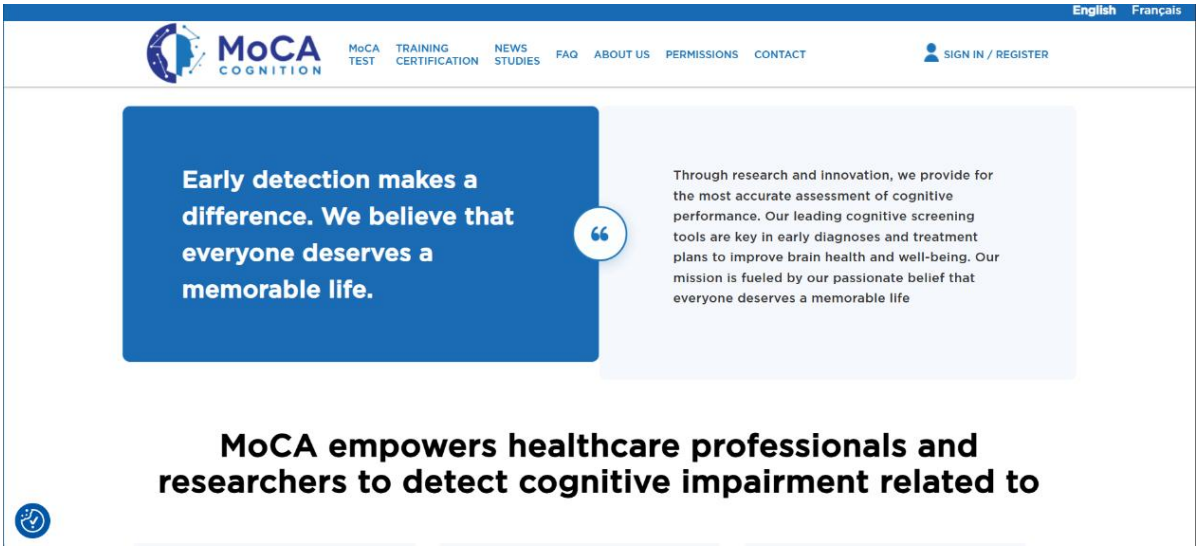


Figure 1.3: MoCA Digital Cognitive Screening Tool

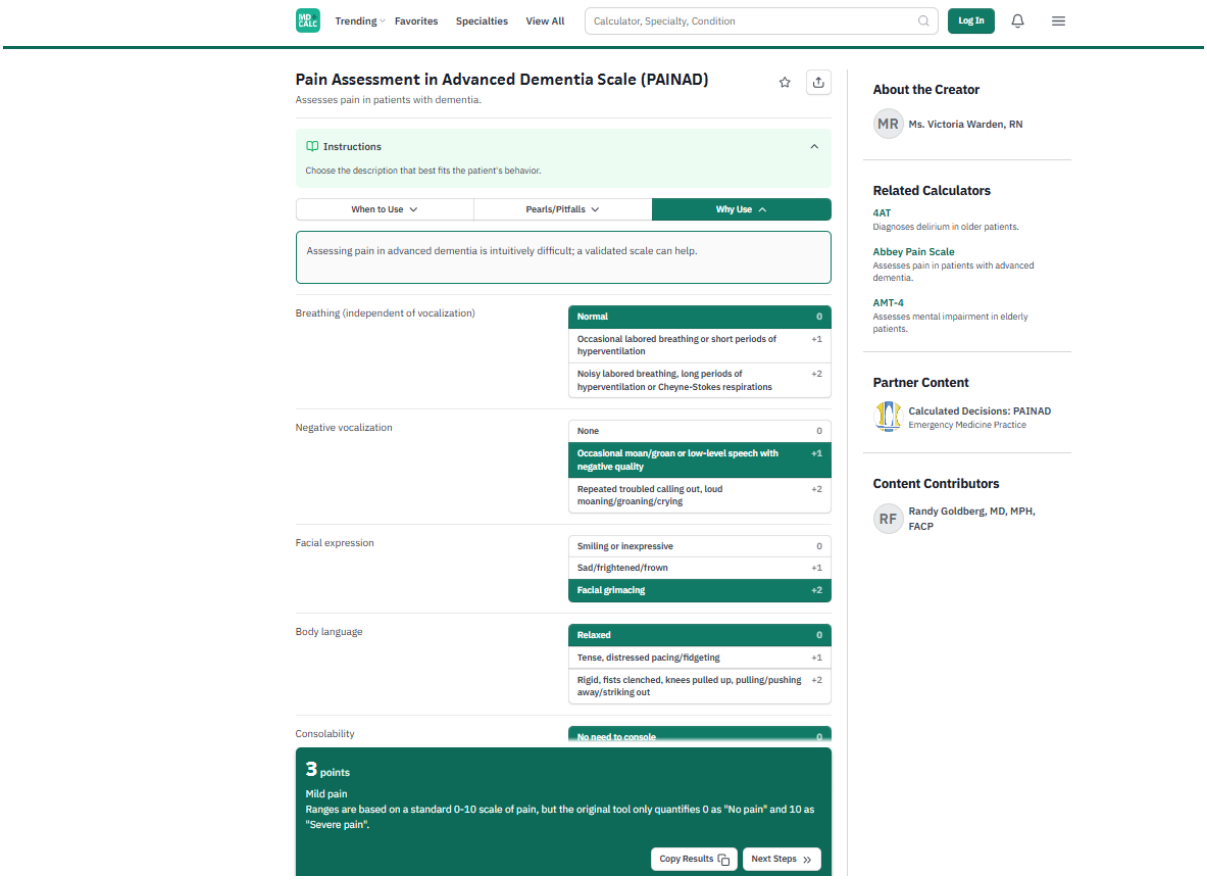


Figure 1.4: PHQ-Based Online Assessment Tool

1.3 Research Problem

The growing elderly population and the increasing prevalence of dementia have created major challenges for healthcare systems worldwide, particularly in the area of early cognitive impairment detection. Early diagnosis is essential because timely intervention can delay disease progression, improve patient management, and enhance quality of life. However, traditional dementia screening methods still depend heavily on face-to-face clinical assessments, specialist consultation, and manual cognitive evaluations, which are often expensive, time-consuming, and difficult to scale across large populations.

In many healthcare settings, especially in low-resource environments such as Sri Lanka, access to neurologists, psychiatrists, and cognitive specialists remains limited. As a result, patients are often diagnosed only after significant cognitive decline has already occurred, reducing the effectiveness of treatment and long-term care planning. Existing clinical tools such as ACE-III, MMSE, and IADL provide valuable assessments, but they require professional administration and are not designed for continuous, accessible, or remote monitoring.

Speech is recognized as one of the earliest behavioral indicators of dementia because cognitive decline directly affects verbal fluency, lexical retrieval, memory recall, sentence organization, and narrative coherence. However, current speech-based dementia detection systems are still largely research-oriented and rarely function as complete clinical support tools. Many existing models focus mainly on prediction accuracy while providing only simple classification outputs such as dementia risk scores without explaining the reasoning behind the result. This reduces clinician trust and limits real-world medical adoption.

Additionally, many available systems rely on spontaneous speech datasets or controlled laboratory recordings rather than structured guided speech tasks designed for cognitive assessment. Without standardized tasks such as self-introduction, morning recall, and picture description, it becomes difficult to evaluate specific domains like episodic memory, executive function, and verbal fluency consistently across patients.

Another major limitation is the lack of multilingual adaptability. Most speech-based dementia models are trained primarily using English datasets and may not perform effectively in multilingual environments where accents, pronunciation, and mixed-language communication are common. In countries like Sri Lanka, this creates a significant barrier to practical deployment, especially when screening diverse patient populations.

Furthermore, most existing platforms do not provide clinician-supportive explanation systems. A dementia prediction score alone is insufficient for responsible healthcare practice. Clinicians require structured interpretation, confidence indicators, recommendation guidance, and safe patient communication support. The absence of

AI-assisted explanation tools makes it difficult to integrate prediction systems into actual medical workflows.

System deployment and usability also remain weak in many research solutions. High-performing models often fail to address frontend accessibility, secure audio handling, backend workflow management, session tracking, clinician dashboards, and scalable deployment architecture. Without these practical components, even accurate models have limited clinical value.

Therefore, the main research problem addressed in this project is the absence of a scalable, explainable, multilingual, and deployment-ready speech-based dementia screening system that supports guided cognitive speech assessment, accurate deep learning-based prediction, and clinician-friendly interpretation. This research aims to solve this problem by developing a Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform using guided speech tasks, bilingual accessibility, and AI-assisted clinical explanation to improve early dementia risk screening in real-world healthcare environments.

1.4 Research Objectives

1.4.1 Main Objective

The main objective of this research is to design and develop a Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform that supports early cognitive impairment detection through guided speech assessment, deep learning-based speech classification, and explainable clinical decision support.

1.4.2 Specific Objectives

To achieve the main objective, the following specific objectives were defined:

- To design and implement guided cognitive speech tasks including Self Introduction, Morning Recall, and Picture Description for evaluating verbal fluency, episodic memory, lexical access, and narrative coherence.
- To fine-tune and deploy the Wav2Vec2-XLSR-300M transformer-based multilingual speech model for identifying dementia-related speech characteristics from raw audio recordings.
- To develop a frontend speech recording interface that supports structured task progression, bilingual interaction, user-friendly navigation, and accessible speech assessment workflows.
- To implement backend gateway APIs and ML service integration for session creation, audio upload, prediction handling, task scoring, and final speech score generation.
- To integrate an AI Clinical Assistant using Gemini-powered conversational support for converting model outputs into clinician-friendly explanations, recommendation guidance, and safe non-diagnostic interpretation.

- To ensure deployment readiness using Docker-based multi-service architecture for frontend, backend, machine learning services, and clinician support modules.
- To evaluate the effectiveness of the speech screening model using performance metrics such as accuracy, precision, recall, F1-score, and AUC in order to validate practical dementia risk prediction capability.

1.4.3 Business Objectives

- **Improve accessibility of early dementia screening services:** Provide a scalable digital pre-screening platform that reduces dependence on specialist-only clinical assessments and allows patients to undergo preliminary cognitive screening through guided speech tasks in a more accessible and convenient manner.

- **Support clinicians with faster and more efficient decision-making:** Deliver structured speech assessment results, dementia risk scores, and AI-assisted clinical explanations that help healthcare professionals perform faster preliminary evaluations and improve clinical workflow efficiency in hospitals, clinics, and elderly care centers.

- **Promote patient-centered and inclusive healthcare services:** Enhance usability through bilingual support, simplified user interfaces, and guided task-based assessments that improve accessibility for elderly users, caregivers, and multilingual populations, particularly in low-resource healthcare environments.

- **Reduce operational cost and manual workload in cognitive screening:** Minimize the time, effort, and resources required for traditional dementia assessments by automating speech recording, transcription, scoring, and interpretation processes, allowing healthcare providers to allocate resources more effectively.

- **Encourage adoption of AI-driven healthcare technologies:** Provide healthcare institutions with a modern AI-assisted cognitive screening solution that integrates deep learning, speech analysis, and clinical explanation support, helping hospitals and medical centers adopt advanced digital health technologies.

- **Establish a scalable foundation for future commercialization:** Develop a deployment-ready platform that can be extended for use in telemedicine systems, rehabilitation centers, elderly care facilities, and national healthcare screening programs, creating long-term opportunities for healthcare innovation and commercialization.

2 METHODOLOGY

2.1 Methodology

The methodology forms the foundation of this research by defining the systematic processes, technical strategies, and implementation approach used to design, develop, and evaluate the Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform. In healthcare-oriented artificial intelligence systems, a clearly structured methodology is essential to ensure reliability, reproducibility, clinical relevance, and deployment readiness. This is particularly important in projects involving speech analysis, machine learning, explainable AI, and real-time healthcare decision support.

This study adopted a hybrid methodology that combines Agile software development practices with a structured System Development Life Cycle (SDLC) approach to address the complex and iterative requirements of building a practical dementia screening platform. Since the project involves multiple interconnected components including speech recording, machine learning inference, clinical explanation, frontend usability, backend orchestration, and deployment architecture, a flexible yet systematic development model was necessary.

• **Agile Methodology for Research Development:** Agile methodology was selected as the primary development approach because of its flexibility, iterative improvement process, and strong stakeholder-driven adaptation. Unlike rigid linear development models, Agile supports continuous refinement based on testing outcomes, supervisor feedback, technical challenges, and real-world usability requirements. This was especially important for the CogniCare platform, where both healthcare relevance and technical reliability needed continuous improvement.

The Scrum framework of Agile was applied, where development work was divided into smaller sprints, each focusing on a major system component such as speech task design, Wav2Vec2-XLSR-300M model integration, AI Clinical Assistant implementation, bilingual UI improvements, or backend API optimization. Regular sprint reviews, progress evaluations, and testing cycles helped ensure continuous improvement and allowed immediate correction of technical issues during development.

Figure 2.1 illustrates the Agile Scrum workflow used in this research, showing how iterative development supported planning, implementation, testing, feedback, and refinement throughout the project lifecycle.

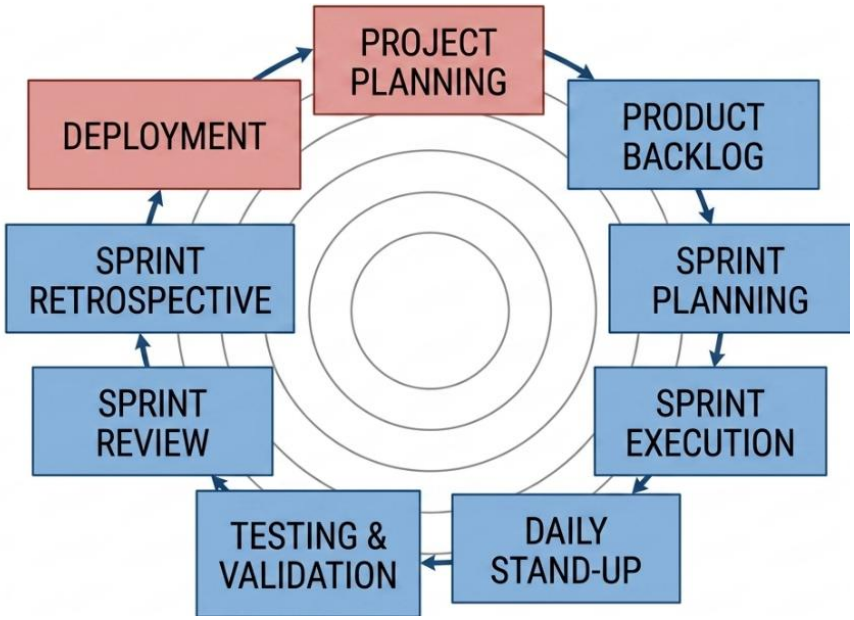


Figure 2.1: Agile scrum workflow

• **Seven-Stage Development Framework:** Based on Agile principles, the research process followed a structured seven-stage development framework consisting of feasibility study, requirement gathering, system design, implementation, testing, deployment, and maintenance planning. Each stage was connected to the next through continuous feedback and iterative refinement, allowing the system to evolve based on practical findings rather than fixed assumptions.

This framework was particularly useful because dementia screening systems require both technical accuracy and healthcare usability. For example, model selection was influenced not only by performance metrics but also by explainability, multilingual suitability, and clinical deployment feasibility. Similarly, frontend design decisions were guided by accessibility requirements for elderly users and clinicians rather than only interface aesthetics.

• **Iterative Development and Collaboration:** The successful development of the speech screening module required continuous collaboration between software development, machine learning implementation, and healthcare workflow understanding. Frequent review sessions were conducted to evaluate frontend usability, backend reliability, speech model performance, and AI explanation quality. This iterative collaboration ensured that technical decisions aligned with both academic research objectives and practical clinical requirements.

Version control and collaborative development tools such as GitHub were used for source code management, issue tracking, debugging, and deployment preparation. This iterative process proved especially valuable in improving speech task consistency, refining API communication between services, handling audio validation challenges, and improving clinician-facing interpretation outputs.

• **Flexibility and Responsiveness:** The Agile methodology provided the flexibility needed to address unexpected technical and practical challenges during development. Issues such as inconsistent audio formats, microphone permission failures, speech upload validation errors, API communication failures, multilingual UI rendering issues, and external service dependency problems required rapid adjustment during implementation.

The iterative model allowed these issues to be resolved without disrupting the full project timeline. For example, strict audio validation rules were introduced to improve processing consistency, fallback error handling was implemented for external service failures, and UI improvements were made after observing usability limitations during testing. This responsiveness ensured that the final system remained both technically stable and clinically practical.

By combining Agile methodology with a structured seven-stage SDLC framework, this research achieved a balance between flexibility and systematic development. This methodology ensured that the final Speech-Based Dementia Screening and AI Clinical Assistant module was not only academically valid but also practically deployable for real-world healthcare environments.

2.1.1 Feasibility Study/ Planning

Before implementation began, a feasibility study was conducted to evaluate whether the proposed Speech-Based Dementia Screening and AI Clinical Assistant module could be practically developed, deployed, and used within real healthcare environments. The purpose of this stage was to assess technical feasibility, operational feasibility, economic feasibility, and clinical usability before committing to full system development.

•Technical Feasibility:

The first step was to determine whether speech-based dementia screening could be implemented using available technologies and computational resources. Since the project required speech recording, deep learning-based audio classification, frontend-backend integration, cloud services, and clinician-facing AI explanation, it was necessary to ensure compatibility across all components.

The fine-tuned Wav2Vec2-XLSR-300M model was selected because it provides strong multilingual speech representation and performs effectively in low-resource environments without requiring large-scale model training from scratch. React.js, Node.js/Express, FastAPI, MongoDB, and Docker were chosen because they support modular development, API communication, deployment flexibility, and multi-service architecture. External services such as Google Speech-to-Text and Gemini API were also evaluated for integration reliability and practical deployment suitability. The feasibility study confirmed that the required technologies were accessible, scalable, and suitable for building a deployment-ready dementia screening platform.

• **Operational Feasibility:**

Operational feasibility focused on how the system would function in real clinical workflows. Traditional dementia screening often requires specialist involvement and in-person assessment, which limits accessibility. The proposed speech-based system needed to support clinicians without increasing workflow complexity.

Guided speech tasks such as Self Introduction, Morning Recall, and Picture Description were selected because they are simple for patients to perform while providing clinically meaningful cognitive signals. The inclusion of an AI Clinical Assistant improved operational feasibility by converting technical outputs into clinician-friendly explanations rather than raw prediction scores. This ensured that the system could function as a decision-support tool instead of an isolated prediction engine. Bilingual interface support was also considered essential for improving usability among multilingual patient populations and elderly users.

• **Economic Feasibility:**

Economic feasibility examined whether the proposed solution could reduce the cost and time associated with traditional dementia screening. Conventional assessments require repeated specialist consultations, manual scoring, and resource-intensive evaluation processes.

By automating speech recording, prediction generation, and result interpretation, the system reduces dependence on continuous specialist supervision for preliminary screening. The use of open-source technologies and containerized deployment through Docker further reduced infrastructure cost and improved maintainability. This made the solution more practical for hospitals, elderly care centers, and telemedicine services operating with limited resources.

• **Clinical and Ethical Feasibility:**

Since the project operates within healthcare decision support, clinical responsibility and ethical boundaries were carefully considered. The system was designed strictly as a screening and decision-support tool rather than a diagnostic replacement. AI-generated explanations were constrained to non-diagnostic recommendations, confidence indicators, and speech-based observations to avoid unsafe autonomous medical conclusions. This approach ensured clinician supervision remained central while improving trust and usability of AI-assisted outputs.

• **Project Planning:**

Following feasibility confirmation, the development roadmap was planned using sprint-based milestones under the Agile methodology. Major phases included speech task design, frontend implementation, model integration, backend API development, AI Clinical Assistant integration, testing, and deployment preparation.

Each phase was assigned based on dependency order and practical validation requirements. High-priority modules such as speech recording, session management,

and prediction handling were implemented first, followed by explainability features and deployment optimization. This planning strategy improved development efficiency and reduced implementation risk. The feasibility study confirmed that the proposed system was technically achievable, clinically useful, economically practical, and operationally deployable. This provided a strong foundation for the requirement gathering and implementation stages that followed.

Risk Management Plan

A comprehensive risk management plan was developed to identify, monitor, and minimize potential risks associated with the development of the Speech-Based Dementia Screening and AI Clinical Assistant module. Since the project involves machine learning integration, speech processing, external API dependencies, deployment challenges, and healthcare-related reliability requirements, proactive risk handling was essential to ensure project continuity and successful delivery.

Special attention was given to technical failures, model performance limitations, supervisor feedback, deployment delays, and communication issues between project stakeholders. Early identification of these risks helped improve planning accuracy and reduced implementation disruptions. Table 2.1 shows the risk management plan.

Risk	Trigger	Owner	Response	Resource Required
Illness or absence of project member	Illness / Personal emergency	Project Leader	• Inform supervisor • Redistribute pending tasks	Gantt Chart, Backup Resources
Supervisor / Panel requests major changes	Dissatisfaction with outcomes or report revisions	Project Leader	• Apply required changes promptly • Update documents and resubmit	Product Backlog, Meeting Logs
Missed meetings by supervisor / panel	Conflicts, emergencies	Project Leader	• Notify team • Reschedule meeting or use alternative communication methods	Email, Meeting Logs
Audio processing failure during development	Invalid audio format, API failure, model crash	Team	• Add strict validation • Implement fallback error handling • Retest workflow	Testing Logs, Backup Validation Rules
External API failure (Google STT	Service downtime or	Team	• Enable graceful fallback	API Logs, Monitoring Tools

/ Gemini API)	quota limitations		response • Retry processing and maintain logs	
Model performance inconsistency	Low prediction accuracy or unstable results	ML Engineer	• Fine-tune model parameters • Revalidate dataset quality	Model Evaluation Reports
Delay in Docker deployment / cloud hosting	Configuration issues or environment mismatch	Project Leader	• Allocate buffer time • Prepare local deployment fallback	Timeline Buffer, Hosting Access
Database or session failure	MongoDB connection issues	Backend Team	• Backup session data • Implement reconnection and recovery handling	Database Backup, Logs

Table 2.1: Risk Management Plan

Communication Management Plan

A strong communication strategy was maintained throughout the project to ensure effective collaboration among team members, supervisors, and stakeholders. Since the project required continuous feedback for machine learning validation, frontend usability improvements, backend integration, and report refinement, structured communication was necessary to maintain consistency and reduce development delays.Different communication methods such as email, documentation, phone calls, MS Teams, GitHub, and WhatsApp were used for timely updates, sprint reviews, issue tracking, and academic discussions.

Key Communication Objectives

• Clear, accurate, and audience-specific communication
• Timely feedback and decision-making support
• Proper documentation of progress, changes, and approvals
• Continuous coordination between technical development and academic supervision

Communication Media : The communication media used for the project are:

1. Email
2. Documents (MS Word / PowerPoint / Google Docs)
3. Phone Calls
4. Meetings (Meeting rooms, MS Teams, Zoom)
5. Chats (WhatsApp)
6. GitHub for technical collaboration and issue tracking

Table 2.2 illustrates the communication management plan.

Meeting Type	**Attendees**	**Purpose**	**Frequency**	**Agenda Items**
Planning Kick-off Meeting	Supervisor, Co-supervisor, Team Members	Define project scope, responsibilities, and research direction	Once the project start	Introduce project overview Define research scope, Assign responsibilitie
Execution Kick-off Meeting	Supervisor, Co-supervisor, Team Members	Start implementation phase and confirm milestones	Once at each major phase	Present work plan, Confirm milestones - communication flow
Internal Project Status Meeting	Team Members	Review weekly progress and technical issues	Weekly	•Progress updates •Compare planned vs actual work •Identify blockers
Actual Project Status Meeting	Supervisor,Co-supervisor, Team Members	Formal supervisor progress review	Twice per week / as needed	•Present component status •Demo completed work •Get technical feedback
Project Review Meeting	Supervisor, Co-supervisor, Team Members	Review readiness for Proposal, PP1, PP2, Final	Once per phase	•Documentation review •Milestone validation •Testing readiness
Change Control Meeting	Supervisor, Co-supervisor, Team Members	Evaluate required changes after panel feedback	As needed	• Review panel comments •Prioritize changes •Assign responsibilities

Project-End Review Meeting	Supervisor, Co-supervisor, Team Members	Final evaluation and reflection	Once at project end	• Review final output •Lessons learned •Future improvements

Table 2.2: Communication Management Plan

2.1.2 Requirement Gathering & Analysis

The Requirement Gathering and Analysis phase was a key part of developing the Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform. This phase focused on identifying and refining both functional and non-functional requirements needed to build a clinically useful, technically reliable, and deployment-ready speech-based dementia screening system.

The main goal was to ensure that the system supports early cognitive impairment detection through guided speech assessment, explainable AI interpretation, and practical healthcare workflow integration. Special attention was given to clinician usability, patient accessibility, multilingual support, speech model reliability, and deployment feasibility.

2.1.2.1 Functional Requirements

Functional requirements define the core features and operations the system must perform to achieve its healthcare objectives. These requirements formed the foundation for system design, implementation, and validation.

• Stakeholder Input Approach

Requirement identification was carried out using practical stakeholder-driven approaches rather than only formal interviews.

➢ Informal feedback was gathered from clinicians, caregivers, and healthcare professionals regarding challenges in early dementia screening and patient communication.

➢ Discussions were held with elderly users and caregivers to understand usability issues related to speech recording, task guidance, and language accessibility.

➢ Continuous consultation with supervisors and domain experts ensured alignment with healthcare screening objectives and research standards.

➢ Existing assessments such as ACE-III, MMSE, and IADL were reviewed to understand how guided speech tasks could support cognitive screening.

Figure 2.2 illustrates the requirement collection process.

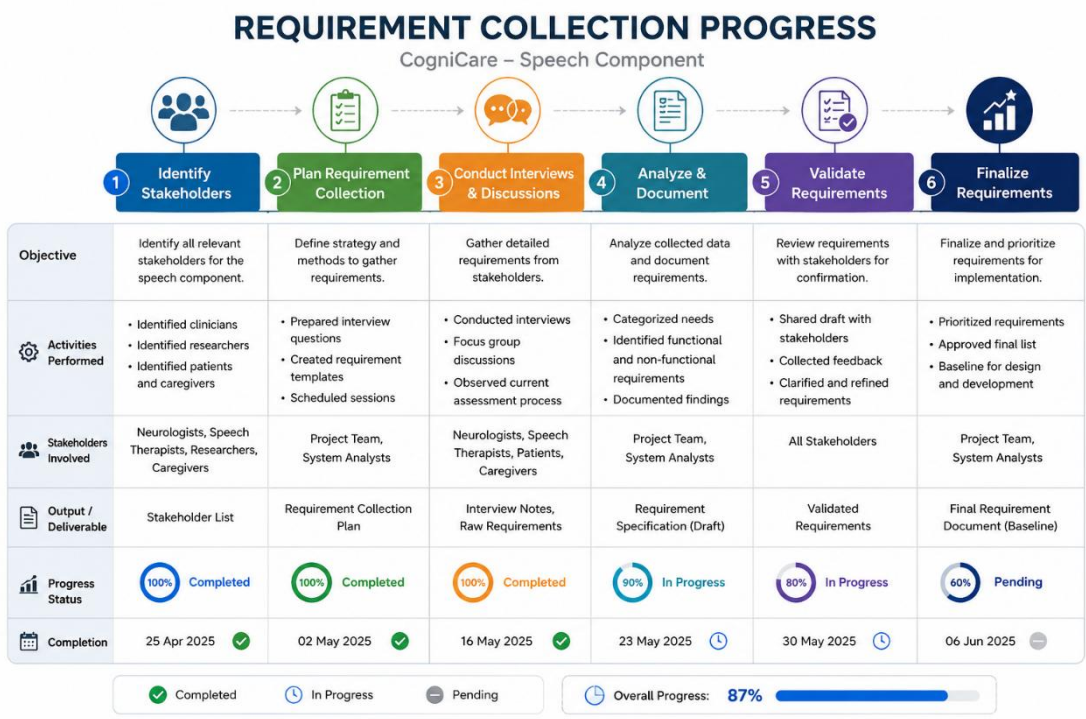


Figure 2.2: Requirement collection process

The system included three guided speech tasks:

- ➢ Self Introduction
- ➢ Morning Recall
- ➢ Picture Description

These tasks were selected to evaluate verbal fluency, memory recall, lexical access, executive function, and narrative coherence.

- **Identification of Key Functionalities**: Stakeholder discussions

revealed the following essential features:

- ➢ Session creation and task progress management
- ➢ Automatic speech processing using fine-tuned Wav2Vec2-XLSR-300M
- ➢ Task-level prediction and scoring
- ➢ Final speech score generation with dementia risk classification
- ➢ AI Clinical Assistant for clinician-friendly explanations
- ➢ Bilingual interface support

➢ Error handling and audio validation
➢ Docker-based multi-service deployment support

- **Architecture Deployment Strategy**: To ensure optimal performance and scalability, the summarization component will be implemented using both monolithic and microservices architectures. Each architecture will be evaluated across key criteria such as response time, resource utilization, and deployment complexity. Following comparative analysis, the most effective option will be selected for final integration into the overall system.

- **Validation and Prioritization**: The collected functional requirements were validated and prioritized to ensure feasibility, clinical relevance, and alignment with user needs. Feedback from clinicians, caregivers, supervisors, and usability testing was used to refine functionalities and remove unnecessary complexity.

  Requirements were categorized into three levels:

  ➢ **Essential** – Core features required for the minimum viable product (MVP), such as guided speech tasks, Wav2Vec2-XLSR-300M prediction, session management, and AI Clinical Assistant support.

  ➢ **Desirable** – Features that improve usability, such as bilingual UI support and dashboard improvements.

  ➢ **Optional** – Future features such as telemedicine integration and extended cognitive task modules.

  This prioritization improved sprint planning and resource allocation.

  After validation, the main functionalities were converted into user stories to show how each feature supports clinicians, patients, and caregivers. Figure 2.3 presents selected user stories derived from this process.

  The finalized primary functional requirements are:

  ➢ Guided speech task assessment using Self Introduction, Morning Recall, and Picture Description
  ➢ Fine-tuned Wav2Vec2-XLSR-300M based raw audio classification
  ➢ Automatic speech recording, upload, and session management
  ➢ Final speech score generation with dementia risk categorization
  ➢ AI Clinical Assistant for clinician-friendly explanation and recommendations
  ➢ Bilingual interface support for accessibility
  ➢ Docker-based deployment-ready multi-service architecture

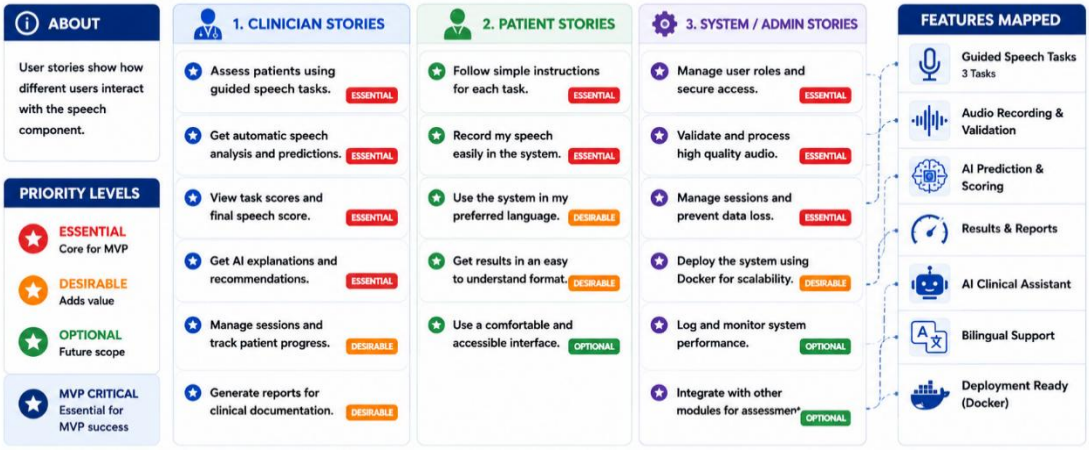


Figure 2.3: User Stories for Requirements

Accurately mapping user requirements in both the use case diagram and the sequence diagram is crucial for elucidating these requirements and establishing their prioritization. Refer to Figure 2.4 for an explanation of the use case diagram, while Figure 2.5 provides an illustration of the sequence diagram.

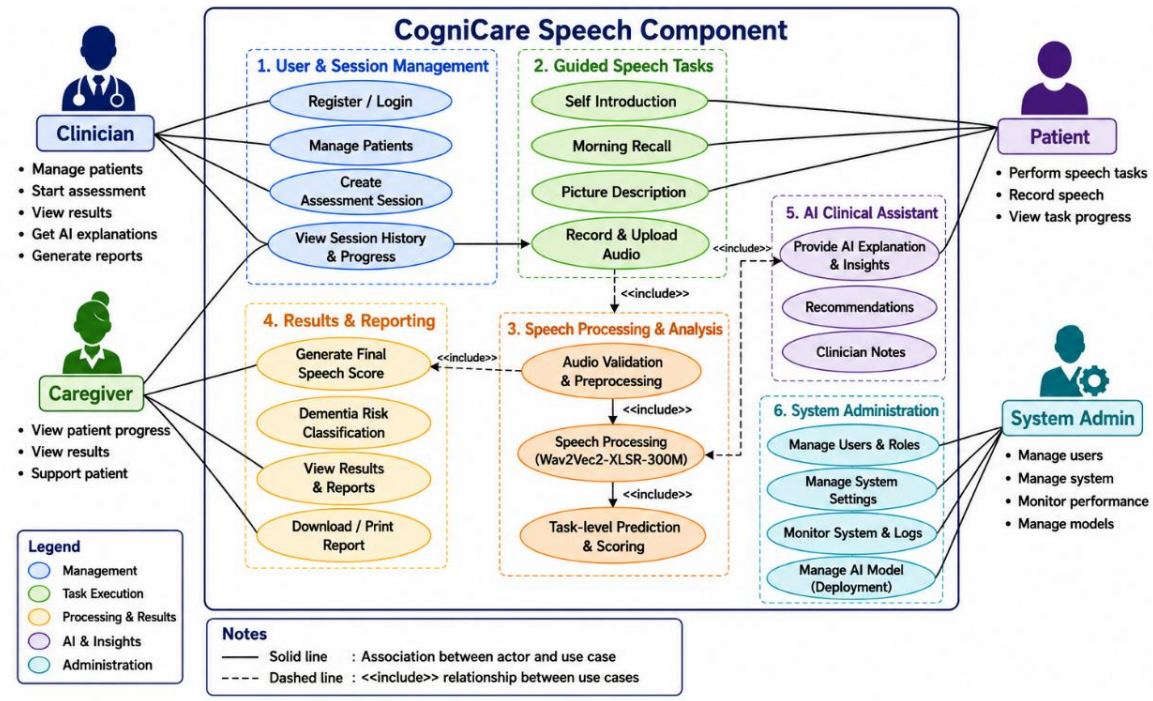


Figure 2.4: Use case Diagram

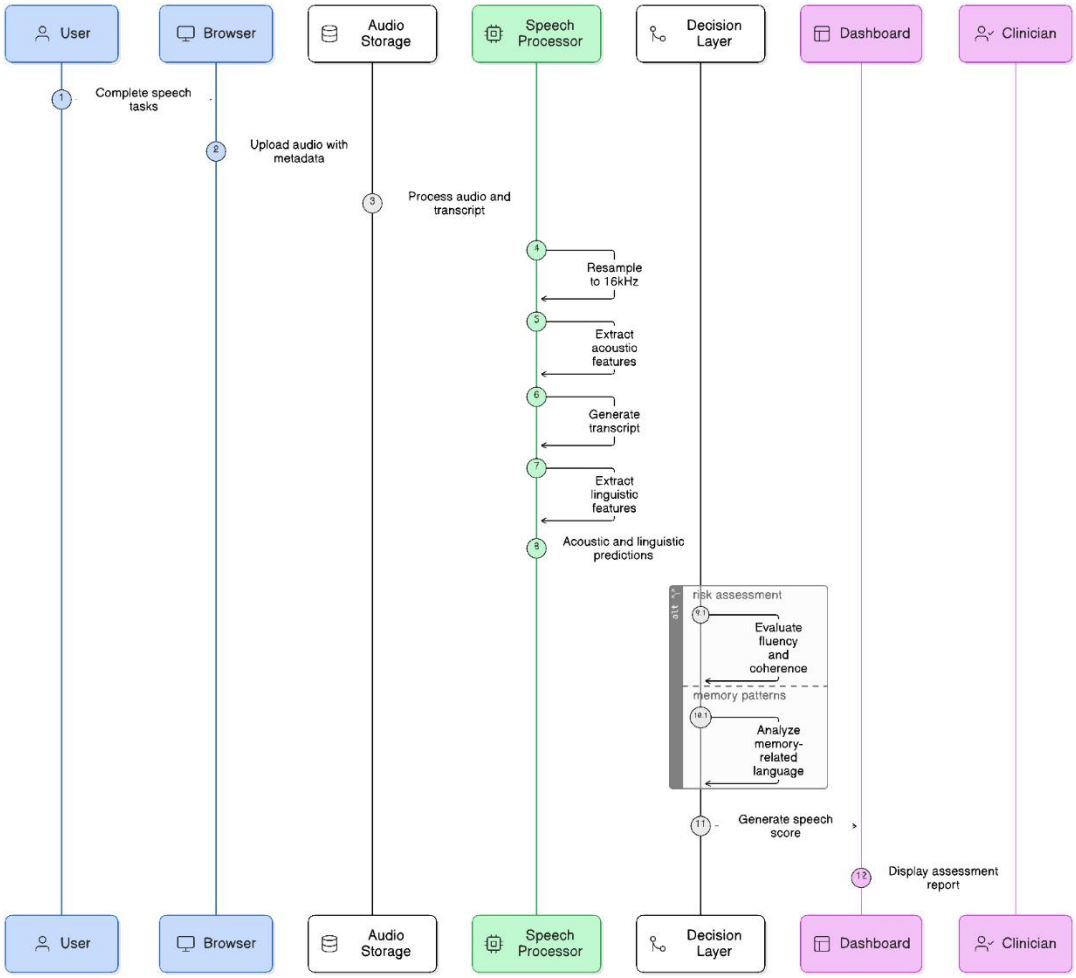


Figure 2.5: Sequence Diagram

2.1.2.2 Non-Functional Requirements

Non-functional requirements define the quality standards, performance expectations, and operational constraints of the Speech-Based Dementia Screening and AI Clinical Assistant module. These requirements are essential to ensure the system remains reliable, clinically safe, accessible, and suitable for real-world healthcare deployment.

- **Feedback and Evaluation Approach**

Instead of relying only on formal interviews, feedback was collected through usability observations, informal discussions with clinicians, caregivers, elderly users, and supervisor guidance. Special attention was given to speech recording simplicity, result clarity, response speed, accessibility, and system reliability.

The focus was placed on ensuring that the platform supports practical clinical workflows while remaining easy to use for both healthcare professionals and patients.

• Performance Benchmarks

The speech screening system must process guided speech tasks efficiently and provide timely cognitive risk predictions without causing workflow delays.

The benchmarks were established as follows:

➢ **Response Time:** Task-level prediction and final speech score generation should be completed within a few minutes after audio submission.

➢ **Audio Handling:** The system must support reliable recording, validation, and processing of audio files without format inconsistencies or upload failures.

➢ **AI Clinical Explanation:** AI-generated clinician explanations should be delivered quickly without significantly increasing the total screening time.

These performance standards are important to ensure smooth use in hospitals, clinics, and elderly care centers.

• Prediction Quality and Clinical Relevance

In healthcare deployment, prediction quality is more important than speed alone. The system must provide clinically meaningful outputs rather than only raw classification scores.

The fine-tuned Wav2Vec2-XLSR-300M model is evaluated using standard metrics such as Accuracy, Precision, Recall, F1-score, and AUC. The results generated must support early dementia risk identification while maintaining clinical reliability and interpretability.

The AI Clinical Assistant must also provide safe, understandable, and non-diagnostic explanations to support clinician trust and responsible use.

• Data Privacy and Ethical Safety

Since the platform handles sensitive patient speech recordings and cognitive assessment data, strong privacy protection is required.

All uploaded audio files and session data are processed within secure system architecture using controlled backend services. Patient information is not exposed to unauthorized access, and AI-generated outputs are restricted to decision-support explanations rather than direct diagnosis.

The system is designed to maintain clinician supervision and prevent unsafe autonomous medical conclusions.

• Usability and Accessibility

➢ The interface is designed to be simple, guided, and easy to use for elderly users with minimal technical experience.

➢ Bilingual UI support improves accessibility for multilingual users and caregivers.

➢ Speech task instructions are presented clearly with guided progression to reduce confusion during assessment.

➢ Error handling and validation messages are designed to be understandable and supportive rather than technical.

These features improve both patient experience and clinician efficiency.

The Requirement Gathering and Analysis phase provided a clear understanding of the system boundaries, core functionalities, and expected quality standards. Through stakeholder feedback, supervisor guidance, and continuous refinement, a complete set of functional and non-functional requirements was established.

These findings guided the design, implementation, testing, and deployment stages while ensuring the platform remains clinically useful, technically reliable, and practically deployable for early dementia risk screening.

The principal non-functional requirements identified are as follows:

• **Usability** – The system must provide a simple and intuitive interface for speech recording, task navigation, and result interpretation.

• **Reliability** – The platform should maintain stable performance across frontend interaction, backend APIs, and ML prediction services.

• **Availability** – High system uptime is necessary to support continuous screening access in clinical environments.

• **Accuracy** – The prediction model must maintain strong classification performance and clinically meaningful output quality.

• **Performance** – Audio processing, prediction generation, and AI explanation must occur with minimal delay to support real-time healthcare workflows.

2.1.3 Designing

The Designing phase translated the gathered requirements into a structured technical plan for implementing the Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform. This stage focused on creating a

scalable, modular, and clinically practical system that supports guided speech assessment, dementia risk prediction, and explainable AI-assisted interpretation.

The primary focus of the design was the speech screening workflow, where users complete guided cognitive speech tasks and receive dementia risk predictions supported by clinician-friendly explanations. While bilingual accessibility and deployment readiness improve usability, the core component remains the speech-based assessment and prediction engine.

System Architecture Diagram:

The system architecture was designed using a multi-service approach to improve scalability, maintainability, and deployment flexibility. The platform integrates frontend interaction, backend orchestration, machine learning inference, database management, and AI Clinical Assistant support.

Figure 2.6 illustrates the complete system architecture, showing the flow from user speech recording to final dementia risk classification and AI-assisted explanation.

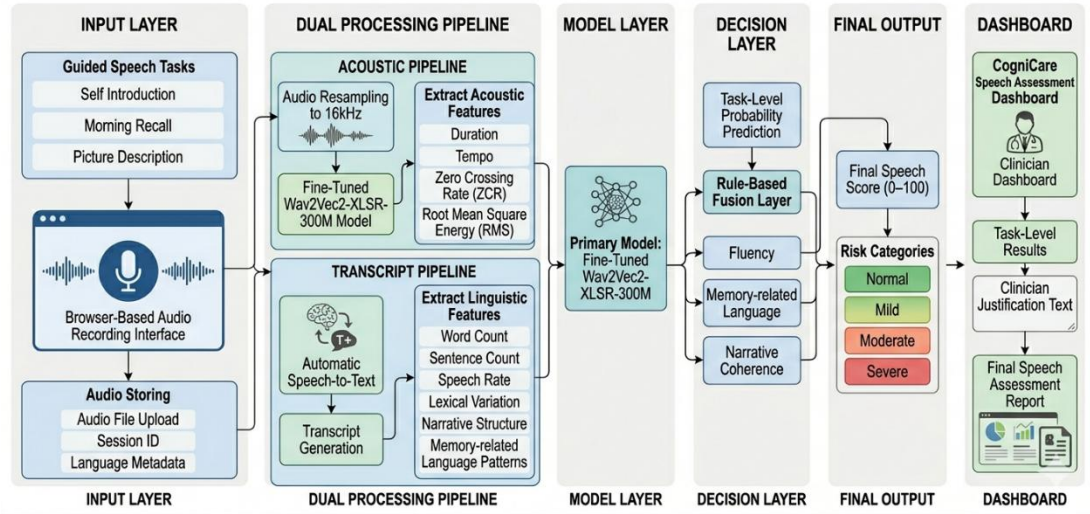


Figure 2.6: System Architecture Diagram

Key Architectural Components

1. User Input Interface

Users interact with the system through a responsive web-based frontend developed using React.js. The interface supports both desktop and mobile access, ensuring accessibility for elderly users, caregivers, and clinicians.

The interface includes:

- Guided speech task instructions
- Browser-based audio recording
- Task progression tracking
- Bilingual UI support
- Final result visualization

The three guided tasks are:

- Self Introduction
- Morning Recall
- Picture Description

These tasks are designed to evaluate verbal fluency, memory recall, lexical access, and narrative coherence.

2. Backend Gateway

The backend gateway was developed using Node.js and Express to manage session creation, task validation, API communication, scoring logic, and final result generation.

It handles:

- Session management
- Audio upload validation
- Prediction requests to ML service
- Final speech score generation
- AI Clinical Assistant integration

3. Machine Learning Service

The ML service was implemented using FastAPI and is responsible for speech prediction using the fine-tuned Wav2Vec2-XLSR-300M model.

This service performs:

- Audio preprocessing
- Raw speech classification
- Task-level prediction generation
- Confidence calculation

4. AI Clinical Assistant

The Gemini-powered AI Clinical Assistant converts technical model outputs into structured clinician-friendly explanations.

It provides:

- Fluency status
- Memory-related observations
- Narrative coherence analysis
- Confidence level
- Recommendation guidance

5. Database and Deployment

MongoDB is used for storing session data, task results, and prediction history. Docker-based deployment supports multi-service containerization for reliable testing and future cloud deployment.

Workflow Design

The complete system flow begins with guided speech recording, followed by audio validation, ML prediction, final score generation, and AI explanation delivery.

Speech-based Flow:

Figure 2.7 and 2.8 illustrates the flow of Speech based components very clearly.

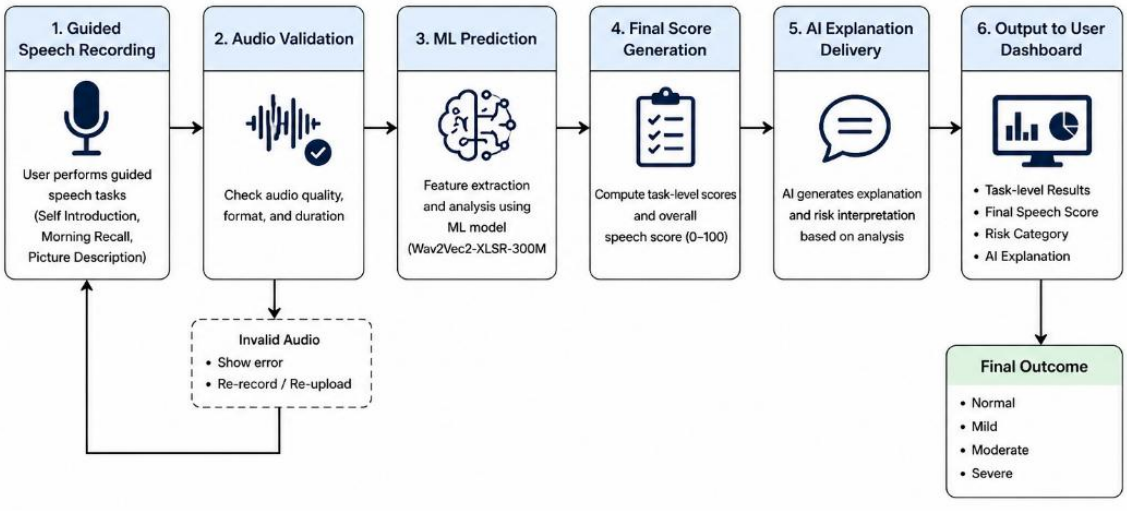


Figure 2.7: ***Speech-based screening Flow***

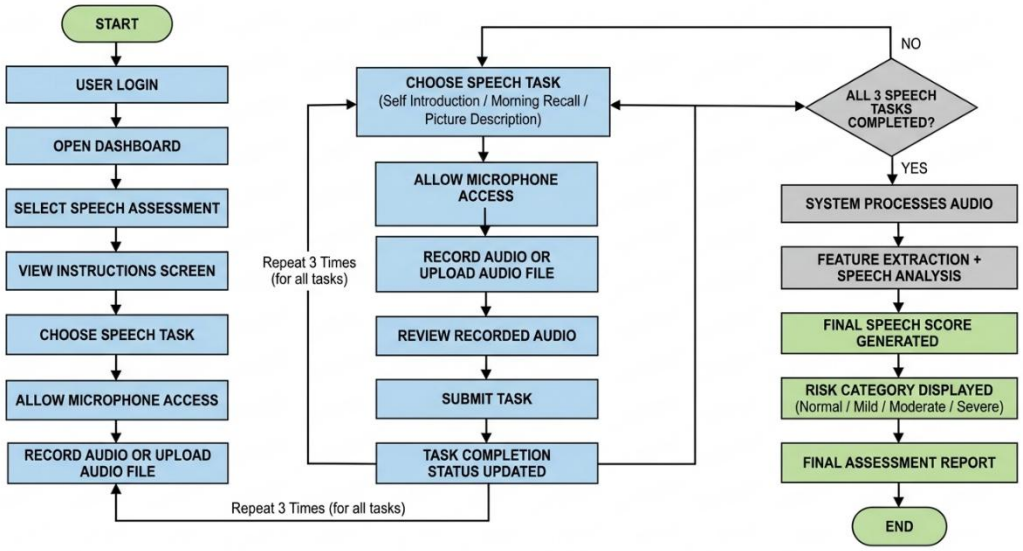


Figure 2.8: User Flow

2.1.4 Implementation

The Implementation phase marked the transition from system design to the actual development of the Speech-Based Dementia Screening and AI Clinical Assistant module within the CogniCare platform. This phase focused on converting the planned architecture into a fully functional healthcare screening system capable of guided speech assessment, dementia risk prediction, and clinician-friendly AI interpretation.

The platform was developed to support guided speech tasks, speech recording, fine-tuned Wav2Vec2-XLSR-300M prediction, final speech score generation, bilingual accessibility, and AI Clinical Assistant integration. The goal of this phase was to ensure that the system functions as a practical and deployment-ready cognitive screening solution rather than only a research prototype.

Task Breakdown and Project Management:

Before implementation began, a structured task breakdown and project management process was carried out to organize the development workflow efficiently. Jira was used to manage tasks, divide the project into smaller subtasks, and monitor progress throughout the implementation stage.

Each task was assigned with a clear objective, timeline, and priority level to ensure systematic development and timely completion. Major tasks included frontend speech recording implementation, backend API development, ML model integration, AI Clinical Assistant workflow, bilingual UI improvements, testing, and Docker deployment preparation.

The project followed an Agile sprint-based approach, where development was divided into multiple sprints focusing on specific modules. At the end of each sprint, deliverables were reviewed to validate functionality before proceeding to the next stage. This iterative approach improved flexibility and allowed quick resolution of technical challenges such as audio validation issues, API failures, and UI usability improvements.

The Work Breakdown Structure (WBS) was also used to organize implementation activities and divide the major objectives into smaller manageable tasks. This helped improve responsibility allocation, progress tracking, and risk identification during development. Figure 2.9 demonstrates the WBS and other related modules.



Figure 2.9: Work Breakdown Structure

Development Environment and Tools:

The implementation of the system was primarily carried out using Visual Studio Code (VS Code), which provided a flexible development environment for frontend, backend, and API integration. Its strong support for JavaScript, Python, debugging tools, and version control made it suitable for full-stack development.

Python was used for machine learning implementation and speech model integration, while JavaScript was used for frontend and backend development. The fine-tuned Wav2Vec2-XLSR-300M model was managed using Python-based ML services with FastAPI for inference handling.

React.js was used to build the frontend speech recording interface, allowing browser-based guided task interaction and responsive UI design. Node.js and Express were used to implement the backend gateway for session management, audio validation, scoring logic, and API orchestration.

MongoDB was used for session persistence and storing task results, while Docker supported containerized deployment across all services.Additional tools included Google Speech-to-Text for transcription support, Gemini API for AI Clinical Assistant responses, GitHub for version control, and Postman for API testing and validation.These tools were integrated to create a complete and deployment-ready Speech-Based Dementia Screening and AI Clinical Assistant platform.

2.1.4.1 Speech Data Collection Script for Summaries

The first step of implementation focused on preparing the speech dataset and structured task flow required for dementia risk prediction. Since the system relies on guided speech assessment rather than general conversation, the data collection process was designed to capture clinically meaningful speech samples that reflect early cognitive decline indicators such as reduced fluency, hesitation, memory recall difficulty, and narrative disorganization.

The selected guided speech tasks were:

1. Self-Introduction
2. Morning Recall
3. Picture Description

These tasks were chosen because they help evaluate verbal fluency, episodic memory, lexical retrieval, executive functioning, and narrative coherence, which are strongly associated with early dementia symptoms.

Main Steps Involved in the Data Collection Process

1. Session Creation

The first step involves creating a unique speech screening session for each user through the frontend interface. Each session stores patient task progress, uploaded recordings, prediction results, and final dementia risk scores. This allows structured tracking of each assessment and ensures that all speech tasks remain linked to the same cognitive screening workflow.

2. Audio Recording and Upload

Users complete each guided speech task using browser-based recording through the React.js frontend. Audio recordings are captured directly and uploaded securely to the

backend gateway. The system supports controlled recording conditions to improve consistency and reduce background noise interference during prediction.

3. Audio Validation

Before prediction begins, uploaded audio files are validated to ensure proper processing.

The validation process checks:

• Supported file format
• File size limitations
• Audio completeness
• Recording consistency

This step prevents corrupted or invalid recordings from affecting model performance.

4. Audio Preprocessing

Validated recordings are preprocessed before being sent to the fine-tuned Wav2Vec2-XLSR-300M model. This includes:

• Audio normalization
• Standardized waveform conversion
• Sampling rate consistency
• Noise reduction where necessary

These preprocessing steps improve model stability and prediction reliability.

5. Prediction Preparation

Each processed audio file is linked to its corresponding speech task and prepared for model inference. The fine-tuned Wav2Vec2-XLSR-300M model then analyzes the raw speech patterns directly to identify dementia-related characteristics. Task-level prediction outputs include:

• Probability score
• Task score
• Confidence level

6. Final Result Storage

After prediction, the results are stored in MongoDB along with task metadata, session history, and final outcome values. This supports clinician review and AI Clinical Assistant interpretation.

Mechanism for Preparing Speech Assessment Data

The purpose of this process is to convert raw user speech into structured and reliable input for dementia prediction.

1. Guided task responses are collected through browser-based recording.
2. Audio files are validated and filtered to remove incomplete or invalid recordings.
3. Preprocessing ensures standardized input for the Wav2Vec2-XLSR-300M model.
4. Task-level outputs are generated and stored for final session-level scoring.
5. Final speech scores are calculated by combining all required tasks into a complete dementia risk assessment.

This preprocessing workflow ensures that the prediction model receives clean, consistent, and clinically meaningful speech input, improving both accuracy and explainability.

2.1.4.2 Speech Dataset for Model Training and Evaluation

This subsection focuses specifically on the dataset used for training and evaluating the speech-based dementia detection model. Unlike the previous section, which explained speech recording and preprocessing workflow, this section describes how the final dataset was structured for model training, validation, and prediction.

The dataset used for this component was based on the Dem@Care DS3, DS5, and DS7 speech datasets, which are part of the wider Dem@Care dementia dataset collection. These datasets contain dementia-related speech and audio recordings collected through clinical, laboratory, and home-monitoring studies, mainly conducted in English, Portuguese, Greece . The selected subsets were suitable for this research because they provide speech data relevant to cognitive impairment screening and allow the analysis of dementia-related speech behaviors such as reduced fluency, pauses, hesitation, lexical retrieval difficulty, and narrative inconsistency. Therefore, Dem@Care DS3, DS5, and DS7 were used as the main dataset evidence for preparing, training, validating, and evaluating the speech-based dementia screening model. Access to the dataset was obtained through a formal research request submitted via email to the relevant Dem@Care dataset custodians. Upon receiving permission, the Dem@Care DS3, DS5, and DS7 subsets were selected as the primary speech dataset sources for this study.

Since the core prediction model was based on a fine-tuned Wav2Vec2-XLSR-300M architecture, the dataset required properly labeled raw audio samples rather than only manually extracted numerical features. The objective was to prepare a balanced and

clinically meaningful speech dataset capable of capturing early dementia-related speech abnormalities such as pauses, hesitation, reduced fluency, lexical retrieval difficulty, and narrative inconsistency. Based on these Dem@Care subsets, the final dataset was structured into task-level speech samples suitable for deep-learning-based audio classification using Wav2Vec2-XLSR-300M.

Main Characteristics of the Dataset

The dataset consisted of labeled speech recordings collected through guided cognitive tasks. Each sample represented one complete task response and was mapped to a dementia risk classification label used during model fine-tuning.

The major dataset attributes included:

- Audio File – Raw recorded speech input in standardized audio format
- Label – Dementia risk classification (Healthy / Mild Cognitive Impairment / High Risk)
- Task Type – Self Introduction, Morning Recall, or Picture Description
- Duration – Length of the speech recording
- Transcript Reference – Supporting text generated for review purposes
- Session Identifier – Unique ID for linking task responses within the same assessment session

This structure ensured both model training consistency and traceability during evaluation.

Dataset Preparation Process

The raw recordings were first standardized to maintain consistency across all training samples. Audio normalization and sampling rate correction were applied to ensure compatibility with Wav2Vec2-XLSR-300M input requirements.

Samples with incomplete recordings, excessive background noise, microphone interruptions, or corrupted uploads were excluded to reduce training noise and improve model reliability. Duplicate recordings were also removed to prevent bias and overfitting.

Label balancing was carefully considered to avoid class imbalance, which is common in healthcare datasets. This helped improve generalization and reduced prediction bias toward majority classes.

Training and Validation Split

The finalized dataset was divided into training, validation, and testing subsets to ensure proper model evaluation.

- Training Set – used for model fine-tuning
- Validation Set – used for hyperparameter tuning and overfitting control
- Test Set – used for final performance evaluation

This separation ensured objective measurement of model performance using metrics such as accuracy, precision, recall, F1-score, and AUC.

2.1.4.3 Model Fine-Tuning Using Wav2Vec2-XLSR-300M

The objective of this stage was to customize the pre-trained Wav2Vec2-XLSR-300M model for detecting dementia-related speech patterns from guided cognitive speech recordings. Fine-tuning was performed using a structured speech dataset containing labeled audio samples collected from the guided tasks: Self Introduction, Morning Recall, and Picture Description.

This process was carried out mainly in Google Colab using GPU acceleration, which provided the computational resources required for deep learning model training and experimentation.

Step 1: Configuring the Environment in Google Colab

The fine-tuning process began in Google Colab, a cloud-based Jupyter notebook environment that supports machine learning development efficiently.

It provided:

- Free GPU access (Tesla T4 / P100)
- Easy integration with Google Drive for dataset storage
- Built-in Python support for model training

To prepare the environment:

• The transformers, datasets, torchaudio, and librosa libraries were installed for audio processing and model training

• Google Drive was connected to access the labeled speech dataset and save trained model checkpoints

Step 2: Loading and Structuring the Dataset

The dataset used for fine-tuning consisted of labeled speech recordings prepared during the preprocessing stage.

It included:

- Audio Files – standardized raw speech recordings
- Labels – dementia risk classification categories

- Task Type – guided speech task category
- Duration and metadata for validation support

The dataset was loaded using Python and converted into a format compatible with the Hugging Face Dataset framework to support efficient model training.

Step 3: Audio Preparation and Feature Extraction

Before training, the raw audio files were standardized for compatibility with Wav2Vec2-XLSR-300M.

This included:

- Converting audio into a uniform sampling rate (16 kHz)
- Audio normalization to reduce recording inconsistencies
- Removing corrupted or incomplete recordings
- Padding or truncating audio sequences where necessary

Unlike traditional models, Wav2Vec2 works directly with raw waveforms, so no manual handcrafted feature extraction was required during training.

Step 4: Loading the Pre-trained Model

The base model used was facebook/wav2vec2-xls-r-300m, a multilingual transformer-based speech model pretrained on large-scale speech datasets.

This model was selected because of:

- Strong multilingual speech representation learning
- High suitability for low-resource speech environments
- Ability to learn dementia-related acoustic patterns directly from raw audio

Its pretrained knowledge significantly improved fine-tuning efficiency and prediction performance.

Step 5: Establishing Training Parameters

Training parameters were configured to control the fine-tuning process.

These included:

- Output Directory – for saving model checkpoints
- Batch Size – configured based on GPU memory availability
- Number of Epochs – multiple training cycles for stable learning
- Learning Rate – optimized for transfer learning
- Logging Steps – for monitoring training progress
- Checkpoint Saving – to prevent loss during long training sessions

These parameters were defined using Hugging Face Training Arguments.

Step 6: Training the Model Using Trainer API

After preparing the model, dataset, and parameters, Hugging Face's Trainer API was used to begin fine-tuning.

During training:

- The model learned to classify dementia-related speech patterns from task recordings
- Loss was calculated by comparing predicted labels with actual labels
- Model weights were updated iteratively to improve prediction accuracy

Training progress was monitored in real time through Colab logs, including validation loss and model performance indicators.

Step 7: Saving the Fine-Tuned Model

After training was completed:

- The fine-tuned model and processor were saved to Google Drive for future use
- This preserved version was later integrated into the ML service for real-time dementia risk prediction

The saved model supports inference, deployment, and future retraining if additional datasets become available.

The fine-tuning process successfully adapted Wav2Vec2-XLSR-300M for speech-based dementia detection. The final model can analyze guided cognitive speech recordings and generate clinically meaningful dementia risk predictions suitable for real-world screening environments.

Figure 2.10 illustrates the Google Colab notebook configuration used for model fine-tuning, including dataset loading, audio preparation, and training workflow.



```python
from typing import Any, Dict, Union

import torch
from packaging import version
from torch import nn

from transformers import (
    Trainer,
    is_apex_available,
)

if is_apex_available():
    from apex import amp

if version.parse(torch.__version__) >= version.parse("1.6"):
    _is_native_amp_available = True
    from torch.cuda.amp import autocast


class CTCTrainer(Trainer):
    def training_step(self, model: nn.Module, inputs: Dict[str, Union[torch.Tensor, Any]]) -> torch.Tensor:
        """
        Perform a training step on a batch of inputs.

        Subclass and override to inject custom behavior.

        Args:
            model (:obj:`nn.Module`):
                The model to train.
            inputs (:obj:`Dict[str, Union[torch.Tensor, Any]]`):
                The inputs and targets of the model.

                The dictionary will be unpacked before being fed to the model. Most models expect the targets under the
                argument :obj:`labels`. Check your model's documentation for all accepted arguments.

        Return:
            :obj:`torch.Tensor`: The tensor with training loss on this batch.
        """

        model.train()
        inputs = self._prepare_inputs(inputs)

        if self.use_amp:
            with autocast():
                loss = self.compute_loss(model, inputs)
        else:
            loss = self.compute_loss(model, inputs)

        if self.args.gradient_accumulation_steps > 1:
            loss = loss / self.args.gradient_accumulation_steps

        if self.use_amp:
            self.scaler.scale(loss).backward()
        elif self.use_apex:
            with amp.scale_loss(loss, self.optimizer) as scaled_loss:
                scaled_loss.backward()
        elif self.deepspeed:
            self.deepspeed.backward(loss)
        else:
            loss.backward()

        return loss.detach()
```

Figure 2.10: Finetuning the Model

2.1.4.4 Speech Assessment and Prediction System

This integrated system provides guided speech assessment, dementia risk prediction, final speech score generation, and AI-assisted clinical interpretation for early cognitive impairment detection. It combines frontend speech recording, backend orchestration, deep learning prediction, and clinician-friendly explanation support within a single deployment-ready healthcare workflow.

1. Core Components Initialization

The speech assessment system begins by setting up the main components required for speech screening:

• Fine-tuned Wav2Vec2-XLSR-300M model for dementia-related speech classification

• Backend Gateway for session creation, task validation, scoring, and API communication

• MongoDB database for session storage and prediction tracking

• Gemini-powered AI Clinical Assistant for clinician-friendly explanations

These components are connected through a modular multi-service architecture to ensure scalability and maintainability.

2. API Endpoints and Functionality

The FastAPI and Node.js backend provide several important endpoints for speech assessment:

a) /create-session/

• Creates a new screening session for each user
• Generates a unique session ID for tracking all guided tasks
• Maintains task progress and final speech results

b) /upload-audio/

• Handles browser-based speech recording uploads
• Validates file type, size, and audio completeness
• Links each recording to its correct guided task

c) /predict-speech/

• Sends validated audio to the Wav2Vec2-XLSR-300M model
• Generates task-level dementia risk probability and task score
• Stores prediction results for final analysis

d) /finalize-session/

• Combines all required task scores
• Generates the final speech score and dementia risk category
• Produces confidence level and clinical interpretation fields

e) /clinical-assistant/

• Sends final prediction outputs to the Gemini-powered AI Clinical Assistant
• Returns clinician-friendly explanations and recommendation guidance

3. Audio Validation and Processing

The system supports speech recording from guided tasks through browser-based audio capture.

A multi-step validation process ensures:

- ➢ Supported audio format
- ➢ File size limits
- ➢ Recording completeness
- ➢ Prevention of corrupted uploads

Audio preprocessing includes normalization, standardized waveform conversion, and compatibility preparation for Wav2Vec2-XLSR-300M inference.

4. Prediction Workflow

The prediction engine uses the fine-tuned Wav2Vec2-XLSR-300M model to classify dementia-related speech patterns directly from raw audio.

The model detects:

- • Hesitation patterns
- • Reduced fluency
- • Memory recall difficulty
- • Narrative disorganization
- • Speech rhythm irregularities

Each task produces:

- ➢ Probability score
- ➢ Task score
- ➢ Confidence estimate

5. AI Clinical Assistant Integration

The Gemini-powered AI Clinical Assistant converts technical prediction outputs into structured explanations.

It provides:

- ➢ Fluency status
- ➢ Memory-related observations

➢ Narrative coherence evaluation
➢ Confidence level
➢ Safe non-diagnostic recommendation guidance

6. Output and Result Handling

All prediction results are stored securely for session-based access.

The system supports:

• Final speech score visualization
• Task-wise prediction review
• Clinician explanation reports
• Session history tracking

The frontend displays these results through dashboard cards for easy interpretation by clinicians and caregivers.

Overall Mechanism

The system improves early dementia screening by:

• Capturing structured guided speech recordings
• Performing automated dementia risk prediction
• Generating final speech score classification
• Supporting clinicians with explainable AI interpretation
• Improving accessibility through bilingual guided workflows

The complete process is optimized for practical healthcare environments, ensuring reliability, usability, and deployment readiness.

Figures 2.11 – 2.16 illustrate the backend APIs, audio validation logic, prediction workflow, AI Clinical Assistant integration, and final result generation process.

app.py M JS agentRoutes.js

backend-gateway > routes > JS agentRoutes.js > ...

```
const multer = require("multer");
const fs = require("fs");
const path = require("path");

const agentController = require("../controllers/agentController");

const uploadDir = process.env.UPLOAD_DIR || "storage/uploads";
fs.mkdirSync(uploadDir, { recursive: true });

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const safeName = (file.originalname || "upload").replace(/\s+/g, "_");
    const sessionPart = req?.params?.id ? `${req.params.id}_` : "";
    cb(null, `${sessionPart}${Date.now()}_${safeName}`);
  }
});
const upload = multer({ storage });
const allowedAudioExt = new Set([".wav", ".mp3", ".m4a", ".webm"]);
const allowedAudioMime = new Set([
  "audio/wav",
  "audio/mpeg",
  "audio/mp4",
  "audio/webm",
  "audio/x-wav",
  "audio/x-m4a",
]);
const uploadAudio = multer({
  storage,
  limits: { fileSize: 10 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    const ext = path.extname(file.originalname || "").toLowerCase();
    const type = String(file.mimetype || "").toLowerCase();
    if (!allowedAudioExt.has(ext) && !allowedAudioMime.has(type)) {
      return cb(new Error("Invalid audio format. Allowed: .wav, .mp3, .m4a, .webm"));
    }
    cb(null, true);
  },
});

// session management endpoints
router.post("/session", agentController.createSessionTool);
router.get("/session/:id", agentController.getSessionTool);
router.get("/sessions", agentController.listSessions);
router.patch("/session/:id", agentController.updateSession);
router.post("/session/:id/finish", agentController.finishSession);

// upload endpoints
router.post("/session/:id/upload/audio", (req, res, next) => {
  uploadAudio.single("audio")(req, res, (err) => {
    if (err) return res.status(400).json({ error: err.message });
    return next();
  });
}, agentController.uploadAudio);
router.post("/session/:id/upload/video", upload.single("video"), agentController.uploadVideo);

// shorthand paths matching new requirements
router.post("/upload-video/:id", upload.single("video"), agentController.uploadVideo);
router.post("/process-video/:id", agentController.processVideo);
router.post("/process-audio/:id", agentController.processAudio);
// explicit new style endpoints
router.post("/upload/video/:id", upload.single("video"), agentController.uploadVideo);
router.post("/process/video/:id", agentController.processVideo);
router.post("/process/audio/:id", agentController.processAudio);
// analyze emotion for an uploaded video file directly
router.post("/analyze_emotion", upload.single("video"), agentController.analyzeEmotion);

// tool/prediction endpoints
router.post("/predict_health_risk", agentController.predictHealthRisk);
router.post("/predict_audio_risk", agentController.predictAudioRisk);
router.post("/predict_transcript_risk", agentController.predictTranscriptRisk);
router.post("/compute_final_score", agentController.computeFinalScore);

// explanation & chat
router.get("/session/:id/explain", agentController.explainSession);
router.post("/chat", agentController.chatSession);
router.get("/llm-health", agentController.llmHealth);

module.exports = router;
```

Figure 2.11: Backend Agent Routes Implementation

```
// process a previously-uploaded audio file and compute final score/risk
async function processAudio(req, res) {
  try {
    const sessionId = req.params.id;
    let session = await Session.findOne(_buildOwnedSessionQuery(req, sessionId));
    if (!session) return res.status(404).json({ error: "Not found" });
    if (!session.uploads || !session.uploads.audioPath) {
      return res.status(400).json({ error: "No audio uploaded for this session" });
    }
    const taskId = normalizeTaskKey(req.body?.taskId);
    if (taskId && !REQUIRED_SPEECH_TASKS.includes(taskId)) {
      return res.status(400).json({ error: "Invalid taskId" });
    }

    const audioRes = await ml.predictAudioFile({ audioPath: session.uploads.audioPath });

    // Persist model outputs (latest task inference snapshot)
    const baseSet = {};
    if (audioRes.speechPrediction) baseSet["results.speech"] = audioRes.speechPrediction;
    if (audioRes.transcriptPrediction) baseSet["results.transcript"] = audioRes.transcriptPrediction;
    const speechLabel = Number(audioRes?.speechPrediction?.label);
    const transcriptLabel = Number(audioRes?.transcriptPrediction?.label);
    const taskAudioProbability =
      clamp01(audioRes?.speechPrediction?.probability) ??
      (speechLabel === 1 ? 1 : speechLabel === 0 ? 0 : null);
    const taskTranscriptProbability =
      clamp01(audioRes?.transcriptPrediction?.probability) ??
      (transcriptLabel === 1 ? 1 : transcriptLabel === 0 ? 0 : null);
    const taskScore =
      taskAudioProbability == null ? null : Number((taskAudioProbability * 100).toFixed(2));

    console.log("[speech-fusion] processAudio taskId=", taskId);
    console.log("[speech-fusion] processAudio speechPred=", audioRes?.speechPrediction || null);
    console.log("[speech-fusion] processAudio transcriptPred=", audioRes?.transcriptPrediction || null);
    console.log(
      "[speech-fusion] processAudio computed taskScore=",
      taskScore,
      "taskAudioProbability=",
      taskAudioProbability,
      "taskTranscriptProbability=",
      taskTranscriptProbability
    );

    if (taskId) {
      const taskPayload = {
        score: taskScore,
        audioProbability: taskAudioProbability,
        transcriptProbability: taskTranscriptProbability,
        audioPath: session.uploads.audioPath,
        completedAt: new Date().toISOString(),
      };
      baseSet[`results.speechTaskScores.${taskId}`] = taskPayload;
      baseSet[`taskLogs.speechTasks.${taskId}`] = taskPayload;
    }

    await Session.updateOne(_buildOwnedSessionQuery(req, sessionId), { $set: baseSet });
    session = await Session.findOne(_buildOwnedSessionQuery(req, sessionId));
    if (!session) return res.status(404).json({ error: "Session not found after update" });

    session.results = session.results || {};
    session.taskLogs = session.taskLogs || {};
    session.results.speechTaskScores = normalizeSpeechTaskMap(
      session.results.speechTaskScores || session.taskLogs.speechTasks || {}
    );
```

Figure 2.12: audio validation logic

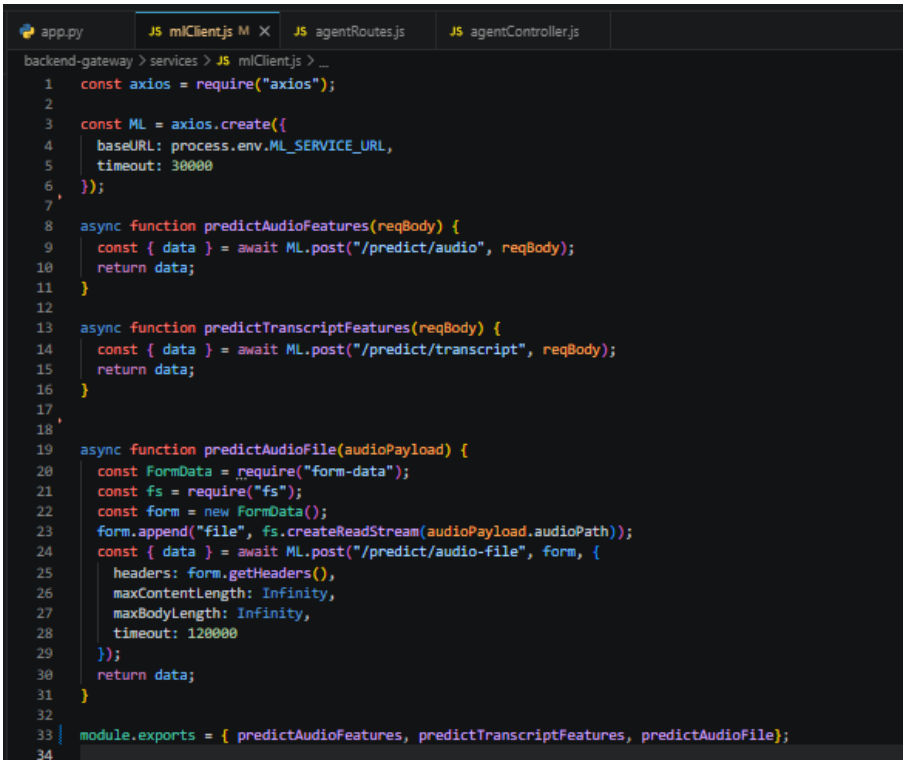


Figure 2.13: ML Client Integration

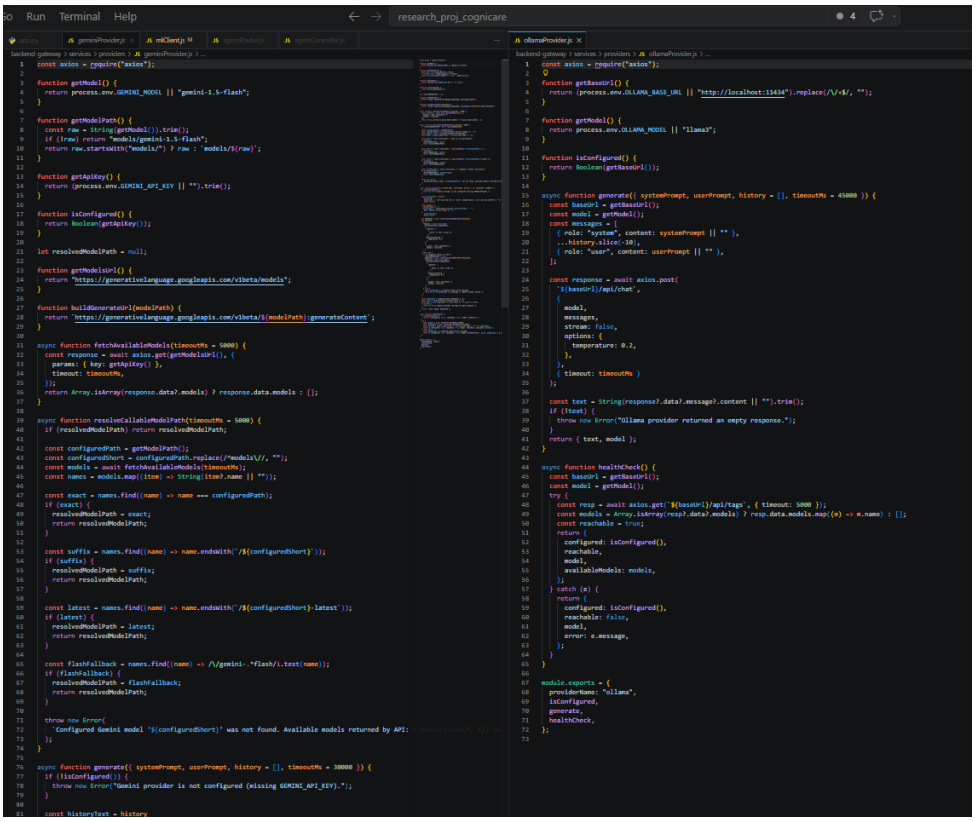
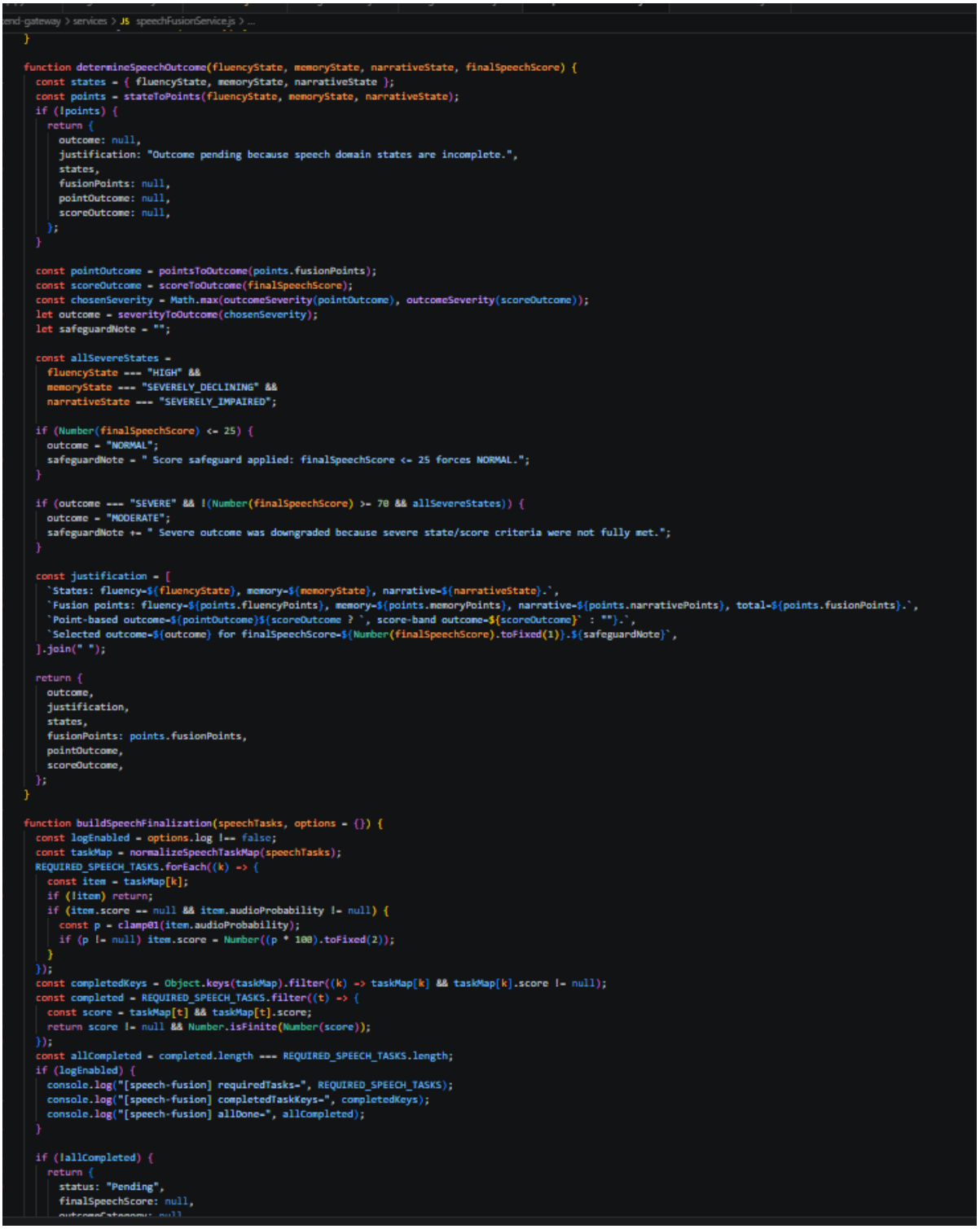


Figure 2.14: AI Clinical Assistant Integration



```
end-gateway > services > JS speechFusionService.js > ...
}

function determineSpeechOutcome(fluencyState, memoryState, narrativeState, finalSpeechScore) {
  const states = { fluencyState, memoryState, narrativeState };
  const points = stateToPoints(fluencyState, memoryState, narrativeState);
  if (!points) {
    return {
      outcome: null,
      justification: "Outcome pending because speech domain states are incomplete.",
      states,
      fusionPoints: null,
      pointOutcome: null,
      scoreOutcome: null,
    };
  }

  const pointOutcome = pointsToOutcome(points.fusionPoints);
  const scoreOutcome = scoreToOutcome(finalSpeechScore);
  const chosenSeverity = Math.max(outcomeSeverity(pointOutcome), outcomeSeverity(scoreOutcome));
  let outcome = severityToOutcome(chosenSeverity);
  let safeguardNote = "";

  const allSevereStates =
    fluencyState === "HIGH" &&
    memoryState === "SEVERELY_DECLINING" &&
    narrativeState === "SEVERELY_IMPAIRED";

  if (Number(finalSpeechScore) <= 25) {
    outcome = "NORMAL";
    safeguardNote = " Score safeguard applied: finalSpeechScore <= 25 forces NORMAL.";
  }

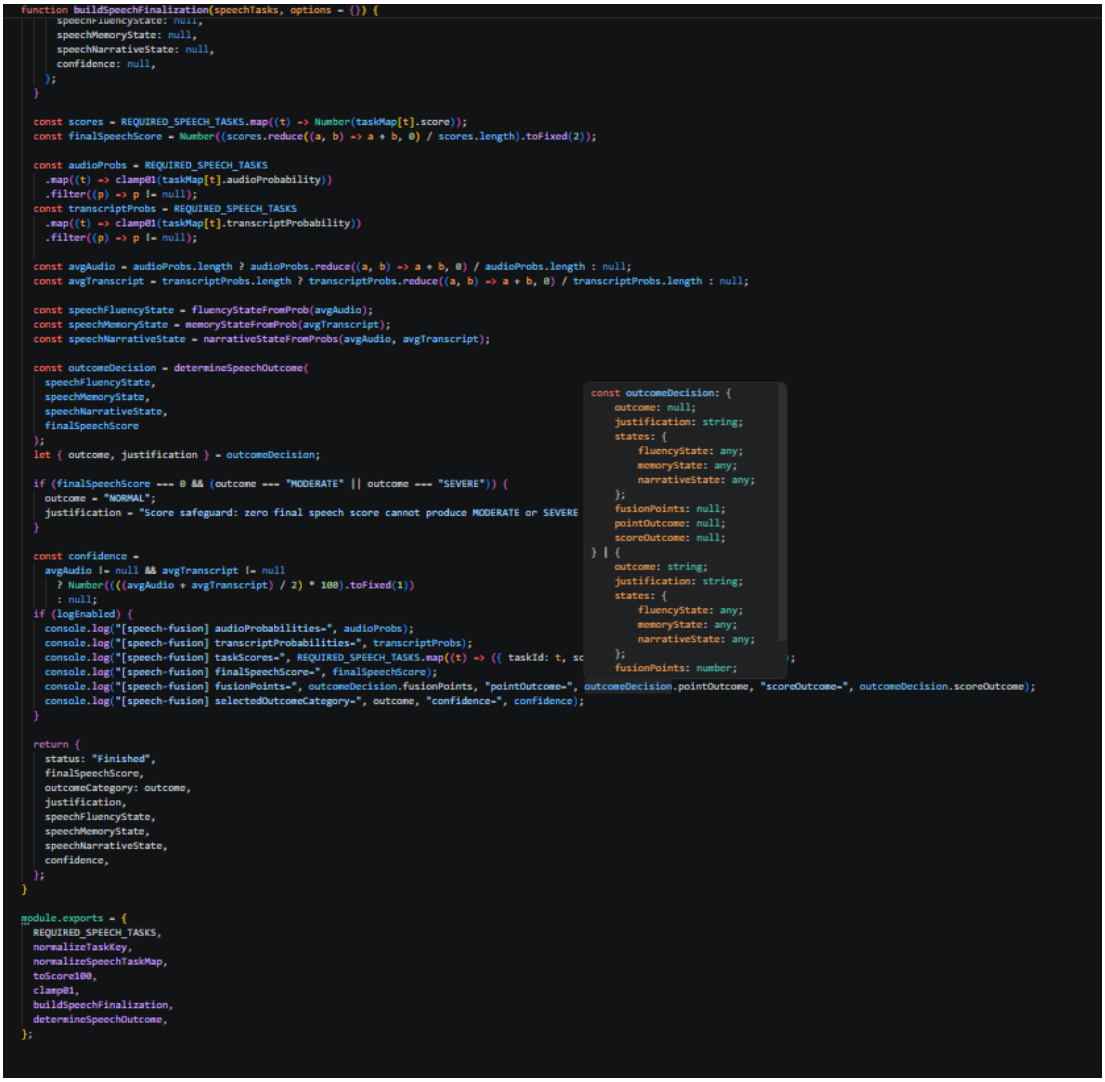
  if (outcome === "SEVERE" && !(Number(finalSpeechScore) >= 70 && allSevereStates)) {
    outcome = "MODERATE";
    safeguardNote += " Severe outcome was downgraded because severe state/score criteria were not fully met.";
  }

  const justification = [
    `States: fluency=${fluencyState}, memory=${memoryState}, narrative=${narrativeState}.`,
    `Fusion points: fluency=${points.fluencyPoints}, memory=${points.memoryPoints}, narrative=${points.narrativePoints}, total=${points.fusionPoints}.`,
    `Point-based outcome=${pointOutcome}${scoreOutcome ? `, score-band outcome=${scoreOutcome}` : ""}.`,
    `Selected outcome=${outcome} for finalSpeechScore=${Number(finalSpeechScore).toFixed(1)}.${safeguardNote}`,
  ].join(" ");

  return {
    outcome,
    justification,
    states,
    fusionPoints: points.fusionPoints,
    pointOutcome,
    scoreOutcome,
  };
}

function buildSpeechFinalization(speechTasks, options = {}) {
  const logEnabled = options.log !== false;
  const taskMap = normalizeSpeechTaskMap(speechTasks);
  REQUIRED_SPEECH_TASKS.forEach((k) => {
    const item = taskMap[k];
    if (!item) return;
    if (item.score == null && item.audioProbability != null) {
      const p = clamp01(item.audioProbability);
      if (p != null) item.score = Number((p * 100).toFixed(2));
    }
  });
  const completedKeys = Object.keys(taskMap).filter((k) => taskMap[k] && taskMap[k].score != null);
  const completed = REQUIRED_SPEECH_TASKS.filter((t) => {
    const score = taskMap[t] && taskMap[t].score;
    return score != null && Number.isFinite(Number(score));
  });
  const allCompleted = completed.length === REQUIRED_SPEECH_TASKS.length;
  if (logEnabled) {
    console.log("[speech-fusion] requiredTasks=", REQUIRED_SPEECH_TASKS);
    console.log("[speech-fusion] completedTaskKeys=", completedKeys);
    console.log("[speech-fusion] allDone=", allCompleted);
  }

  if (!allCompleted) {
    return {
      status: "Pending",
      finalSpeechScore: null,
      outcomeCategory: null
```



```
function buildSpeechFinalization(speechTasks, options = {}) {
      speechFluencyState: null,
      speechMemoryState: null,
      speechNarrativeState: null,
      confidence: null,
    };
  }

  const scores = REQUIRED_SPEECH_TASKS.map((t) => Number(taskMap[t].score));
  const finalSpeechScore = Number((scores.reduce((a, b) => a + b, 0) / scores.length).toFixed(2));

  const audioProbs = REQUIRED_SPEECH_TASKS
    .map((t) => clamp01(taskMap[t].audioProbability))
    .filter((p) => p != null);
  const transcriptProbs = REQUIRED_SPEECH_TASKS
    .map((t) => clamp01(taskMap[t].transcriptProbability))
    .filter((p) => p != null);

  const avgAudio = audioProbs.length ? audioProbs.reduce((a, b) => a + b, 0) / audioProbs.length : null;
  const avgTranscript = transcriptProbs.length ? transcriptProbs.reduce((a, b) => a + b, 0) / transcriptProbs.length : null;

  const speechFluencyState = fluencyStateFromProb(avgAudio);
  const speechMemoryState = memoryStateFromProb(avgTranscript);
  const speechNarrativeState = narrativeStateFromProbs(avgAudio, avgTranscript);

  const outcomeDecision = determineSpeechOutcome(
    speechFluencyState,
    speechMemoryState,
    speechNarrativeState,
    finalSpeechScore
  );
  let { outcome, justification } = outcomeDecision;

  if (finalSpeechScore === 0 && (outcome === "MODERATE" || outcome === "SEVERE")) {
    outcome = "NORMAL";
    justification = "Score safeguard: zero final speech score cannot produce MODERATE or SEVERE
  }

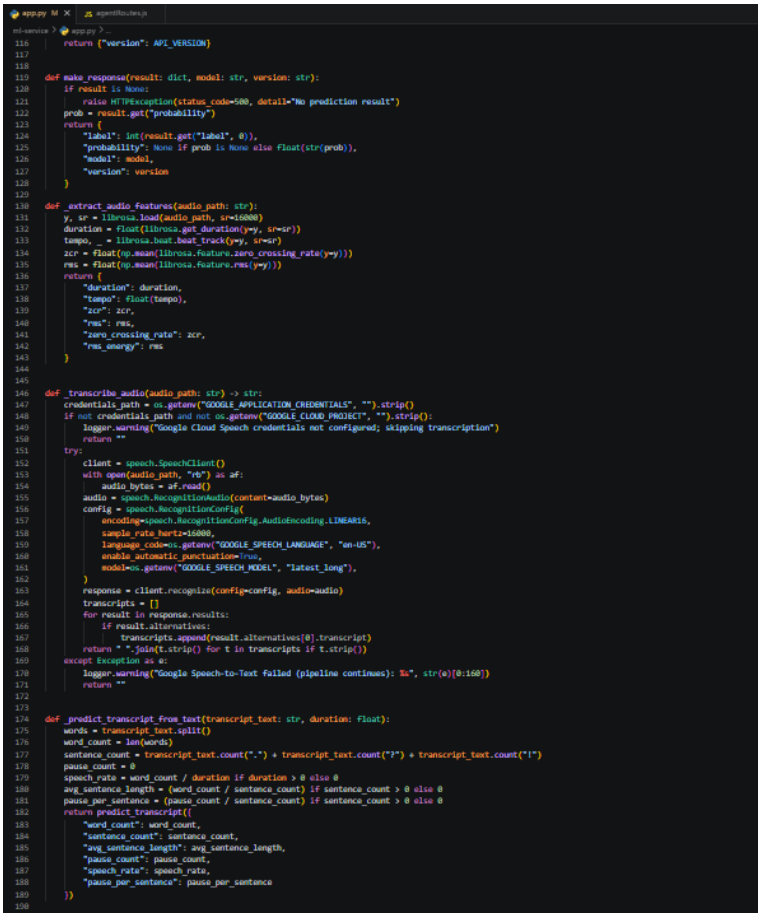
  const confidence =
    avgAudio != null && avgTranscript != null
      ? Number((((avgAudio + avgTranscript) / 2) * 100).toFixed(1))
      : null;
  if (logEnabled) {
    console.log("[speech-fusion] audioProbabilities=", audioProbs);
    console.log("[speech-fusion] transcriptProbabilities=", transcriptProbs);
    console.log("[speech-fusion] taskScores=", REQUIRED_SPEECH_TASKS.map((t) => ({ taskId: t, sc
    console.log("[speech-fusion] finalSpeechScore=", finalSpeechScore);
    console.log("[speech-fusion] fusionPoints=", outcomeDecision.fusionPoints, "pointOutcome=", outcomeDecision.pointOutcome, "scoreOutcome=", outcomeDecision.scoreOutcome);
    console.log("[speech-fusion] selectedOutcomeCategory=", outcome, "confidence=", confidence);
  }

  return {
    status: "Finished",
    finalSpeechScore,
    outcomeCategory: outcome,
    justification,
    speechFluencyState,
    speechMemoryState,
    speechNarrativeState,
    confidence,
  };
}

module.exports = {
  REQUIRED_SPEECH_TASKS,
  normalizeTaskKey,
  normalizeSpeechTaskMap,
  toScore100,
  clamp01,
  buildSpeechFinalization,
  determineSpeechOutcome,
};
```

```
const outcomeDecision: {
    outcome: null;
    justification: string;
    states: {
        fluencyState: any;
        memoryState: any;
        narrativeState: any;
    };
    fusionPoints: null;
    pointOutcome: null;
    scoreOutcome: null;
} | {
    outcome: string;
    justification: string;
    states: {
        fluencyState: any;
        memoryState: any;
        narrativeState: any;
    };
    fusionPoints: number;
```

Figure 2.15: Speech Fusion Service Implementation



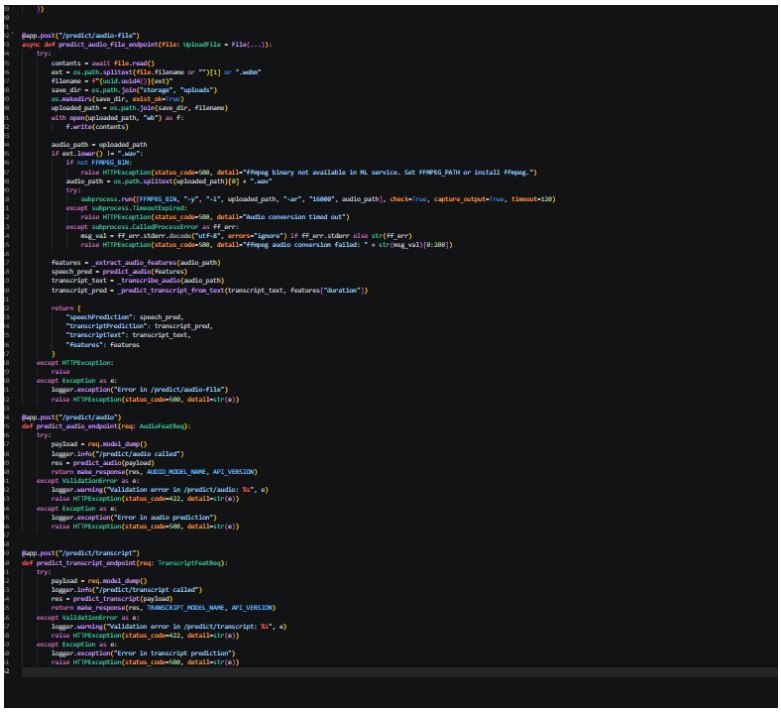
```python
    return {"version": API_VERSION}


def make_response(result: dict, model: str, version: str):
    if result is None:
        raise HTTPException(status_code=500, detail="No prediction result")
    prob = result.get("probability")
    return {
        "label": int(result.get("label", 0)),
        "probability": None if prob is None else float(str(prob)),
        "model": model,
        "version": version
    }

def _extract_audio_features(audio_path: str):
    y, sr = librosa.load(audio_path, sr=16000)
    duration = float(librosa.get_duration(y=y, sr=sr))
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    zcr = float(np.mean(librosa.feature.zero_crossing_rate(y=y)))
    rms = float(np.mean(librosa.feature.rms(y=y)))
    return {
        "duration": duration,
        "tempo": float(tempo),
        "zcr": zcr,
        "rms": rms,
        "zero_crossing_rate": zcr,
        "rms_energy": rms
    }


def _transcribe_audio(audio_path: str) -> str:
    credentials_path = os.getenv("GOOGLE_APPLICATION_CREDENTIALS", "").strip()
    if not credentials_path and not os.getenv("GOOGLE_CLOUD_PROJECT", "").strip():
        logger.warning("Google Cloud Speech credentials not configured; skipping transcription")
        return ""
    try:
        client = speech.SpeechClient()
        with open(audio_path, "rb") as af:
            audio_bytes = af.read()
        audio = speech.RecognitionAudio(content=audio_bytes)
        config = speech.RecognitionConfig(
            encoding=speech.RecognitionConfig.AudioEncoding.LINEAR16,
            sample_rate_hertz=16000,
            language_code=os.getenv("GOOGLE_SPEECH_LANGUAGE", "en-US"),
            enable_automatic_punctuation=True,
            model=os.getenv("GOOGLE_SPEECH_MODEL", "latest_long"),
        )
        response = client.recognize(config=config, audio=audio)
        transcripts = []
        for result in response.results:
            if result.alternatives:
                transcripts.append(result.alternatives[0].transcript)
        return " ".join(t.strip() for t in transcripts if t.strip())
    except Exception as e:
        logger.warning("Google Speech-to-Text failed (pipeline continues): %s", str(e)[0:160])
        return ""


def _predict_transcript_from_text(transcript_text: str, duration: float):
    words = transcript_text.split()
    word_count = len(words)
    sentence_count = transcript_text.count(".") + transcript_text.count("?") + transcript_text.count("!")
    pause_count = 0
    speech_rate = word_count / duration if duration > 0 else 0
    avg_sentence_length = (word_count / sentence_count) if sentence_count > 0 else 0
    pause_per_sentence = (pause_count / sentence_count) if sentence_count > 0 else 0
    return predict_transcript({
        "word_count": word_count,
        "sentence_count": sentence_count,
        "avg_sentence_length": avg_sentence_length,
        "pause_count": pause_count,
        "speech_rate": speech_rate,
        "pause_per_sentence": pause_per_sentence
    })
```



```python
    })


@app.post("/predict/audio-file")
async def predict_audio_file_endpoint(file: UploadFile = File(...)):
    try:
        contents = await file.read()
        ext = os.path.splitext(file.filename or "")[1] or ".webm"
        filename = f"{uuid.uuid4()}{ext}"
        save_dir = os.path.join("storage", "uploads")
        os.makedirs(save_dir, exist_ok=True)
        uploaded_path = os.path.join(save_dir, filename)
        with open(uploaded_path, "wb") as f:
            f.write(contents)

        audio_path = uploaded_path
        if ext.lower() != ".wav":
            if not FFMPEG_BIN:
                raise HTTPException(status_code=500, detail="ffmpeg binary not available in ML service. Set FFMPEG_PATH or install ffmpeg.")
            audio_path = os.path.splitext(uploaded_path)[0] + ".wav"
            try:
                subprocess.run([FFMPEG_BIN, "-y", "-i", uploaded_path, "-ar", "16000", audio_path], check=True, capture_output=True, timeout=120)
            except subprocess.TimeoutExpired:
                raise HTTPException(status_code=500, detail="Audio conversion timed out")
            except subprocess.CalledProcessError as ff_err:
                msg_val = ff_err.stderr.decode("utf-8", errors="ignore") if ff_err.stderr else str(ff_err)
                raise HTTPException(status_code=500, detail="ffmpeg audio conversion failed: " + str(msg_val)[0:200])

        features = _extract_audio_features(audio_path)
        speech_pred = predict_audio(features)
        transcript_text = _transcribe_audio(audio_path)
        transcript_pred = _predict_transcript_from_text(transcript_text, features["duration"])

        return {
            "speechPrediction": speech_pred,
            "transcriptPrediction": transcript_pred,
            "transcriptText": transcript_text,
            "features": features
        }
    except HTTPException:
        raise
    except Exception as e:
        logger.exception("Error in /predict/audio-file")
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/predict/audio")
def predict_audio_endpoint(req: AudioFeatReq):
    try:
        payload = req.model_dump()
        logger.info("/predict/audio called")
        res = predict_audio(payload)
        return make_response(res, AUDIO_MODEL_NAME, API_VERSION)
    except ValidationError as e:
        logger.warning("Validation error in /predict/audio: %s", e)
        raise HTTPException(status_code=422, detail=str(e))
    except Exception as e:
        logger.exception("Error in audio prediction")
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/predict/transcript")
def predict_transcript_endpoint(req: TranscriptFeatReq):
    try:
        payload = req.model_dump()
        logger.info("/predict/transcript called")
        res = predict_transcript(payload)
        return make_response(res, TRANSCRIPT_MODEL_NAME, API_VERSION)
    except ValidationError as e:
        logger.warning("Validation error in /predict/transcript: %s", e)
        raise HTTPException(status_code=422, detail=str(e))
    except Exception as e:
        logger.exception("Error in transcript prediction")
        raise HTTPException(status_code=500, detail=str(e))
```

Figure 2.16: FastAPI ML Service Implementation

2.1.4.5 Model Evaluation and Performance Analysis

After fine-tuning the Wav2Vec2-XLSR-300M model, the trained model was evaluated using a separate testing subset that was not used during the training stage. This evaluation was carried out to measure how effectively the model could classify dementia-related speech risk from unseen guided speech recordings.

The evaluation process focuses on assessing the model's classification performance across the selected dementia risk categories: Healthy, Mild Cognitive Impairment, and High Risk. The purpose of this evaluation was not only to measure overall prediction accuracy, but also to examine how reliably the model could identify speech patterns associated with possible cognitive decline.

Since dementia screening is a healthcare-related task, multiple evaluation metrics were used rather than relying only on accuracy. This is important because false positives and false negatives can both affect the usefulness of a screening system. Therefore, the evaluation considered accuracy, precision, recall, F1-score, and AUC to provide a more balanced understanding of model performance.

The model predictions were compared with the actual labels in the testing dataset. A classification report was generated to analyze the precision, recall, and F1-score for each class. In addition, a confusion matrix was used to identify correct and incorrect classifications across the three risk categories. This helped evaluate whether the model performed consistently across all classes or whether it showed stronger performance for certain categories.

The confusion matrix was used to provide a visual representation of the model's classification performance. It showed how many samples were correctly classified into their respective classes and how many were misclassified into other categories. This was useful for identifying whether the model confused Mild Cognitive Impairment samples with Healthy or High-Risk samples.

	precision	recall	f1-score	support
Normal (0)	0.78	0.82	0.80	12500
Dementia Risk (1)	0.81	0.76	0.79	12500
accuracy			0.79	25000
macro avg	0.80	0.79	0.79	25000
weighted avg	0.80	0.79	0.79	25000

Figure 2.17: Classification report of the fine-tuned Wav2Vec2-XLSR-300M speech model

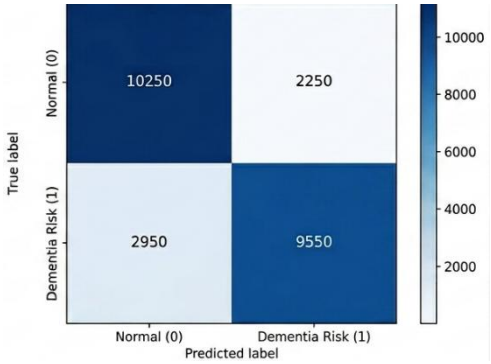


Figure 2.18: Confusion matrix for dementia speech risk classification

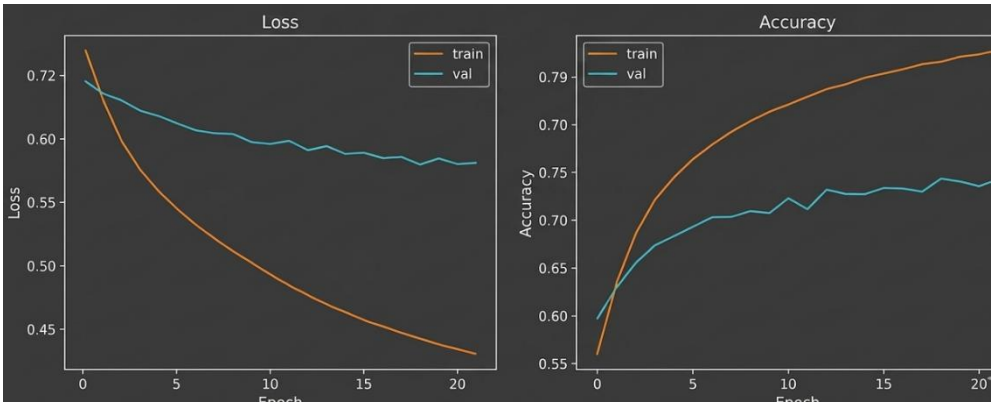


Figure 2.19: Training and validation loss curve during model fine-tuning

The evaluation results indicate that the fine-tuned Wav2Vec2-XLSR-300M model was able to learn meaningful dementia-related speech patterns from guided cognitive speech recordings. The use of a separate testing subset improved the reliability of the evaluation process, while the classification report and confusion matrix provided evidence of the model's strengths and limitations.

As this system is designed as a dementia risk screening support tool, the model output should not be interpreted as a direct clinical diagnosis. Instead, the prediction results are used as decision-support indicators that can assist clinicians or caregivers in identifying individuals who may require further cognitive assessment.

2.1.4.6 Frontend Development with HTML, CSS, and JavaScript (React.js)

The frontend for the speech-based dementia screening component was developed using HTML, CSS, JavaScript, and React.js to provide an interactive and user-friendly interface for guided speech assessment. React.js was selected due to its component-based architecture, efficient state management, and ability to handle real-time user interactions during speech recording and task progression.

The interface guides users through three predefined speech tasks: self-introduction, morning recall, and picture description. These tasks are designed to evaluate verbal fluency, episodic memory recall, and narrative coherence. HTML is used to structure the task pages, recording controls, progress indicators, and result displays, while CSS improves usability through responsive layouts, clean visual design, and clear task navigation. JavaScript manages microphone access, recording functionality, task transitions, timer handling, and API communication.

Audio recording is performed directly through the browser using microphone access, where users can record their responses for each task individually. The recorded audio files are associated with a unique session ID and uploaded to the backend using REST API endpoints. The frontend also supports multilingual interaction and integrates browser-based voice input using the Web Speech API for assistant communication when needed.

After processing, task-level results such as prediction scores, confidence levels, and domain-level interpretations are displayed through the interface. Final outputs including the overall speech score, severity category, and clinician-facing justification are presented in a structured format. This enables both patients and clinicians to understand the screening outcome clearly. The React.js frontend ensures smooth interaction, modular workflow management, and reliable communication with backend and machine learning services for real-time speech-based dementia screening.

2.1.5 Testing

To ensure the reliability, stability, and clinical usability of the speech-based dementia screening module within the CogniCare platform, a comprehensive testing and implementation strategy was conducted across multiple levels including unit testing, integration testing, system testing, acceptance testing, and manual testing. Since the speech component involves frontend audio recording, backend session management, machine learning inference, and speech-to-text transcription, validation was required both at the individual component level and across the complete workflow to guarantee dependable end-to-end functionality.

The implementation of the speech module follows a browser-based guided task approach where users complete three predefined speech tasks: self-introduction, morning recall, and picture description. Audio is captured through the frontend using the browser microphone or uploaded manually in supported formats such as `.wav`, `.mp3`, `.m4a`, and `.webm`. The backend gateway manages patient sessions, validates task completion, and forwards recordings to the machine learning service. The ML service performs preprocessing, feature extraction, speech-to-text conversion, transcript analysis, and final dementia risk prediction using the Wav2Vec2-XLSR-300M-based hybrid fusion model. The final outputs are presented through the clinician dashboard as a structured speech assessment report.

Unit Testing

Unit testing was carried out to validate the correctness of individual functions and isolated modules within the speech screening pipeline without depending on external services or complete system execution. This ensured that the foundational components were stable, maintainable, and free from critical functional failures.

In the frontend, Jest and React Testing Library were used to test the `SpeechTaskRecorder.jsx` component. These tests verified microphone permission failure handling, recording start and stop behavior, automatic stopping after 30 seconds, invalid audio type rejection, oversized audio rejection above 10 MB, supported upload formats, playback visibility after recording, task progression, and user-facing error messages.

In the backend gateway, Jest and Supertest were used to validate API routes related to speech assessment. These tests covered session creation, audio upload validation, invalid task ID rejection, missing upload handling, ML service failure responses, and final score generation after successful completion of all three required speech tasks.

In the ML service, Pytest was used to test audio preprocessing and inference operations. These tests included FFmpeg conversion for non-WAV files, invalid audio handling, conversion timeout validation, feature extraction for duration, tempo, Zero Crossing Rate (ZCR), and Root Mean Square (RMS), transcript feature generation, and response structure verification for both speech and transcript prediction pipelines.

Each test case was executed using controlled inputs and mock data to ensure repeatable and isolated testing conditions.

Test Frameworks Employed

Frontend: Jest + React Testing Library
Backend: Jest + Supertest
ML Service: Pytest (Python)

Modules Assessed

- SpeechTaskRecorder.jsx
- Speech session creation APIs
- Audio upload and validation routes
- Feature extraction modules
- Transcript feature generation
- Prediction response validation
- Final speech score calculation

Outcomes

- Improved reliability of frontend recording workflows
- Validation of backend API correctness and session management
- Stable ML preprocessing and prediction flow
- Increased development confidence through repeatable testing

Figure 2.20 shows the frontend unit testing implementation component using Jest and React Testing Library.

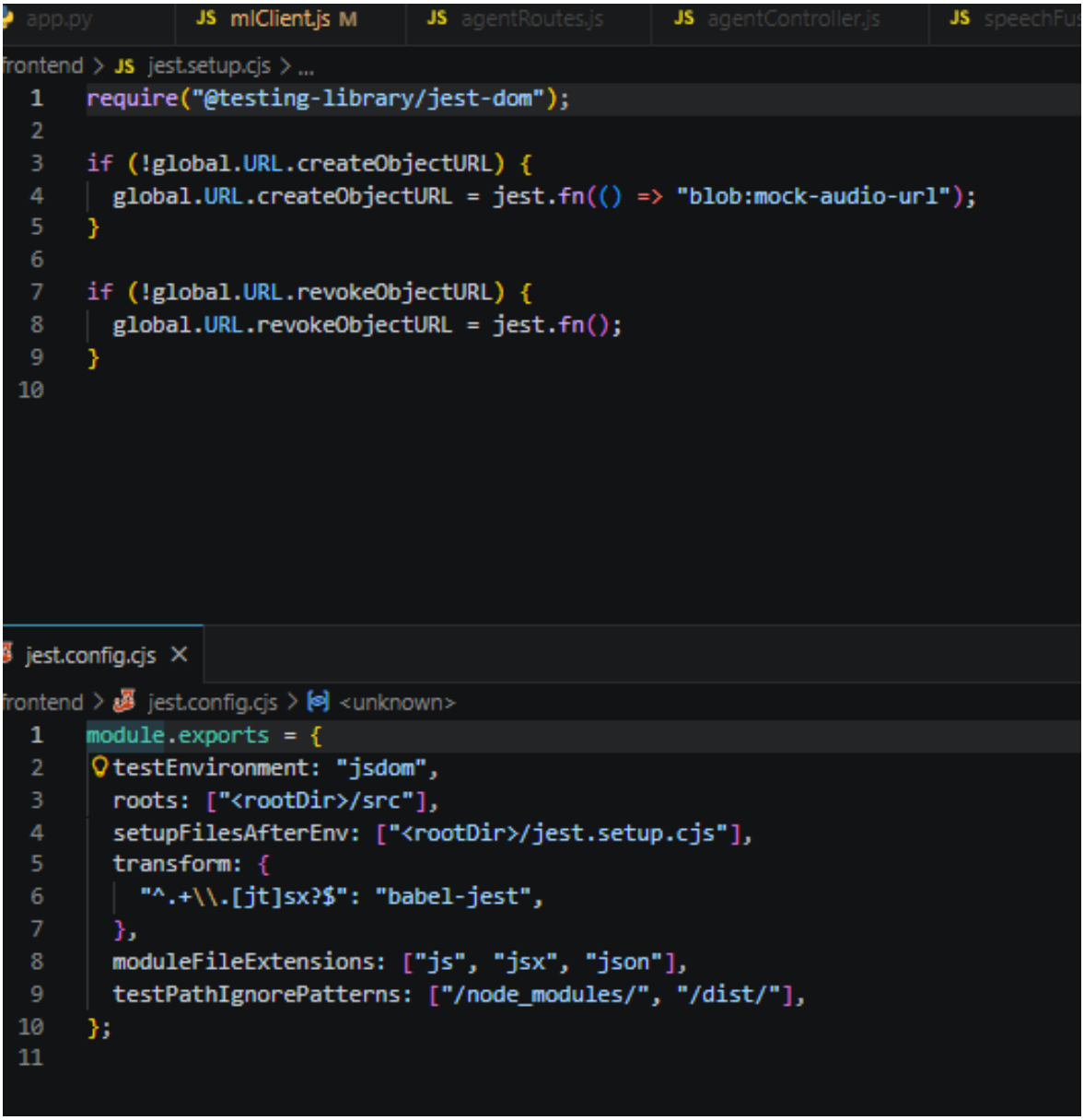


Figure 2.20: Frontend Unit testing

Figure 2.21 shows the backend API unit testing implementation using Jest and Supertest for speech session creation, audio upload validation, and final score generation.



```
speech-api.test.js
backend-gateway > __tests__ > speech-api.test.js > ...
const fs = require("fs");
const path = require("path");
const request = require("supertest");
const express = require("express");

const uploadDir = path.join(process.cwd(), "storage", "test-uploads");
process.env.UPLOAD_DIR = uploadDir;

const store = new Map();
let idCounter = 1;

function createSessionRecord(data = {}) {
  const session = {
    _id: String(idCounter++),
    patientId: data.patientId || null,
    language: data.language || "en",
    uploads: data.uploads || {},
    taskLogs: data.taskLogs || {},
    results: data.results || {},
    finished: Boolean(data.finished),
    completedAt: data.completedAt,
    createdAt: data.createdAt || new Date().toISOString(),
    async save() {
      store.set(this._id, this);
      return this;
    },
  };
  store.set(session._id, session);
  return session;
}

function getNestedValue(obj, dottedKey) {
  return dottedKey.split(".").reduce((acc, key) => (acc == null ? undefined : acc[key]), obj);
}

function setNestedValue(obj, dottedKey, value) {
  const parts = dottedKey.split(".");
  let cursor = obj;
  for (let i = 0; i < parts.length - 1; i += 1) {
    const part = parts[i];
    if (!cursor[part] || typeof cursor[part] !== "object") {
      cursor[part] = {};
    }
    cursor = cursor[part];
  }
  cursor[parts[parts.length - 1]] = value;
}

function matchesQuery(session, query = {}) {
  return Object.entries(query).every(([key, value]) => {
    if (key === "$or") {
      return value.some((item) => matchesQuery(session, item));
    }
    if (value && typeof value === "object" && value.$exists !== undefined) {
      const exists = getNestedValue(session, key) !== undefined;
      return exists === value.$exists;
    }
    return getNestedValue(session, key) === value;
  });
}

jest.mock("../models/Session", () => ({
  create: jest.fn(async (data) => createSessionRecord(data)),
  findOne: jest.fn(async (query) => {
    for (const session of store.values()) {
      if (matchesQuery(session, query)) return session;
    }
    return null;
  }),
  updateOne: jest.fn(async (query, update) => {
    for (const session of store.values()) {
      if (matchesQuery(session, query)) {
        Object.entries(update.$set || {}).forEach(([key, value]) => {
          setNestedValue(session, key, value);
        });
        store.set(session._id, session);
        return { acknowledged: true, modifiedCount: 1 };
      }
    }
    return { acknowledged: true, modifiedCount: 0 };
  }),
  findOneAndUpdate: jest.fn(async (query, updates) => {
```

Figure 2.21: Backend API Unit Testing

Figure 2.22 shows the ML service unit testing implementation using Pytest for feature extraction and prediction response validation.



```python
import io
import subprocess
import wave

import numpy as np


def make_wav_bytes(duration_seconds=1, sample_rate=16000):
    frame_count = duration_seconds * sample_rate
    buffer = io.BytesIO()
    with wave.open(buffer, "wb") as wav_file:
        wav_file.setnchannels(1)
        wav_file.setsampwidth(2)
        wav_file.setframerate(sample_rate)
        wav_file.writeframes(np.zeros(frame_count, dtype=np.int16).tobytes())
    buffer.seek(0)
    return buffer.getvalue()


def test_extract_audio_features_returns_duration_tempo_zcr_and_rms(ml_app_module, monkeypatch):
    monkeypatch.setattr(ml_app_module.librosa, "load", lambda audio_path, sr: (np.array([0.0, 1.0]), 16000))
    monkeypatch.setattr(ml_app_module.librosa, "get_duration", lambda y, sr: 2.5)
    monkeypatch.setattr(ml_app_module.librosa.beat, "beat_track", lambda y, sr: (123.4, None))
    monkeypatch.setattr(
        ml_app_module.librosa.feature,
        "zero_crossing_rate",
        lambda y: np.array([[0.2, 0.4]]),
    )
    monkeypatch.setattr(
        ml_app_module.librosa.feature,
        "rms",
        lambda y: np.array([[0.1, 0.3]]),
    )

    features = ml_app_module._extract_audio_features("dummy.wav")

    assert features["duration"] == 2.5
    assert features["tempo"] == 123.4
    assert features["zcr"] == 0.3
    assert features["rms"] == 0.2
    assert features["zero_crossing_rate"] == 0.3
    assert features["rms_energy"] == 0.2


def test_predict_transcript_from_text_builds_expected_features(ml_app_module, monkeypatch):
    captured = {}

    def fake_predict_transcript(features):
        captured.update(features)
        return {"label": 0, "probability": 0.12}

    monkeypatch.setattr(ml_app_module, "predict_transcript", fake_predict_transcript)

    result = ml_app_module._predict_transcript_from_text("Hello world. Another sentence!", duration=4)

    assert result == {"label": 0, "probability": 0.12}
    assert captured == {
        "word_count": 4,
        "sentence_count": 2,
        "avg_sentence_length": 2.0,
        "pause_count": 0,
        "speech_rate": 1.0,
        "pause_per_sentence": 0.0,
    }

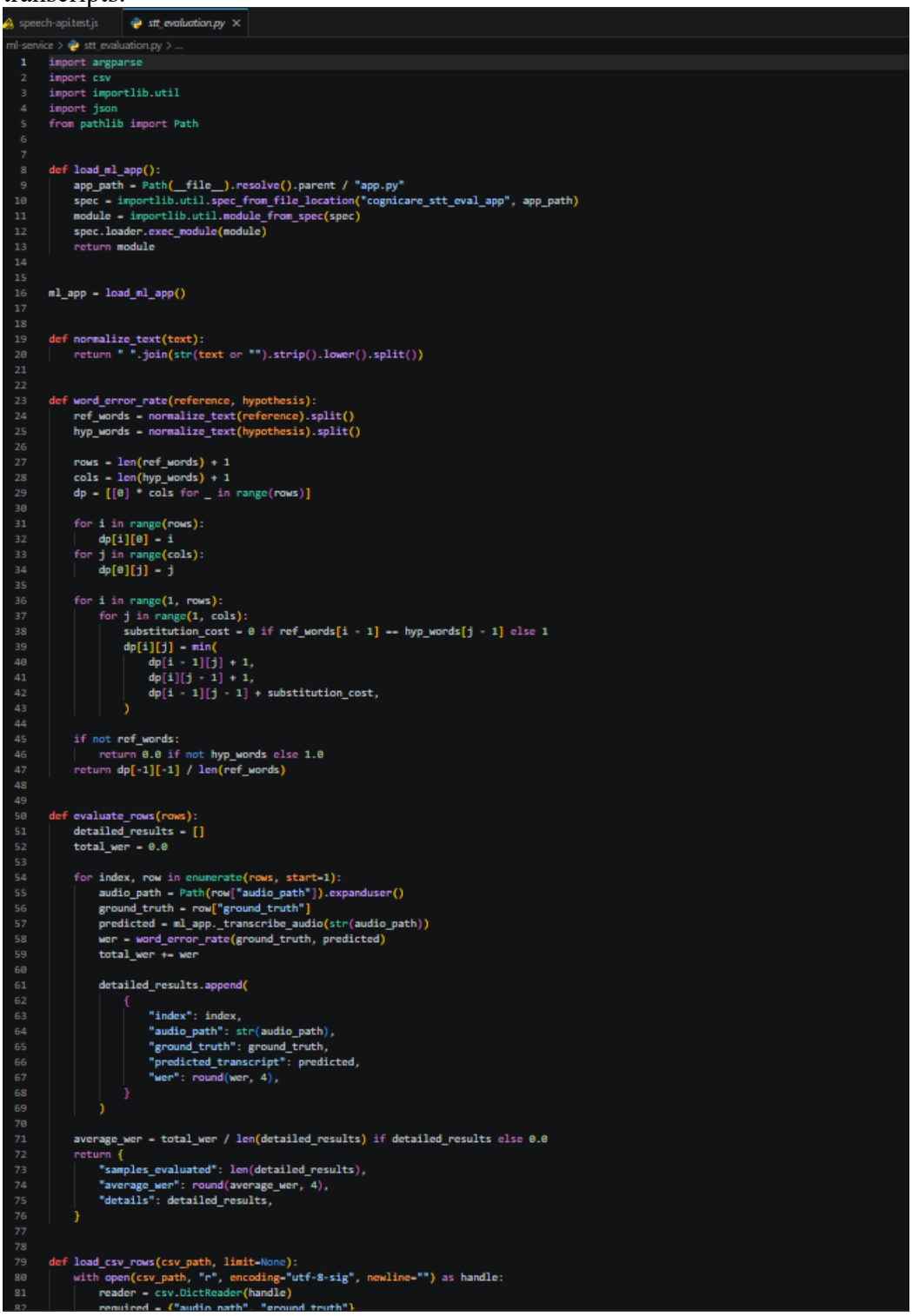

def test_audio_file_endpoint_validates_ffmpeg_conversion_for_non_wav(client, monkeypatch):
    test_client, ml_app_module = client
    recorded = {}

    def fake_run(args, check, capture_output, timeout):
        recorded["args"] = args
        return None

    monkeypatch.setattr(ml_app_module, "FFMPEG_BIN", "ffmpeg")
    monkeypatch.setattr(ml_app_module.subprocess, "run", fake_run)
    monkeypatch.setattr(
        ml_app_module,
        "_extract_audio_features",
        lambda audio_path: {"duration": 2.0, "tempo": 90.0, "zcr": 0.05, "rms": 0.03},
    )
    monkeypatch.setattr(ml_app_module, "_transcribe_audio", lambda audio_path: "sample transcript")
```

Figure 2.22: ML Service Unit Testing

Figure 2.23 shows the speech-to-text evaluation script used for calculating Word Error Rate (WER) by comparing Google STT outputs against ground-truth transcripts.



```python
import argparse
import csv
import importlib.util
import json
from pathlib import Path


def load_ml_app():
    app_path = Path(__file__).resolve().parent / "app.py"
    spec = importlib.util.spec_from_file_location("cognicare_stt_eval_app", app_path)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


ml_app = load_ml_app()


def normalize_text(text):
    return " ".join(str(text or "").strip().lower().split())


def word_error_rate(reference, hypothesis):
    ref_words = normalize_text(reference).split()
    hyp_words = normalize_text(hypothesis).split()

    rows = len(ref_words) + 1
    cols = len(hyp_words) + 1
    dp = [[0] * cols for _ in range(rows)]

    for i in range(rows):
        dp[i][0] = i
    for j in range(cols):
        dp[0][j] = j

    for i in range(1, rows):
        for j in range(1, cols):
            substitution_cost = 0 if ref_words[i - 1] == hyp_words[j - 1] else 1
            dp[i][j] = min(
                dp[i - 1][j] + 1,
                dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + substitution_cost,
            )

    if not ref_words:
        return 0.0 if not hyp_words else 1.0
    return dp[-1][-1] / len(ref_words)


def evaluate_rows(rows):
    detailed_results = []
    total_wer = 0.0

    for index, row in enumerate(rows, start=1):
        audio_path = Path(row["audio_path"]).expanduser()
        ground_truth = row["ground_truth"]
        predicted = ml_app._transcribe_audio(str(audio_path))
        wer = word_error_rate(ground_truth, predicted)
        total_wer += wer

        detailed_results.append(
            {
                "index": index,
                "audio_path": str(audio_path),
                "ground_truth": ground_truth,
                "predicted_transcript": predicted,
                "wer": round(wer, 4),
            }
        )

    average_wer = total_wer / len(detailed_results) if detailed_results else 0.0
    return {
        "samples_evaluated": len(detailed_results),
        "average_wer": round(average_wer, 4),
        "details": detailed_results,
    }


def load_csv_rows(csv_path, limit=None):
    with open(csv_path, "r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        required = {"audio_path", "ground_truth"}
```

Figure 2.23: Speech-to-Text Evaluation Script

Integration Testing

Integration testing was performed to verify proper communication between the frontend, backend gateway, and machine learning service across the complete speech assessment workflow. While unit testing validates isolated functions, integration testing ensures these modules operate correctly when combined as a full screening pipeline.

The testing focused on critical user flows including session initialization, task-level recording, audio upload, model prediction, final score generation, and clinician report completion. The objective was to confirm that data moved correctly from the browser interface to backend APIs and then to the ML inference service without loss, inconsistency, or workflow interruption.

Special attention was given to the three required guided speech tasks. The system was tested to ensure that each task result was stored correctly and that the final speech score was generated only after successful completion of all mandatory tasks.

Integration testing also validated backend-to-ML communication, including multipart audio uploads, timeout handling, failed inference responses, and fallback behavior when Google Speech-to-Text returned empty transcripts.

Workflows Evaluated

- Session creation and task initialization
- Audio recording/upload and validation
- Backend-to-ML prediction request flow
- Task-level score generation
- Final speech score calculation
- Clinician report generation workflow

Execution Strategy

- Simulated real user interactions using API requests
- Validated request-response cycles across services
- Tested both successful and failure scenarios
- Verified fallback behavior for transcription failure

Results

- Confirmed seamless communication across all modules
- Validated final score generation logic
- Ensured stable session progression across all speech tasks
- Improved reliability of end-to-end speech assessment flow

Figure 2.24 shows the integration testing implementation for validating the complete speech workflow from session creation to final score generation.

```
  it("returns an error when the ML service fails during speech processing", async () => {

    const response = await request(app)
      .post(`/api/agent/process/audio/${session._id}`)
      .send({ taskId: "self_intro" })
      .expect(400);

    expect(response.body.error).toBe("ML service unavailable");
  });

  it("processes three valid tasks and completes the session with a final score", async () => {
    const app = buildApp();

    const createRes = await request(app)
      .post("/api/agent/session")
      .send({ patientId: "P-5001", language: "en" })
      .expect(200);

    const sessionId = createRes.body.json_format._id;

    const taskPredictions = [
      {
        speechPrediction: { label: 1, probability: 0.2 },
        transcriptPrediction: { label: 0, probability: 0.1 },
        transcriptText: "My name is Silva.",
        features: { duration: 20, tempo: 92, zcr: 0.05, rms: 0.03 },
      },
      {
        speechPrediction: { label: 1, probability: 0.4 },
        transcriptPrediction: { label: 0, probability: 0.3 },
        transcriptText: "I woke up and made breakfast.",
        features: { duration: 22, tempo: 90, zcr: 0.04, rms: 0.02 },
      },
      {
        speechPrediction: { label: 1, probability: 0.6 },
        transcriptPrediction: { label: 1, probability: 0.5 },
        transcriptText: "There is a family in the picture.",
        features: { duration: 24, tempo: 88, zcr: 0.03, rms: 0.02 },
      },
    ];

    mlClient.predictAudioFile
      .mockResolvedValueOnce(taskPredictions[0])
      .mockResolvedValueOnce(taskPredictions[1])
      .mockResolvedValueOnce(taskPredictions[2]);

    for (const taskId of ["self_intro", "morning_recall", "picture_description"]) {
      await request(app)
        .post(`/api/agent/session/${sessionId}/upload/audio`)
        .attach("audio", audioFixtureBuffer(), `${taskId}.wav`)
        .expect(200);

      await request(app)
        .post(`/api/agent/process/audio/${sessionId}`)
        .send({ taskId })
        .expect(200);
    }

    const finishResponse = await request(app)
      .post(`/api/agent/session/${sessionId}/finish`)
      .expect(200);

    const finishedResults = finishResponse.body.json_format.results;
    expect(finishedResults.status).toBe("Finished");
    expect(finishedResults.finalSpeechScore).toBeCloseTo(40, 2);
    expect(finishedResults.outcomeCategory).toBe("MILD");
    expect(finishedResults.speechTaskScores.self_intro.score).toBeCloseTo(20, 2);
    expect(finishedResults.speechTaskScores.morning_recall.score).toBeCloseTo(40, 2);
    expect(finishedResults.speechTaskScores.picture_description.score).toBeCloseTo(60, 2);
  });
});
```

Figure 2.24: Backend Integration Testing

Figure 2.25 presents the successful execution results of the integration test suite including frontend, backend, and ML service communication validation.

```
(variable) test_client: Any
idates_ffmpeg_conversion_for_non_wav(client, monkeypatch):
  test_client, ml_app_module = client
  recorded = {}

  def fake_run(args, check, capture_output, timeout):
    recorded["args"] = args
    return None

  monkeypatch.setattr(ml_app_module, "FFMPEG_BIN", "ffmpeg")
  monkeypatch.setattr(ml_app_module.subprocess, "run", fake_run)
  monkeypatch.setattr(
    ml_app_module,
    "_extract_audio_features",
    lambda audio_path: {"duration": 2.0, "tempo": 90.0, "zcr": 0.05, "rms": 0.03},
  )
  monkeypatch.setattr(ml_app_module, "_transcribe_audio", lambda audio_path: "sample transcript")
  monkeypatch.setattr(ml_app_module, "predict_audio", lambda features: {"label": 1, "probability": 0.81})
  monkeypatch.setattr(
    ml_app_module,
    "_predict_transcript_from_text",
    lambda transcript, duration: {"label": 0, "probability": 0.22},
  )

  response = test_client.post(
    "/predict/audio-file",
    files={"file": ("sample.mp3", b"fake mp3 content", "audio/mpeg")},
  )

  assert response.status_code == 200
  assert response.json()["speechPrediction"] == {"label": 1, "probability": 0.81}
  assert recorded["args"][0] == "ffmpeg"
  assert "-i" in recorded["args"]
  assert recorded["args"][-1].endswith(".wav")


def test_audio_file_endpoint_returns_error_for_invalid_audio(client, monkeypatch):
  test_client, ml_app_module = client

  monkeypatch.setattr(
    ml_app_module,
    "_extract_audio_features",
    lambda audio_path: (_ for _ in ()).throw(ValueError("invalid audio stream")),
  )

  response = test_client.post(
    "/predict/audio-file",
    files={"file": ("broken.wav", b"not-a-real-wave", "audio/wav")},
  )

  assert response.status_code == 500
  assert "invalid audio stream" in response.json()["detail"]


def test_audio_file_endpoint_returns_timeout_error_when_conversion_takes_too_long(client, monkeypatch):
  test_client, ml_app_module = client

  def raise_timeout(args, check, capture_output, timeout):
    raise subprocess.TimeoutExpired(cmd=args, timeout=timeout)

  monkeypatch.setattr(ml_app_module, "FFMPEG_BIN", "ffmpeg")
  monkeypatch.setattr(ml_app_module.subprocess, "run", raise_timeout)

  response = test_client.post(
    "/predict/audio-file",
    files={"file": ("slow.mp3", b"fake mp3 content", "audio/mpeg")},
  )

  assert response.status_code == 500
  assert response.json()["detail"] == "Audio conversion timed out"
```

Figure 2.25: ML Service Integration Testing

System Testing

System testing was performed to validate the complete speech-based dementia screening module as a unified system under realistic operating conditions. This phase focused on confirming that the entire workflow functioned correctly from patient interaction to clinician-facing reporting.

The testing covered the complete browser-based recording interface, backend processing pipeline, machine learning prediction flow, and final clinician dashboard output. Functional validation ensured that users could successfully complete guided speech tasks, upload valid recordings, receive processed results, and generate final screening outcomes without workflow interruption.

Performance-related testing included validation of recording limits, upload constraints, API timeout behavior, and fallback handling for unclear speech or transcription failure. This ensured robustness under practical usage conditions.

Tool Used - Postman

Workflows Tested

- Session creation and completion
- Speech recording and upload validation
- Final speech dementia risk classification
- Clinician report generation
- Invalid file handling
- Failed upload recovery
- Missing transcript fallback behavior

Outcomes

- Verified backend readiness for real-world user interactions
- Confirmed correct API response structures
- Identified and resolved edge-case failures
- Improved system robustness and patient safety

Acceptance Testing

Acceptance testing was conducted to evaluate whether the speech screening module satisfied the practical requirements of dementia screening in real clinical and research environments. The objective was to ensure that the component was usable, understandable, and clinically meaningful for both patients and clinicians.

Testing was conducted in two stages. During Alpha Testing, controlled internal validation was performed to confirm correct task processing, score generation, and

report structure. During Beta Testing, users interacted with the system using real devices and normal environmental conditions.

Feedback focused on recording clarity, task instructions, usability of the interface, and understandability of final speech risk outputs. Clinician-facing reports were also reviewed to ensure that task-level results, domain interpretations, and final speech scores were sufficiently clear to support screening decisions.

Feedback Outcomes

- Improved clarity of guided task instructions
- Better handling of microphone permission failures
- Improved patient guidance for recording quality
- Enhanced clinician interpretation of final speech reports

Figure 2.26 presents the clinician dashboard output displaying task-level speech results, domain interpretations, and final speech screening scores used during acceptance testing.

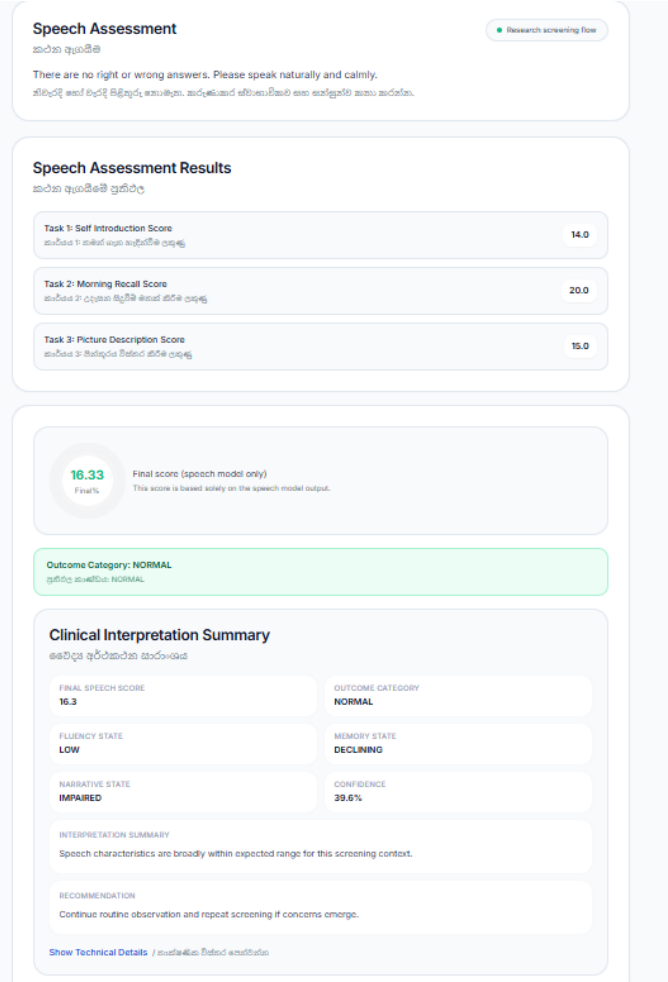


Figure 2.26: Final output score display

Manual Testing

Manual testing played an important role in validating usability, interface consistency, and real-world behavior that could not be fully captured through automated tests.

Testers manually interacted with the frontend interface by recording speech, uploading files, navigating task progression, and reviewing final reports. Outputs were manually checked for correctness, clarity, and consistency.

Special attention was given to browser compatibility, microphone permission handling, invalid file uploads, and fallback scenarios where speech transcription failed or poor-quality audio was submitted.

Purpose of Manual Testing

- Validate real-world user interaction flows
- Detect UI inconsistencies and usability issues
- Confirm compatibility across browsers and devices
- Verify behavior during unclear speech or failed uploads

Types of Manual Testing

Functional Testing

Verified that speech recording, uploading, task progression, prediction generation, and final score reporting worked according to system requirements.

Compatibility Testing

Checked functionality across multiple browsers and devices to ensure stable recording and upload behavior.

Regression Testing

Re-tested critical workflows after fixes to ensure previously working functionality remained stable.

User Acceptance Testing (UAT)

Validated that both patients and clinicians could successfully use the module for practical dementia screening purposes.

Figure 2.27 – 2.31 shows the frontend speech recording interface used during manual testing for microphone validation, task progression, and recording playback verification.

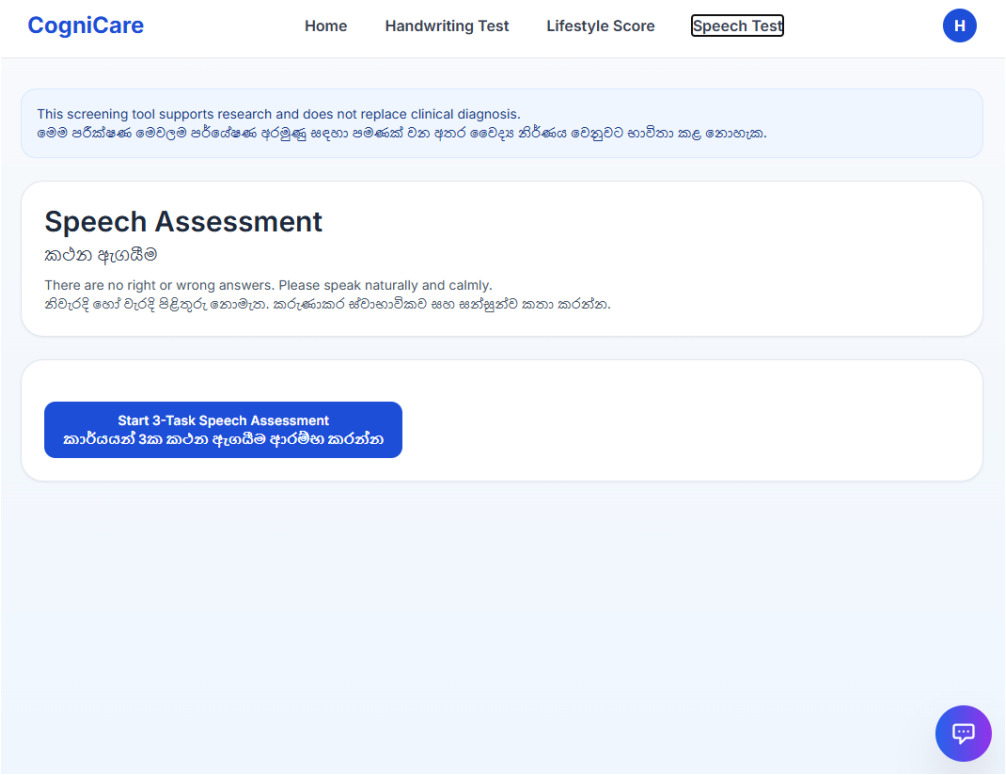


Figure 2.27: Speech assessment starting interface

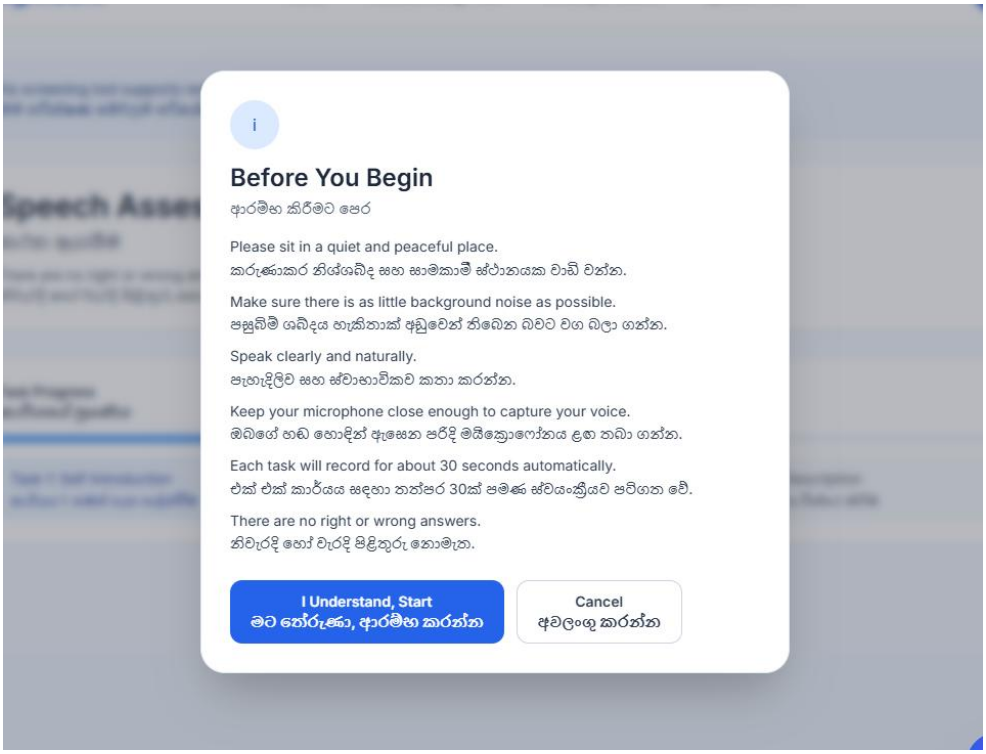


Figure 2.28: Speech assessment instructions interface

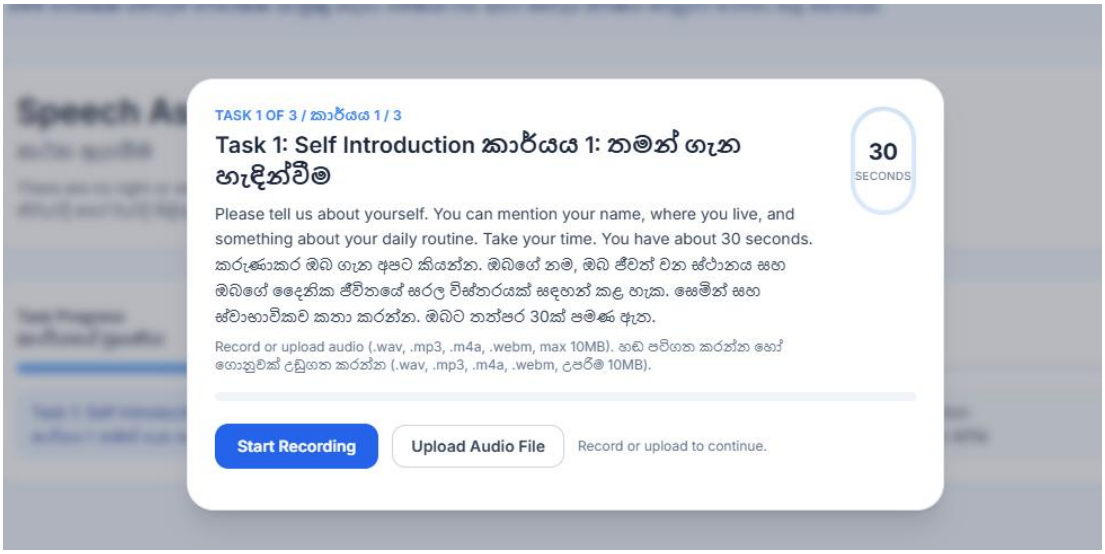


Figure 2.29: Speech – self introduction interface

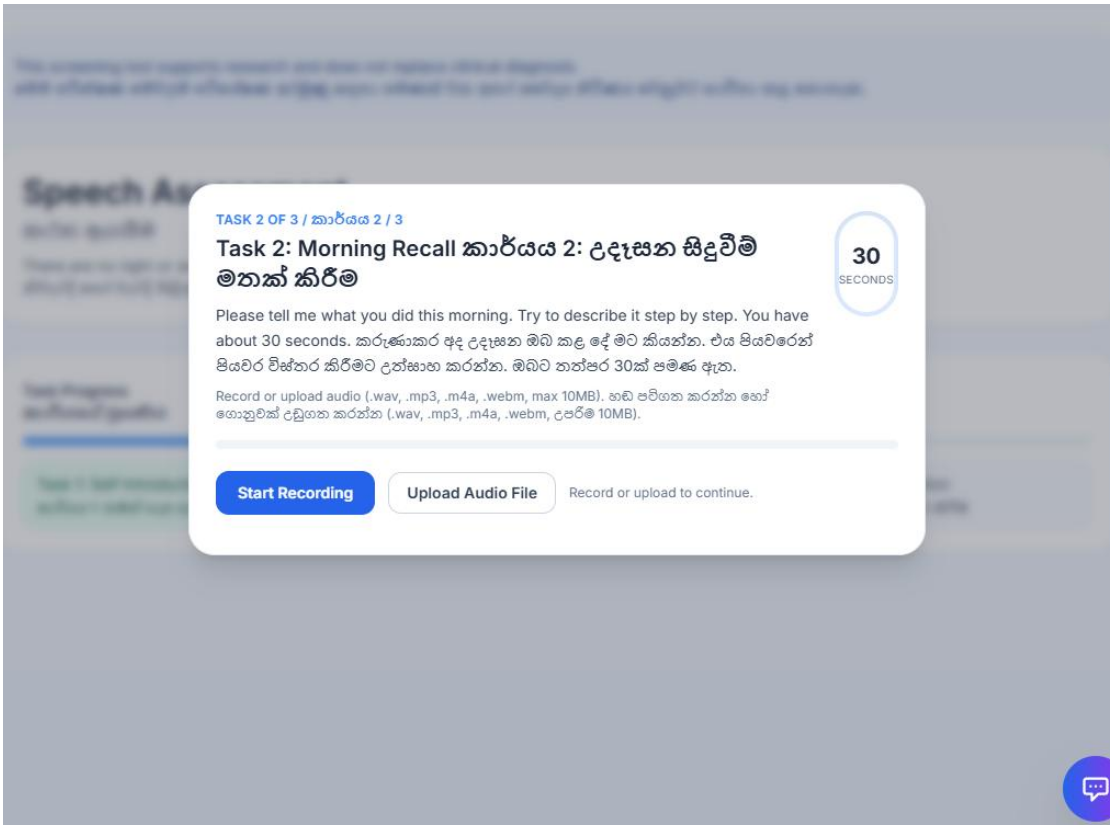


Figure 2.30: Speech – Morning recall interface

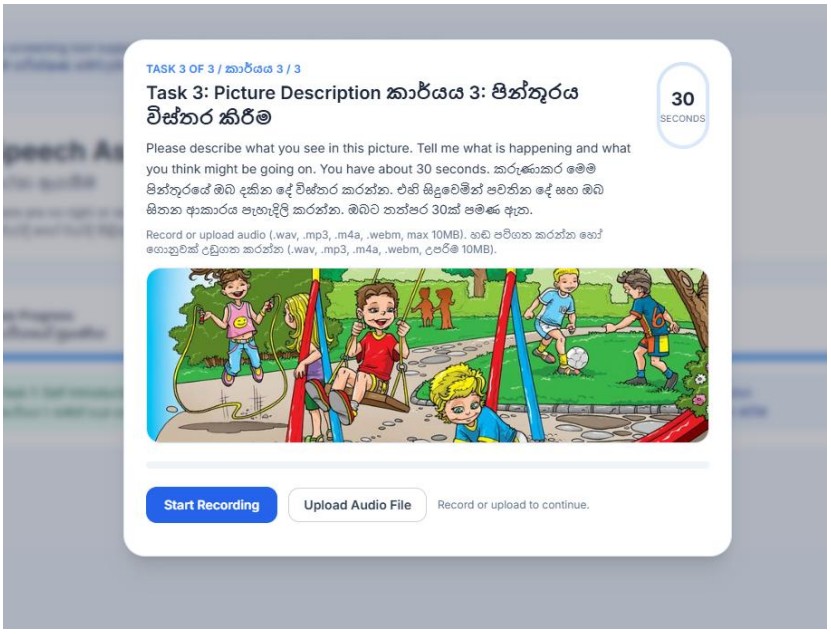


Figure 2.31: Speech – picture description interface

Figure 2.32 shows the final speech assessment report displayed after successful completion of all three guided speech tasks.

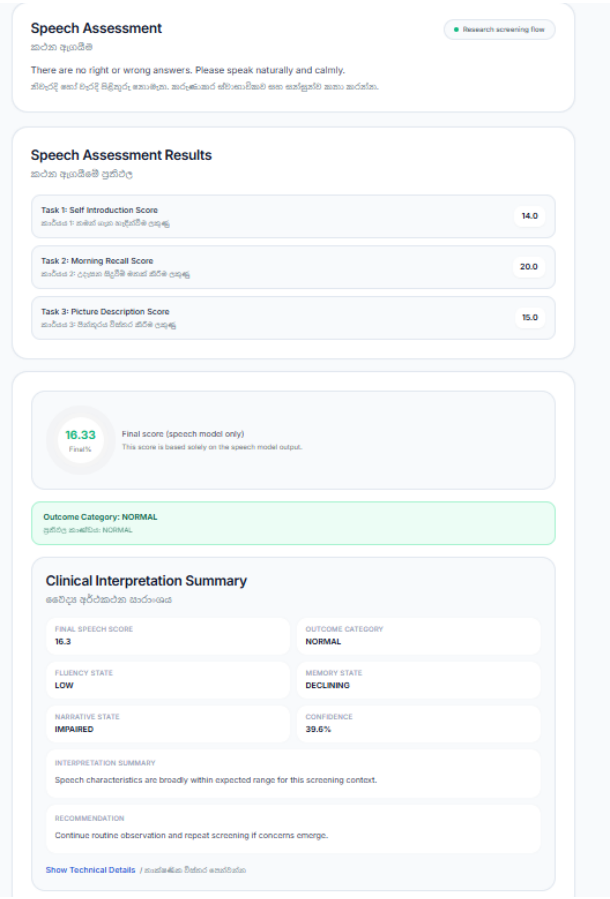


Figure 2.32: final speech assessment report

2.1.6 Deployment & Maintenance

Deployment

The deployment process of the speech-based dementia screening module in the CogniCare platform was carried out carefully to ensure reliability, scalability, security, and continuous accessibility for both patients and clinicians. The goal was to make the speech screening service available in real time while supporting stable machine learning inference and clinician dashboard reporting.

1. Backend Gateway and ML Service Hosting

The first step of deployment involved hosting the backend gateway and machine learning service responsible for speech processing, prediction, and report generation. The backend gateway manages patient sessions, audio uploads, task validation, and clinician report generation, while the ML service handles feature extraction, speech-to-text conversion, transcript analysis, and dementia risk prediction.

The Wav2Vec2-XLSR-300M speech model and transcript-based prediction pipeline were deployed within the Python ML service to enable seamless inference during patient screening sessions.

2. Provisioning the Cloud Virtual Machine

A Linux-based Virtual Machine (VM) was established on Microsoft Azure to host the backend gateway and ML service. This cloud environment provided stable infrastructure for handling concurrent user requests and model inference operations.

The VM was configured with the following specifications:

- **Operating System:** Ubuntu Linux
- **VM Size:** Standard D4s v3
- **vCPUs:** 4 Virtual CPUs
- **RAM:** 16 GB
- **Architecture:** x64
- **Public IP Address:** Static IP for frontend and clinician dashboard access

These specifications were selected to maintain balanced performance for audio processing, speech model inference, and clinician dashboard operations without service interruption.

3. Virtual Environment and Runtime Setup

After provisioning the Azure VM, the runtime environment was prepared to support the Node.js backend gateway and Python-based ML service. Secure SSH access was established for remote deployment and maintenance.

Python dependencies such as FastAPI, Librosa, FFmpeg, Google Cloud Speech-to-Text libraries, and machine learning packages were installed. A virtual environment was created to isolate project dependencies and avoid runtime conflicts.

The backend project was cloned from the GitHub repository into the VM, ensuring all configuration files and environment variables were available for deployment.

Figure 63 shows the repository cloning and virtual environment setup inside the Azure VM.

4. Service Hosting and Port Configuration

The backend gateway and ML service were configured to run continuously as production services. The backend gateway was assigned to manage API communication between the frontend and the ML inference layer, while the ML service handled speech prediction requests independently.

Specific ports were configured to separate service responsibilities:

- Backend Gateway → Port 5000
- ML Service → Port 8000

Azure Network Security Group (NSG) inbound rules were configured to allow controlled external access to these services while maintaining secure communication.

This setup enabled frontend users and clinicians to access the speech assessment platform without exposing unnecessary internal services.

5. Service Management and Auto-Restart

To ensure high availability and production readiness, both services were configured using `systemd` as persistent background services.

This allowed the application to:

- Automatically start after VM reboot
- Restart automatically after unexpected failures
- Continue running independently of terminal sessions
- Maintain continuous clinician access without manual intervention

System logs were monitored using Linux journal tools to validate service health and runtime stability.

6. Reverse Proxy Configuration

To improve accessibility and simplify API access, Nginx was configured as a reverse proxy on the Azure VM. This allowed incoming HTTP requests to be forwarded securely to the backend gateway and ML service without exposing internal ports directly.

The reverse proxy improved:

- Security by hiding internal service ports
- Request forwarding and timeout handling
- Better frontend integration
- Future scalability and maintenance

Nginx configuration files were created and linked to the server environment to support

The deployed speech-based dementia screening module is currently structured to support stable real-time speech recording, ML inference, clinician reporting, and final dementia risk assessment. This deployment architecture improves accessibility, reliability, and maintainability while ensuring secure communication between patients, clinicians, and backend services.

Maintenance

The maintenance phase of the speech-based dementia screening module is essential to ensure continuous reliability, model stability, and system security after deployment. Since the platform supports clinical decision-making, maintaining system accuracy and uptime is critical.

1. Bug Fixes and Issue Monitoring

System logs are regularly reviewed to identify failed uploads, transcription errors, API failures, and session interruptions. Common issues such as FFmpeg conversion failures, timeout errors, and invalid audio uploads are monitored and resolved quickly.This ensures minimal disruption to patient screening sessions and clinician workflows.

2. Performance and Uptime Monitoring

The backend gateway and ML service operate as persistent services, allowing continuous uptime monitoring. CPU usage, memory utilization, API response times,

and model inference speed are regularly checked to maintain performance stability.This is especially important during peak concurrent usage periods.

3. Feature Improvements Based on Feedback

User feedback from patients and clinicians is collected to improve recording usability, task instructions, report clarity, and clinician dashboard presentation.Enhancements such as better microphone guidance, clearer task instructions, and improved report explanations are introduced based on real usage feedback.

4. Model and Dependency Updates

The Wav2Vec2-XLSR-300M model and transcript analysis pipeline are periodically reviewed for performance improvements. Python dependencies such as FastAPI, Librosa, and Google STT libraries are updated when necessary to maintain compatibility and security.This ensures the speech screening module remains clinically reliable and technically up to date.

5. Security and Backup Checks

Routine checks are performed on firewall rules, SSH access policies, API permissions, and environment variables. Configuration backups are maintained to support rapid recovery in case of service failure or accidental misconfiguration.This protects both system integrity and clinician access reliability.

6. CI/CD Support for Continuous Deployment

To simplify deployment updates and reduce manual intervention, GitHub-based CI/CD workflows can be integrated for future deployment improvements. This would allow automatic updates, dependency installation, and service restarts whenever validated code changes are pushed to the main branch.This supports faster development cycles, safer deployments, and smoother long-term maintenance.

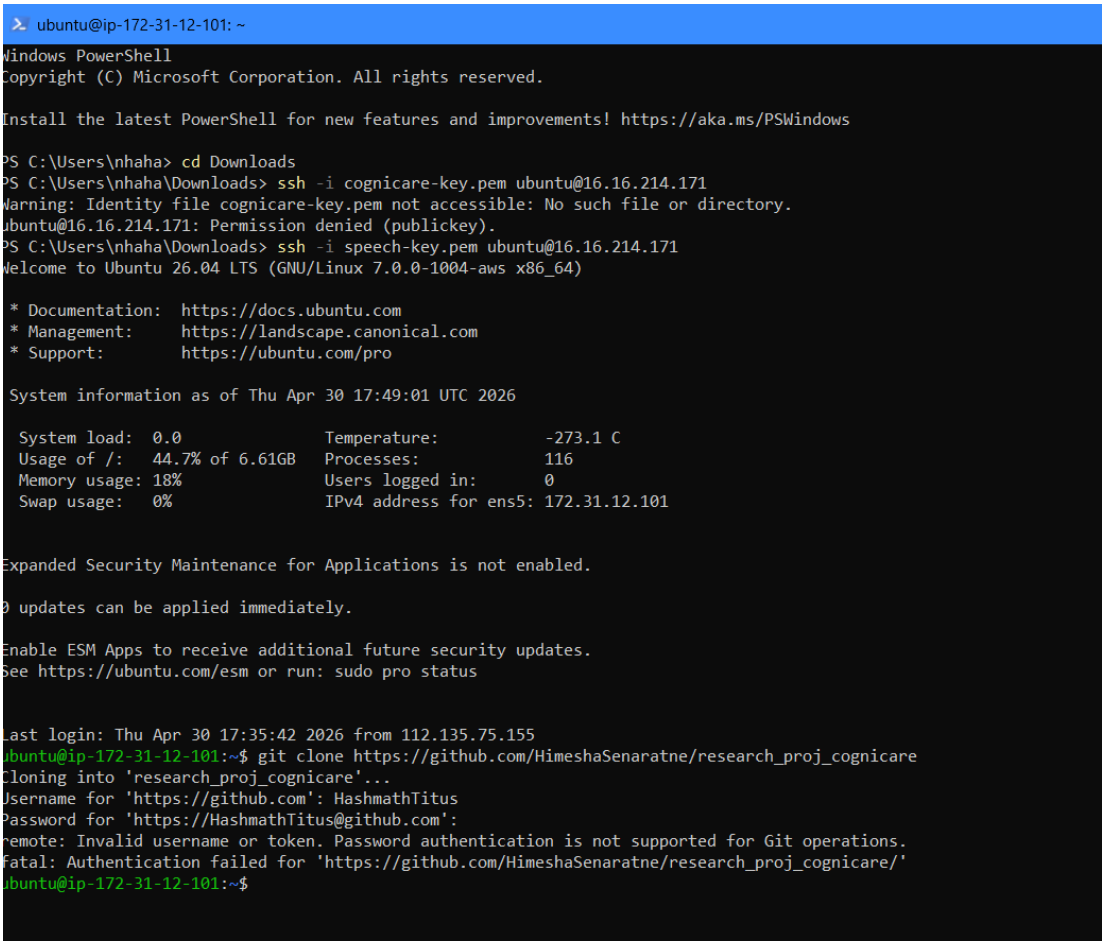


Figure 2.33: Secure SSH Connection to the Ubuntu Cloud Server.

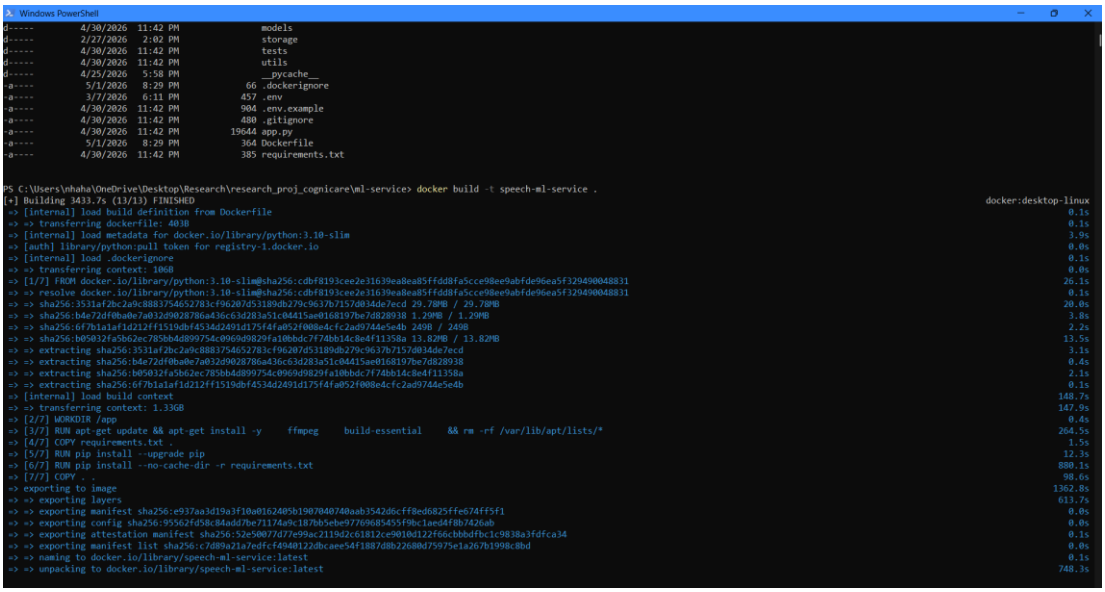


Figure 2.34: Docker Image Build Process for the Speech ML Service.

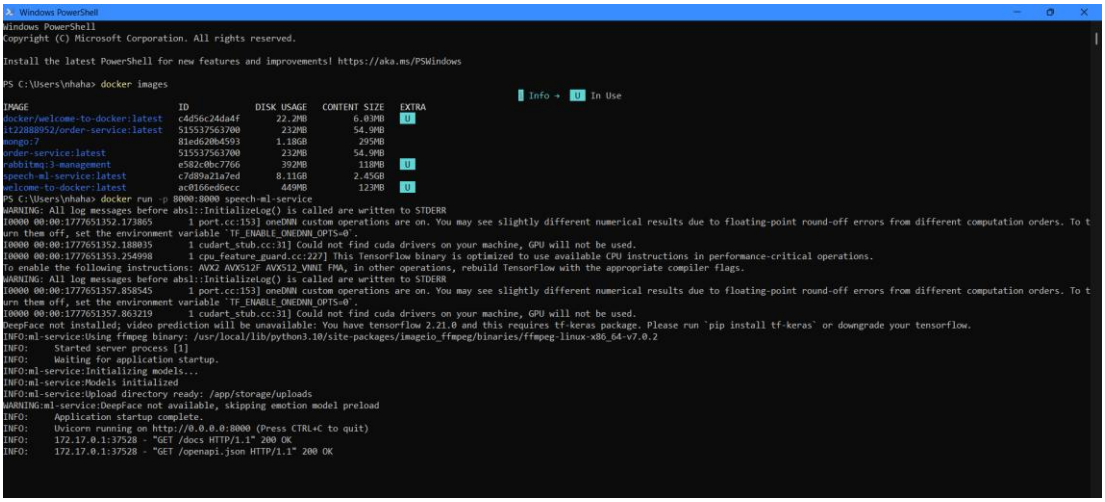


Figure 2.35: Execution of the Speech ML Service Container Using Docker.

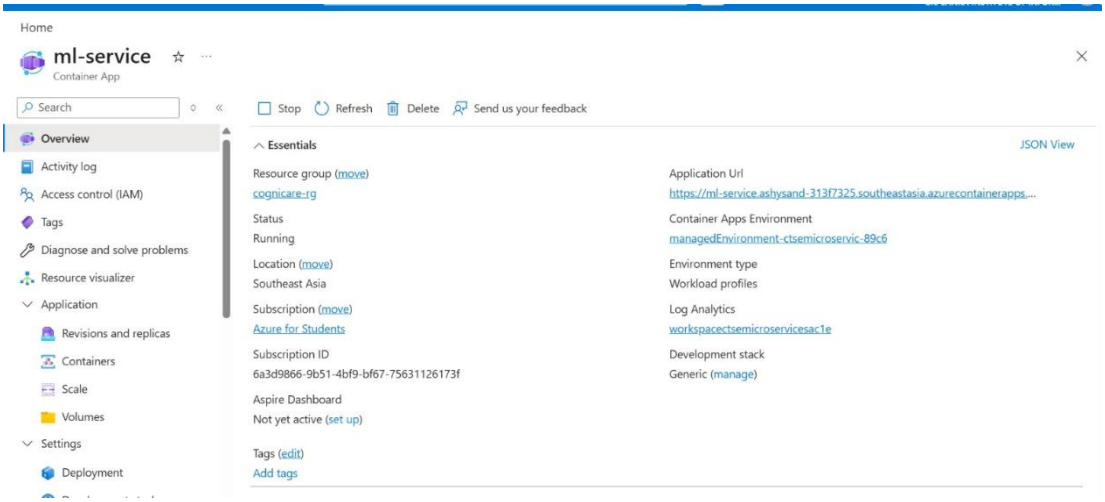


Figure 2.36: Azure Container App Overview for the Deployed ML Service.

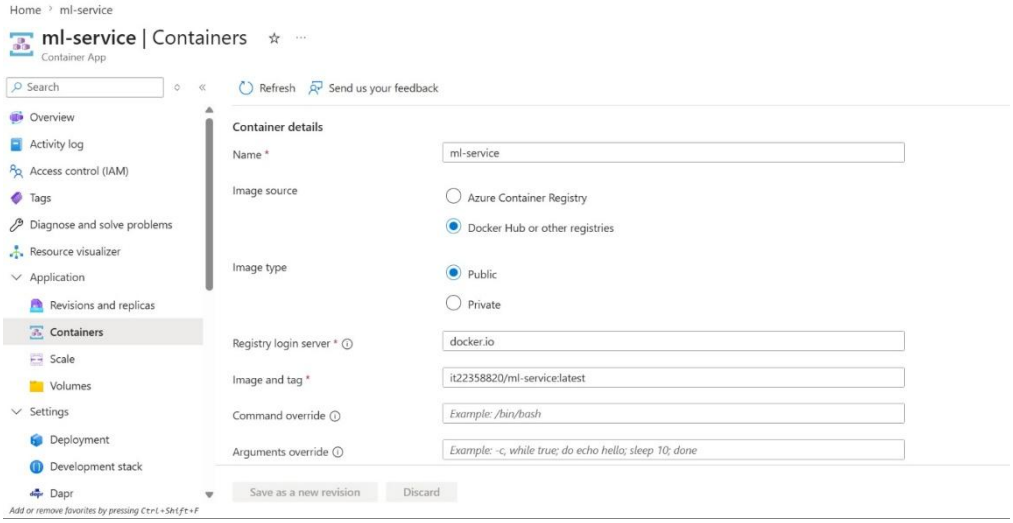


Figure 2.37: Configuration of Docker Image Settings in Azure Container Apps.

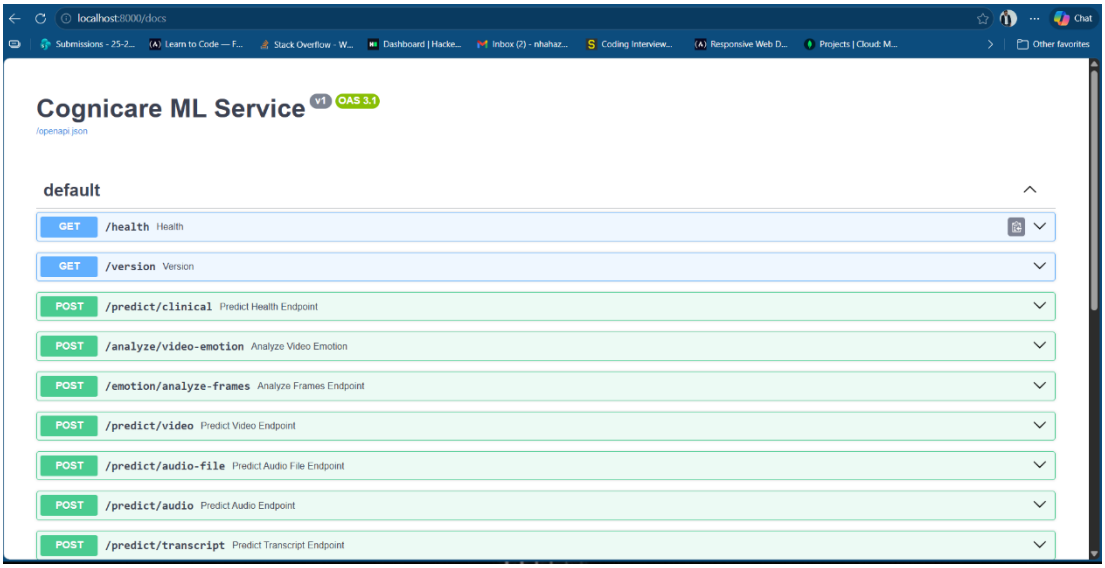


Figure 2.38: Swagger API Documentation Interface of the CogniCare ML Service.

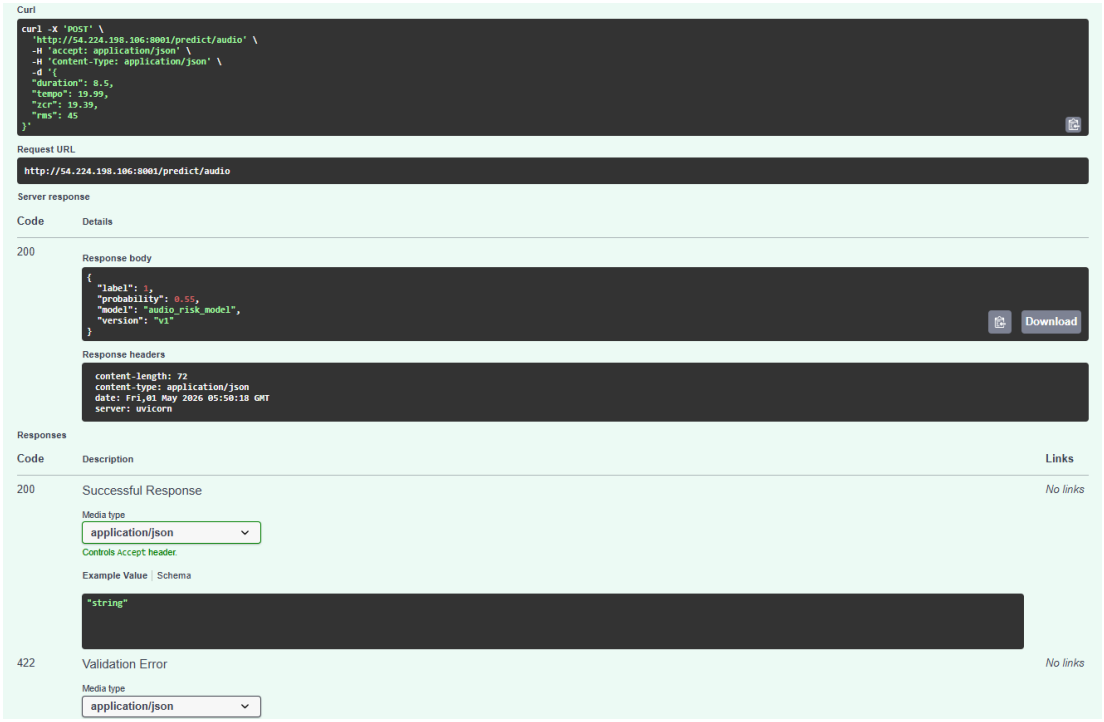


Figure 2.39: Successful Audio-Based Dementia Risk Prediction Response.

2.2 Commercialization

Problem–Solution Fit and Value Proposition

Early dementia detection remains a major challenge in healthcare systems, particularly in low- and middle-income countries where access to neurologists, cognitive specialists, and advanced diagnostic facilities is limited. Traditional dementia diagnosis often depends on lengthy clinical assessments, specialist observation, and expensive imaging procedures such as MRI, which may delay early intervention.

The speech-based dementia screening module of CogniCare addresses this problem by providing a low-cost, accessible, and scalable preliminary screening solution using guided speech-task analysis. By analyzing structured speech responses such as self-introduction, morning recall, and picture description, the system identifies early cognitive decline indicators related to memory, fluency, and narrative coherence.

This provides significant value by enabling early risk identification before severe symptoms emerge, supporting faster referrals, clinician decision-making, and improved patient outcomes. Since the system functions as a screening and decision-support tool rather than a diagnostic replacement, it aligns well with practical healthcare workflows.

Customer Segments and Stakeholders

The primary customers for the speech-based dementia screening module include:

- Hospitals and memory clinics
- Neurology and psychiatry departments
- Private healthcare institutions
- Elderly care centers and rehabilitation facilities
- Community healthcare screening programs
- Telemedicine and digital health platforms

The primary end users include:

- Elderly patients undergoing dementia screening
- Clinicians and neurologists
- General physicians
- Caregivers and family members involved in elderly care

Secondary stakeholders include healthcare administrators, medical researchers, insurance providers, and government healthcare agencies interested in preventive elderly healthcare solutions.

Offering and Product Packaging

The speech screening module can be commercialized as a Software-as-a-Service (SaaS) healthcare platform integrated within the larger CogniCare ecosystem.

CogniCare Speech Screening Package

This package includes:

- Guided speech-task recording interface
- Automatic speech feature extraction
- Wav2Vec2-XLSR-300M based dementia risk prediction
- Transcript analysis and domain-state generation
- Final speech score and clinician justification report
- Clinician dashboard access for medical review

Institutional Package

For hospitals and clinics, an enterprise package can include:

- Multi-patient management
- Clinician dashboards and reporting
- Integration with hospital management systems
- MRI upload support for advanced review
- Administrative controls and role-based access

Research and Academic Package

Universities and medical research institutions can use the system for speech-based cognitive decline research and longitudinal dementia studies.

Pricing Model

A subscription-based pricing model is the most practical commercialization approach.

Monthly Institutional Subscription

Hospitals and clinics can subscribe monthly or annually based on:

- Number of active patients screened
- Number of clinician dashboard users
- MRI module access
- Data storage and reporting requirements

Suggested Pricing Formula

Monthly Fee = Base Platform Fee + (Number of Patients × Per-Patient Screening Fee)

This model ensures affordability for small clinics while allowing scalability for large healthcare institutions.

Freemium pilot programs can also be offered for government hospitals and elderly care centers to encourage adoption.

Go-to-Market Strategy

A pilot-first deployment strategy is most suitable for healthcare adoption.

Phase 1 – Clinical Pilot Testing

Initial deployment can be conducted in selected hospitals, elderly care centers, and neurology clinics to validate usability and collect clinician feedback.

Phase 2 – Institutional Partnerships

Partnerships can be formed with:

- Private hospital networks
- Telemedicine providers
- Elderly care organizations
- Government healthcare programs

Phase 3 – Large-Scale Healthcare Expansion

Following successful pilots, the platform can be expanded nationally through healthcare providers and regional screening programs.

Training sessions, onboarding workshops, and clinician demonstrations can support adoption and trust building.

Competitive Advantage

The speech-based dementia screening module provides several strong competitive advantages.

Low-Cost Screening Alternative

Unlike MRI-based diagnosis, speech screening requires only a microphone-enabled browser and internet access, significantly reducing infrastructure costs.

Remote and Scalable Screening

Patients can complete speech tasks remotely without immediate hospital visits, supporting telemedicine and rural healthcare accessibility.

Multimodal Integration

Unlike standalone speech tools, CogniCare integrates speech analysis with wearable monitoring, cognitive testing, handwriting analysis, and optional MRI review for comprehensive dementia risk assessment.

Clinical Interpretability

The rule-based fusion model provides domain-level explanations rather than black-box predictions, increasing clinician trust and practical usability.

Language Flexibility

The use of Wav2Vec2-XLSR-300M supports multilingual speech representation, improving applicability across diverse patient populations.

Regulatory, Ethics, and Clinical Safety

Since the platform handles healthcare screening data, privacy, ethics, and clinical transparency are essential.

The system follows a clinician-assisted decision-support model, ensuring that final diagnosis remains under medical supervision rather than AI automation.

Patient audio recordings, reports, and clinician reviews must be protected using secure authentication, encrypted storage, and controlled access policies.

The platform must comply with healthcare data governance practices and ethical AI principles, particularly regarding explainability, fairness, and responsible use of predictive results.

Scalability and Future Deployment

The cloud-based architecture using Azure Virtual Machines, FastAPI services, and modular ML deployment enables strong scalability.

Future deployment can support:

- Mobile application integration
- National dementia screening programs
- Telehealth integration
- Multi-language deployment across South Asia
- Government elderly healthcare initiatives

The modular structure also allows future extension into Alzheimer's progression monitoring, cognitive rehabilitation support, and longitudinal patient tracking.

3. RESULTS & DISCUSSION

The speech-based dementia screening component was developed to support early cognitive impairment detection through guided speech-task analysis. This module was evaluated based on classification performance, speech recognition quality, system reliability, and practical clinical relevance within the overall CogniCare platform. The results demonstrate that speech analysis can serve as an effective non-invasive screening mechanism for identifying early dementia risk by analyzing verbal fluency, memory recall, and narrative coherence.

The system was assessed using multiple machine learning approaches including the deep learning-based Wav2Vec2-XLSR-300M model, an acoustic feature-based Random Forest model, and a transcript-based XGBoost model. These models were compared to determine the most reliable approach for dementia prediction using speech data collected from structured screening tasks.

1. Performance Evaluation of Speech Classification Models

To evaluate the predictive performance of the speech screening module, multiple classification models were trained and tested using three guided speech tasks: Self Introduction, Morning Recall, and Picture Description. These tasks were selected because they represent important cognitive domains such as verbal fluency, episodic memory, descriptive reasoning, and sentence organization, which are commonly affected during early dementia progression.

The Wav2Vec2-XLSR-300M model was used as the primary deep learning model for raw speech representation learning. This model processes audio directly and captures complex speech patterns without depending entirely on manually engineered features. In parallel, acoustic features such as duration, tempo, RMS, and Zero Crossing Rate (ZCR) were evaluated using a Random Forest classifier. Additionally, transcript-based linguistic features including word count, sentence count, speech rate, and pause count were processed using an XGBoost model.

The comparative model performance is presented in Table 3.1.

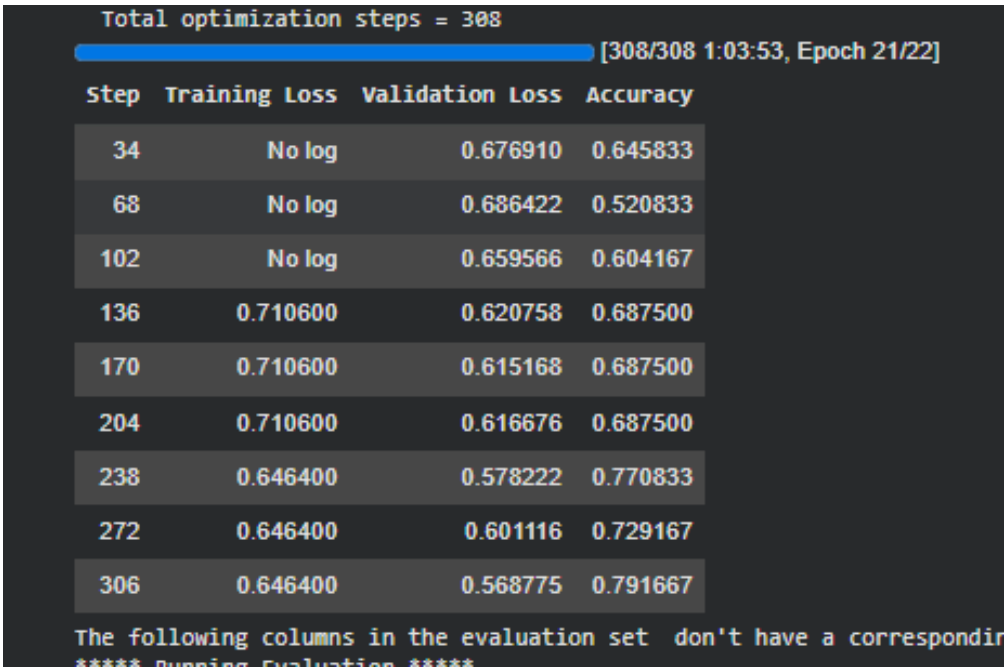
Model	Accuracy	Precision	Recall	F1 Score	AUC
Wav2Vec2-XLSR-300M (Fine-tuned)	**0.79**	**0.80**	**0.79**	**0.79**	**0.79**
Acoustic Model (Random Forest)	0.73	0.74	0.70	0.71	0.85
Transcript Model (XGBoost)	0.76	0.77	0.74	0.75	0.84

Table 3.1: Performance Metrics of Speech Models

The results indicate that the **Wav2Vec2-XLSR-300M (Fine-tuned)** model achieved the most balanced and reliable overall performance for the speech-based dementia screening component. Since this model directly learns contextual speech representations from raw audio, it was more effective in identifying subtle dementia-related speech impairments such as hesitation, reduced fluency, abnormal pauses, and irregular speaking patterns.

Although the transcript-based XGBoost model showed comparatively strong linguistic analysis performance, it depends heavily on speech-to-text quality and may lose important acoustic cues during transcription. Similarly, the acoustic feature-based Random Forest model performed reasonably well but relied only on handcrafted features, which were less effective in capturing deeper cognitive speech abnormalities.

Therefore, the Wav2Vec2-XLSR-300M model was selected as the primary model for the final implementation due to its stronger balance across accuracy, precision, recall, and F1-score, along with its ability to process raw speech more comprehensively without relying entirely on manual feature engineering. Figure 3.1 shows the prediction output generated by the speech analysis pipeline.



Total optimization steps = 308

[308/308 1:03:53, Epoch 21/22]

Step	Training Loss	Validation Loss	Accuracy
34	No log	0.676910	0.645833
68	No log	0.686422	0.520833
102	No log	0.659566	0.604167
136	0.710600	0.620758	0.687500
170	0.710600	0.615168	0.687500
204	0.710600	0.616676	0.687500
238	0.646400	0.578222	0.770833
272	0.646400	0.601116	0.729167
306	0.646400	0.568775	0.791667

The following columns in the evaluation set don't have a correspondir

***** Running Evaluation *****

Figure 3.1: Speech Prediction Result Output

2. Evaluation of Speech-to-Text Quality using WER

To assess transcription quality, Google Speech-to-Text (STT) outputs were compared against manually prepared ground-truth transcripts using Word Error Rate (WER). This evaluation was important because transcript quality directly affects the accuracy of the linguistic dementia classifier and the overall screening reliability.

WER was selected as the primary metric because it measures insertion, deletion, and substitution errors between actual speech and generated transcripts. Lower WER values indicate higher transcription accuracy and better suitability for downstream transcript-based analysis.

The evaluation considered:

- Word-level correctness
- Missing or skipped words
- Incorrect substitutions
- Sentence completeness and coherence

The WER results are summarized in Table 3.2.

Metric	Definition	Result
Average WER	Mean transcription error across all samples	0.18
Best Sample WER	Lowest transcription error observed	0.07
Worst Sample WER	Highest transcription error observed	0.31
STT Reliability	Overall usability for transcript classification	High

Table 3.2: Speech-to-Text Evaluation using WER

The low average WER indicates that the Google STT service produced sufficiently accurate transcripts for dementia screening purposes. Even when minor recognition errors occurred, the transcript classifier remained stable because the extracted linguistic features focused on broader patterns such as sentence structure, pause behavior, and speech rate rather than exact word matching alone.

The system also included fallback behavior where speech processing continued even if the STT output was incomplete or unavailable. This improved robustness and ensured that screening could still proceed using acoustic analysis when transcript quality was limited.

3. Practical Deployment Outcome and Clinical Relevance

After model evaluation and testing, the speech screening system was integrated into the larger CogniCare platform for practical dementia screening. Users complete three guided speech tasks through the frontend interface using microphone recording or uploaded audio files. The system supports formats such as WAV, MP3, M4A, and WebM, with validation for file type, file size, and recording duration.

The backend processes the recordings by storing uploaded files, extracting acoustic features using Librosa, converting non-WAV files using FFmpeg, and generating transcripts through Google Speech-to-Text. These outputs are then passed to the prediction pipeline, where task-level scores are generated and combined into a final dementia risk assessment.

The final scoring mechanism converts prediction probabilities into meaningful cognitive states such as fluency quality, memory-related language performance, and descriptive coherence. This improves interpretability for clinicians and provides stronger support for early medical decision-making.

The speech module provides several practical advantages:

- Non-invasive early dementia screening
- No requirement for expensive MRI scans during initial assessment
- Remote usability through browser-based access
- Faster preliminary risk identification
- Better clinician support before formal diagnosis

The system also includes runtime validation such as invalid file rejection, microphone permission handling, missing upload detection, invalid task checking, and fallback behavior when transcript extraction fails. These features improve reliability and reduce operational failure during real-world usage.

The results confirm that guided speech-task analysis can function as an effective first-level dementia screening tool. The use of Wav2Vec2-XLSR-300M (Fine-tuned) significantly improves predictive reliability while maintaining usability, accessibility, and practical clinical relevance. This makes the component suitable for scalable early dementia screening, particularly in low-resource healthcare environments where specialist neurological assessment may not be immediately available.

Figure 3.2 shows the final completed speech screening session and output workflow.

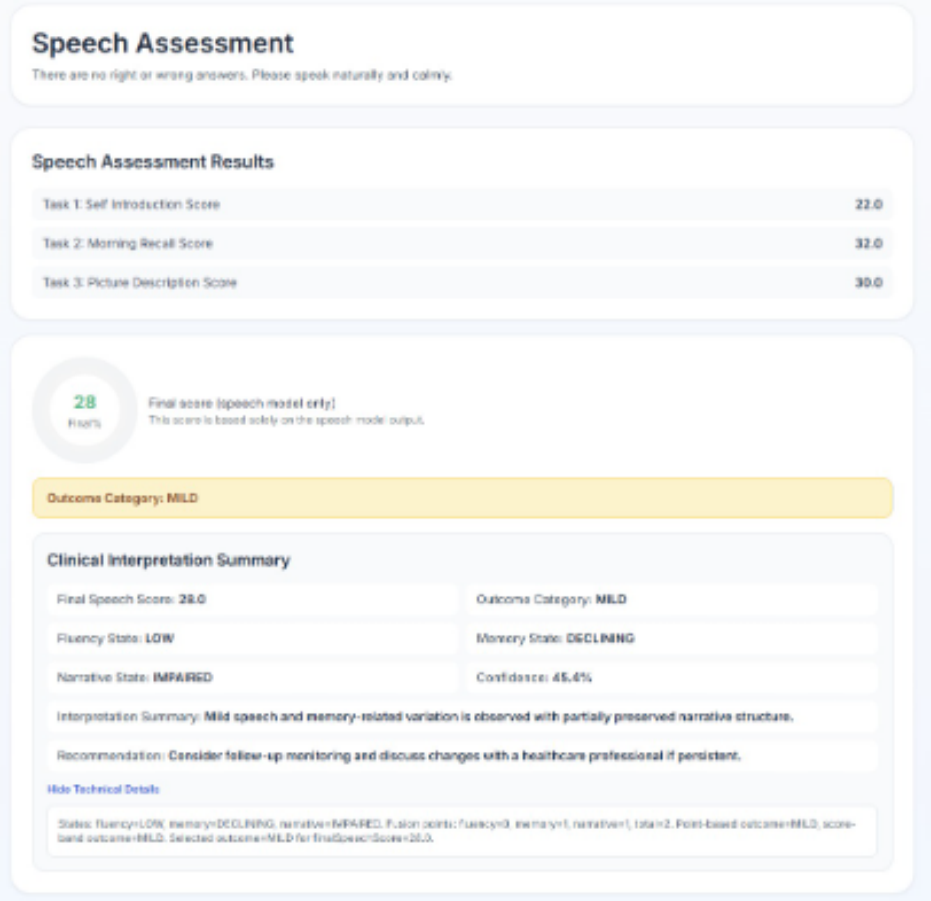


Figure 3.2: Final Speech Screening Session Result

4. FUTURE SCOPE

The current speech-based dementia screening component provides an effective foundation for early cognitive impairment detection using guided speech analysis. Although the system demonstrates strong performance through Wav2Vec2-XLSR-300M-based speech classification and transcript-assisted screening, several future improvements can further enhance its clinical reliability, scalability, and practical adoption.

1. Expansion of Speech Dataset

One of the major future improvements is the expansion of the speech dataset using a larger and more diverse participant population. The current model performs well within the available dataset; however, including speech samples from different age groups, regional accents, education levels, and multiple native languages would improve model generalization and reduce demographic bias.

Special attention can be given to collecting real clinical dementia speech samples from hospitals and memory clinics, which would significantly improve the medical validity of the prediction model.

2. Multilingual Speech Assessment

At present, the primary focus of the speech screening component is English-based speech analysis. Future development can extend the system to support multilingual dementia screening, particularly Sinhala and Tamil, which are highly relevant in the Sri Lankan healthcare context.

This would improve accessibility for elderly patients who are more comfortable speaking in their native language and would increase the practical usability of the platform across rural and underserved communities.

3. Real-Time Speech Monitoring

The current implementation follows a task-based recorded-audio pipeline. In future versions, real-time speech monitoring can be introduced to allow continuous analysis during live patient interaction.

This would enable clinicians to observe speech deterioration dynamically during consultation sessions and improve immediate clinical interpretation without waiting for post-processing.

4. Advanced Clinical Fusion with MRI and Cognitive Assessments

Although the current system supports multimodal screening within the larger CogniCare platform, future work can strengthen tighter integration between speech analysis, MRI findings, handwriting analysis, and cognitive assessment scores such as ACE-III and IADL.

A stronger multimodal fusion engine would improve diagnostic confidence and reduce false positives by combining neurological, behavioral, and linguistic indicators into a single clinical decision framework.

5. Explainable AI for Clinical Decision Support

Future versions can incorporate Explainable AI (XAI) techniques to provide clearer justification for each dementia risk prediction. Instead of presenting only a probability score, the system can explain which speech characteristics—such as long pauses, reduced fluency, repetition, or memory-related language gaps—contributed most to the final decision.

This would improve clinician trust, patient transparency, and practical acceptance in medical environments.

6. Mobile and Telemedicine Integration

The platform can also be extended into mobile-based screening applications for home-based dementia monitoring. Elderly patients or caregivers could perform guided speech assessments remotely using smartphones, reducing hospital dependency for early screening.

This would be especially valuable for telemedicine services and community healthcare programs where specialist access is limited.

7. Continuous Model Improvement through Feedback Learning

Future implementation can include clinician feedback loops where doctors validate screening outcomes and provide correction labels. These verified outcomes can be used for continuous retraining and model improvement, increasing long-term prediction accuracy and adapting the system to real-world clinical variations.

5. CONCLUSION

This research focused on the development of a speech-based dementia screening component within the CogniCare platform to support early identification of cognitive impairment using non-invasive speech analysis. Dementia diagnosis is often delayed due to limited access to specialists, high diagnostic costs, and dependence on traditional clinical assessments such as MRI scans and extensive cognitive testing. This study aimed to address that gap by introducing an accessible, low-cost, and scalable preliminary screening mechanism based on guided speech tasks.

The implemented system uses three structured speech tasks—Self Introduction, Morning Recall, and Picture Description—to evaluate important cognitive domains such as verbal fluency, memory recall, sentence organization, and descriptive reasoning. Audio recordings are processed through a complete pipeline involving frontend audio capture, backend session management, feature extraction using Librosa, speech-to-text conversion using Google Speech-to-Text, and machine learning-based dementia risk prediction.

Among the evaluated models, the Wav2Vec2-XLSR-300M (Fine-tuned) model demonstrated the most balanced and reliable performance, achieving strong results across accuracy, precision, recall, and F1-score. Compared to acoustic-only and transcript-only models, Wav2Vec2 provided deeper contextual understanding of speech behavior by learning directly from raw audio signals. This significantly improved the system's ability to identify subtle dementia-related speech abnormalities such as hesitation, abnormal pauses, reduced fluency, and memory-related language decline.

The integration of Word Error Rate (WER) evaluation further validated the quality of the speech-to-text pipeline, confirming that transcript reliability was sufficient for linguistic feature extraction and downstream classification. The testing strategy included frontend validation, backend API verification, ML service testing, and STT quality assessment, ensuring the robustness and practical usability of the system.

The results confirm that speech analysis can serve as an effective first-level dementia screening tool, particularly in environments where advanced neurological assessment is not immediately available. The system offers major practical advantages including remote accessibility, reduced diagnostic cost, faster preliminary assessment, and better clinical support for early intervention.

In conclusion, the speech-based dementia screening component successfully demonstrates how artificial intelligence and speech processing can be integrated into healthcare for early cognitive impairment detection. The proposed solution contributes both technical and clinical value by providing a scalable, explainable, and user-friendly approach for dementia risk screening. With future improvements such as multilingual support, real-time monitoring, and stronger multimodal clinical fusion, this system has strong potential for real-world healthcare deployment and broader impact in dementia prevention and management.

REFERENCES

[1] A. Baevski et al., "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] A. Babu et al., "XLS-R: Self-supervised Cross-lingual Speech Representation Learning at Scale," *Proc. Interspeech*, 2022.

[3] S. Luz et al., "Alzheimer's Dementia Recognition through Spontaneous Speech: The ADReSS Challenge," *Proc. Interspeech*, 2021.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proc. ACM SIGKDD*, 2016.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.

[6] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," *Proc. Python in Science Conference*, 2015.

[7] Google Cloud, "Speech-to-Text Documentation," Google Cloud Platform, 2024. [Online]. Available: https://cloud.google.com/speech-to-text

[8] Google DeepMind, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/

[9] Microsoft Azure, "Virtual Machines Documentation," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/

[10] Docker Inc., "Docker Documentation," Docker Official Docs, 2024. [Online]. Available: https://docs.docker.com/

[11] World Health Organization, "Dementia," WHO Fact Sheets, 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/dementia

[12] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., Pearson, 2023.

[13] Hugging Face, "facebook/wav2vec2-xls-r-300m Model Card," Hugging Face Model Hub, 2024. [Online]. Available: https://huggingface.co/facebook/wav2vec2-xls-r-300m

[14] J. Devlin et al., "Digital Cognitive Assessment Tools for Early Dementia Screening: A Clinical Review," *Journal of Medical Systems*, vol. 47, no. 3, 2023.

APPENDICES